

IGWT LECTURES

Thesis

Interactive Graphics and Web Technologies

Combined notes from this folder. Generated from Markdown source.

Abstract

Development of a Real-Time Interactive 3D Product Configurator Using Three.js, WebGL, GLSL, and React Three Fiber

Abstract

The rapid evolution of web technologies has transformed modern web applications from static information systems into highly interactive digital experiences. Advances in WebGL, GPU acceleration, and modern JavaScript frameworks have enabled complex three-dimensional (3D) graphics to be rendered directly within web browsers without requiring additional plugins or native software. As a result, industries such as e-commerce, architecture, manufacturing, education, and healthcare increasingly rely on web-based interactive visualization systems to improve user engagement and support decision-making.

This thesis presents the design and implementation of a real-time interactive 3D product configurator developed using Three.js, WebGL, GLSL, and React Three Fiber. The primary objective of the research is to demonstrate how modern web graphics technologies can be integrated into a scalable and maintainable architecture capable of delivering high-quality visual experiences while maintaining real-time performance across desktop and mobile platforms.

The proposed system combines physically based rendering (PBR), custom GLSL shaders, optimized asset loading using the glTF format, efficient texture management, and component-based application architecture provided by React Three Fiber. Performance optimization techniques, including frustum culling, level-of-detail management, texture compression, asynchronous asset loading, and GPU resource optimization, are incorporated to ensure smooth rendering under varying hardware conditions. The application further supports interactive camera controls, material customization, lighting adjustment, and real-time product configuration through an intuitive graphical user interface.

To evaluate the effectiveness of the proposed framework, the implementation is analyzed using quantitative performance metrics, including frame rate (FPS), GPU memory consumption, scene loading time, rendering latency, and user interaction responsiveness. The evaluation also considers scalability by measuring system performance across scenes with different geometric complexities and texture resolutions. The results demonstrate that the proposed architecture is capable of maintaining interactive frame rates while providing visually realistic rendering suitable for commercial web applications.

In addition to implementation and performance evaluation, this thesis discusses the theoretical foundations of computer graphics, the modern WebGL rendering pipeline, shader programming, physically based rendering, and the integration of React-based application architecture with GPU-driven rendering workflows. The study illustrates how these technologies collectively contribute to the development of maintainable, extensible, and high-performance interactive graphics applications for the web.

The contributions of this thesis include a modular software architecture for web-based 3D visualization, practical implementation guidelines for integrating Three.js with React Three Fiber, an evaluation of rendering optimization techniques, and recommendations for future adoption of emerging standards such as WebGPU. The findings demonstrate that contemporary web technologies provide a practical platform for

developing sophisticated interactive graphics applications that approach the visual quality traditionally associated with native desktop software.

Keywords: Interactive Graphics, Computer Graphics, Three.js, WebGL, GLSL, React Three Fiber, Real-Time Rendering, Physically Based Rendering, Web Development, GPU Programming, Product Configurator.

Chapter 1

Introduction

1.1 Background

The rapid advancement of web technologies over the past decade has fundamentally changed the capabilities of modern web applications. What were once static, document-oriented websites have evolved into sophisticated software platforms capable of delivering immersive, interactive, and visually compelling user experiences. The widespread adoption of HTML5, CSS3, JavaScript, and GPU-accelerated graphics technologies has enabled browsers to perform tasks that previously required native desktop applications.

Among these technological advancements, WebGL has emerged as a major milestone in browser-based graphics. Developed as a JavaScript interface to the OpenGL ES graphics library, WebGL allows direct access to GPU hardware from within standard web browsers. By eliminating the need for browser plug-ins, WebGL provides a cross-platform solution for developing real-time three-dimensional (3D) applications that can execute efficiently on desktop computers, laptops, tablets, and mobile devices.

Building upon WebGL, high-level graphics libraries such as Three.js have significantly reduced the complexity associated with low-level graphics programming. Three.js abstracts many aspects of the rendering pipeline while preserving access to advanced graphics features including programmable shaders, physically based rendering (PBR), shadow mapping, post-processing effects, and animation systems. As a result, web developers can focus on designing interactive experiences instead of managing low-level GPU operations.

The emergence of React Three Fiber has further transformed web graphics development by integrating Three.js into the React component ecosystem. This integration enables developers to combine declarative user interface development with real-time graphics rendering, resulting in software architectures that are modular, reusable, and easier to maintain. Such an approach is particularly suitable for large-scale commercial applications where user interface components and 3D visualization must operate seamlessly together.

These technologies have accelerated the adoption of interactive 3D visualization across numerous industries. Online retailers now allow customers to inspect and customize products in real time before purchase. Architects present virtual walkthroughs of buildings directly within web browsers. Educational institutions use interactive simulations to explain scientific concepts, while healthcare organizations increasingly employ three-dimensional visualization for anatomy education and medical planning. These examples demonstrate that real-time graphics are no longer limited to the entertainment industry but have become an essential component of modern digital services.

Despite these advances, developing high-performance browser-based graphics applications remains a challenging engineering task. Developers must balance rendering quality, responsiveness, maintainability, and compatibility across a wide range of hardware capabilities. Achieving smooth performance requires careful optimization of geometry, textures, shaders, asset loading, and GPU memory usage. Consequently, understanding both theoretical computer graphics concepts and practical software engineering techniques has become increasingly important.

This thesis investigates how contemporary web graphics technologies can be integrated to build a professional-quality interactive product configurator that demonstrates realistic rendering, responsive interaction, and scalable software architecture.

1.2 Motivation

Modern consumers increasingly expect digital experiences that closely resemble physical interaction with products. Static images and videos often fail to communicate important product characteristics such as shape, material, scale, texture, and customization options. Consequently, businesses seek technologies that allow customers to interact with products directly before making purchasing decisions.

Traditional native applications can deliver highly realistic graphics but require installation, platform-specific development, and ongoing maintenance. Browser-based applications, in contrast, provide instant accessibility, simplified deployment, and broad compatibility. Advances in WebGL and modern JavaScript frameworks have narrowed the performance gap between browser applications and native software, making web-based interactive visualization an attractive solution.

However, creating these applications requires expertise in computer graphics, GPU programming, user interface engineering, software architecture, and performance optimization. Educational resources often teach these topics independently, making it difficult for students to understand how they combine in real-world projects.

This thesis is motivated by the need for a comprehensive implementation that integrates modern graphics technologies into a cohesive software system while maintaining readability, scalability, and performance.

1.3 Problem Statement

Although WebGL and Three.js provide powerful capabilities for browser-based rendering, many existing implementations focus on isolated demonstrations rather than complete software systems. Common limitations include:

- Inefficient rendering pipelines resulting in reduced frame rates.
- Poor software architecture that complicates maintenance.
- Excessive GPU memory consumption.
- Long loading times caused by unoptimized assets.
- Limited extensibility for future functionality.
- Inadequate integration between 3D graphics and modern user interface frameworks.

These challenges become increasingly significant as application complexity grows. Therefore, there is a need for a structured development approach that integrates modern graphics technologies with established software engineering principles.

1.4 Research Objectives

The primary objective of this research is to design, implement, and evaluate a real-time browser-based 3D product configurator using contemporary web graphics technologies.

The specific objectives are:

1. Design a modular software architecture for interactive graphics applications.
 2. Implement physically based rendering using Three.js.
 3. Integrate React Three Fiber with a modern React application.
 4. Develop custom GLSL shaders for enhanced visual quality.
 5. Optimize rendering performance using GPU-aware techniques.
 6. Evaluate rendering quality and application performance across representative scenes.
-

1.5 Research Questions

This thesis seeks to answer the following research questions:

RQ1.

How can Three.js and React Three Fiber be integrated to produce scalable interactive graphics applications?

RQ2.

Which rendering optimization techniques provide the greatest performance improvements in browser-based applications?

RQ3.

How do custom GLSL shaders improve visual realism compared with standard materials?

RQ4.

Can browser-based graphics applications achieve interactive performance suitable for commercial product visualization?

1.6 Scope of the Study

This study focuses on browser-based real-time graphics applications developed using:

- JavaScript
- React
- React Three Fiber
- Three.js
- WebGL
- GLSL

The research does not address:

- Native desktop graphics APIs
- Mobile application development

- Unity or Unreal Engine
- Distributed rendering
- Ray tracing using dedicated hardware
- Large-scale multiplayer networking

Performance evaluation is limited to a representative range of consumer hardware and browsers.

1.7 Contributions

The primary contributions of this thesis are:

- A modular architecture for browser-based interactive graphics.
 - A complete implementation of a real-time 3D product configurator.
 - Practical guidelines for integrating React Three Fiber with Three.js.
 - An evaluation of rendering optimization techniques.
 - Recommendations for future migration toward WebGPU.
 - A reusable educational example suitable for computer graphics and web technology courses.
-

1.8 Thesis Organization

The remainder of this thesis is organized as follows.

Chapter 2 reviews the literature related to computer graphics, browser rendering, WebGL, Three.js, physically based rendering, shader programming, and interactive visualization.

Chapter 3 introduces the theoretical foundations of computer graphics, including coordinate systems, transformations, cameras, lighting models, rasterization, texture mapping, and the modern GPU rendering pipeline.

Chapter 4 presents the overall system architecture and design of the proposed application, describing software components, rendering workflows, asset management, and user interaction.

Chapter 5 describes the implementation in detail, including React Three Fiber integration, custom GLSL shaders, material systems, animation, lighting, optimization strategies, and user interface development.

Chapter 6 evaluates the application through performance measurements, rendering benchmarks, and qualitative assessment of visual quality.

Chapter 7 discusses the implications of the results, limitations of the implementation, and opportunities for future work.

Chapter 8 concludes the thesis by summarizing its contributions and outlining future directions, including the adoption of WebGPU and emerging real-time rendering techniques.

Chapter 2

Literature Review

2.1 Introduction

Computer graphics has evolved from a specialized research discipline into a foundational technology for scientific visualization, engineering, entertainment, e-commerce, education, and healthcare. Simultaneously, advances in web standards and browser technologies have enabled sophisticated real-time graphics applications to execute directly within web browsers without proprietary plugins. Modern web graphics combine computer graphics theory, GPU programming, software engineering, and user interface design into a unified development ecosystem.

This chapter reviews the evolution of computer graphics, WebGL, Three.js, React Three Fiber, shader programming, physically based rendering, asset management, performance optimization, WebGPU, and existing browser-based product configurators. The chapter concludes by identifying the research gap addressed by this thesis.

2.2 Evolution of Computer Graphics

Computer graphics originated in the 1960s with vector displays used for engineering and military applications. Early systems focused on geometric precision rather than realism because hardware resources were extremely limited. During the 1970s and 1980s, raster graphics became dominant due to improvements in display technology and graphics hardware.

The introduction of dedicated Graphics Processing Units (GPUs) transformed graphics computing. Instead of relying solely on the central processing unit (CPU), GPUs were designed to perform thousands of mathematical operations simultaneously, making real-time rendering feasible. Modern GPUs execute programmable shader programs that control geometry processing, lighting calculations, texture sampling, and pixel generation.

The transition from fixed-function graphics pipelines to programmable pipelines fundamentally changed graphics programming. Earlier graphics APIs provided predefined lighting and transformation functions. Modern APIs require developers to implement these stages through shader programs, providing greater flexibility and enabling advanced rendering techniques such as physically based rendering (PBR), procedural texturing, deferred rendering, and post-processing effects. HWS Mathematics and Computer Science+1

Today, computer graphics extends beyond entertainment into fields such as digital twins, autonomous driving simulation, scientific visualization, medical imaging, architecture, manufacturing, and virtual reality.

2.3 Evolution of Web Technologies for 3D Graphics

The early World Wide Web was designed primarily for delivering hypertext documents. Interactive multimedia applications required proprietary technologies such as Adobe Flash, Java Applets, or browser plug-ins. These approaches suffered from portability, security, and maintenance issues.

The introduction of HTML5, JavaScript improvements, and the `<canvas>` element provided a standardized platform for browser graphics. Subsequently, WebGL enabled browsers to communicate directly with GPU hardware through an API derived from OpenGL ES, eliminating the need for external plugins while maintaining cross-platform compatibility. HWS Mathematics and Computer Science+1

Modern browsers now support:

- Hardware-accelerated rendering
- High-performance JavaScript engines
- WebAssembly
- WebXR
- GPU-accelerated animations
- Progressive Web Applications

These capabilities have transformed the browser into a powerful software platform capable of supporting professional visualization applications.

2.4 WebGL

WebGL is a JavaScript API that exposes GPU functionality within web browsers. Based on OpenGL ES 2.0, WebGL provides low-level access to graphics hardware while maintaining browser security.

Unlike traditional OpenGL, WebGL does not provide built-in transformation or lighting functions. Instead, developers must implement these operations through programmable shaders. Consequently, WebGL offers greater flexibility but requires a deeper understanding of graphics programming concepts such as buffers, matrices, vertex attributes, uniforms, and rendering pipelines. HWS Mathematics and Computer Science+1

A typical WebGL application consists of:

- Vertex buffers
- Index buffers
- Vertex shaders
- Fragment shaders
- Texture resources
- Framebuffers
- Rendering loop

While this architecture provides complete control over rendering, it also increases development complexity. For this reason, higher-level libraries have become the preferred approach for most commercial web applications.

2.5 Three.js

Three.js is the most widely adopted high-level JavaScript library for browser-based 3D graphics. Built on top of WebGL, it abstracts many low-level graphics operations while exposing advanced rendering capabilities when needed. Three.js+1

Three.js provides:

- Scene graph management
- Camera systems
- Geometry generation
- Material systems
- Lighting models
- Animation framework
- Shadow mapping
- Environment mapping
- Post-processing
- glTF support
- WebXR integration

Its object-oriented design enables developers to construct complex 3D scenes using reusable components rather than manipulating GPU buffers directly.

Despite these advantages, developers still require knowledge of graphics principles because rendering quality ultimately depends on scene organization, lighting, material selection, and performance optimization.

2.6 React Three Fiber

React Three Fiber (R3F) integrates Three.js into the React ecosystem using a declarative programming model.

Instead of imperatively creating objects, developers describe the scene as React components. React manages the lifecycle of these components while R3F synchronizes them with the Three.js scene graph.

Benefits include:

- Component reusability
- Improved maintainability
- Integration with React state management
- Easier UI synchronization
- Simplified animation workflows

This architecture is particularly suitable for commercial applications where graphical interfaces and business logic must coexist.

However, React introduces an abstraction layer that requires careful optimization to avoid unnecessary re-rendering and state synchronization overhead.

2.7 Shader Programming

Shaders are small programs executed directly on the GPU.

Modern rendering generally employs two programmable stages:

Vertex Shader

Responsible for:

- Vertex transformation
- Skinning
- Morph targets
- Normal calculation
- Coordinate conversion

Fragment Shader

Responsible for:

- Color computation
- Lighting
- Texture sampling
- Transparency
- Procedural effects
- Post-processing

Shader programming enables developers to implement visual effects that cannot be achieved using predefined materials.

Applications include:

- Water
- Fire
- Clouds
- Terrain
- Dissolve effects
- Glass
- Procedural animation

Because shaders execute massively in parallel, efficient shader design is critical for achieving high frame rates.

2.8 Physically Based Rendering

Physically Based Rendering (PBR) has become the industry standard for realistic rendering.

Unlike traditional Phong shading, PBR models the interaction between light and surfaces using physically motivated parameters such as:

- Base Color
- Metalness
- Roughness
- Ambient Occlusion
- Normal Maps
- Emissive Maps

Advantages include:

- Consistent lighting
- Material realism
- Predictable appearance
- Compatibility across rendering engines

Three.js implements PBR through materials such as `MeshStandardMaterial` and `MeshPhysicalMaterial`, enabling developers to achieve realistic visuals without implementing complete lighting models from scratch. Wikipedia+1

2.9 glTF Asset Format

Efficient asset management is essential for browser-based graphics.

The Graphics Language Transmission Format (glTF) has become the preferred standard for transmitting 3D assets because it is optimized for runtime performance.

Compared with OBJ or FBX, glTF supports:

- PBR materials
- Animations
- Cameras
- Lights
- Skeletons
- Compression
- Binary storage

Three.js includes native loaders for glTF, reducing loading times and simplifying integration into web applications.

2.10 Performance Optimization

Maintaining interactive frame rates requires careful optimization.

Common techniques include:

Frustum Culling

Objects outside the camera view are not rendered.

Level of Detail (LOD)

Different geometric models are selected according to viewing distance.

Instanced Rendering

Multiple identical objects are rendered using a single draw call.

Texture Compression

GPU-friendly texture formats reduce memory consumption.

Lazy Loading

Assets are loaded only when required.

Shader Optimization

Reducing expensive calculations improves rendering performance.

These techniques collectively improve scalability without significantly reducing visual quality.

2.11 WebGPU

Although WebGL remains the dominant browser graphics API, WebGPU represents the next generation of browser graphics programming.

Compared with WebGL, WebGPU provides:

- Lower CPU overhead
- Compute shaders
- Modern GPU pipeline management
- Better multithreading support
- Improved resource control

WebGPU is inspired by modern graphics APIs such as Vulkan, Metal, and Direct3D 12. As browser support matures, it is expected to become an important platform for real-time graphics and general-purpose GPU computing. HWS Mathematics and Computer Science+1

2.12 Existing Web-Based Product Configurators

Interactive product configurators have become common in industries including automotive manufacturing, furniture, consumer electronics, fashion, and architecture.

Typical functionality includes:

- 360° object rotation
- Material selection
- Color customization
- Accessory configuration

- Lighting adjustment
- Animation
- Real-time pricing
- AR preview

Despite their popularity, many existing systems remain proprietary, making it difficult for researchers and educators to study their architecture or performance characteristics. Public demonstrations often emphasize visual appearance but provide little information regarding rendering pipelines, optimization strategies, or software engineering practices.

Consequently, there remains educational value in documenting a complete implementation that integrates graphics theory with modern web development.

2.13 Research Gap

The literature demonstrates significant progress in browser-based graphics technologies. However, several gaps remain.

First, many publications focus on individual technologies such as WebGL, shader programming, or React without explaining how they integrate into a cohesive application architecture.

Second, tutorials frequently emphasize visual demonstrations while providing limited discussion of scalability, maintainability, or software engineering practices.

Third, comparative evaluations of optimization strategies in browser-based graphics applications remain limited, particularly from an educational perspective.

Finally, there is a shortage of comprehensive reference implementations that combine computer graphics theory, modern JavaScript frameworks, GPU programming, and interactive product visualization within a single documented project.

This thesis addresses these gaps by presenting a modular implementation of a browser-based real-time 3D product configurator, accompanied by architectural analysis and performance evaluation.

2.14 Chapter Summary

This chapter reviewed the historical development of computer graphics and the technologies that underpin modern browser-based rendering. The discussion covered WebGL, Three.js, React Three Fiber, shader programming, physically based rendering, the glTF asset format, performance optimization techniques, WebGPU, and current approaches to interactive product visualization.

The review demonstrates that modern web technologies are sufficiently mature to support sophisticated real-time graphics applications. However, effectively integrating these technologies requires careful architectural design, performance optimization, and a solid understanding of computer graphics principles. The following chapter therefore introduces the theoretical foundations that support the implementation presented later in this thesis.

Chapter 3

Computer Graphics Fundamentals

Part 1: Mathematical Foundations

3.1 Introduction

Computer graphics is fundamentally built upon mathematics. Every object rendered on a computer screen is represented numerically and manipulated using mathematical operations. Whether rotating a camera, animating a character, simulating physics, or rendering realistic lighting, graphics systems rely heavily on vectors, matrices, coordinate systems, and linear algebra.

Modern graphics APIs such as OpenGL, WebGL, Vulkan, Direct3D, and WebGPU all employ similar mathematical foundations. Even high-level libraries such as Three.js internally perform matrix multiplications, coordinate transformations, and vector calculations before issuing rendering commands to the GPU.

Therefore, understanding the mathematical principles behind computer graphics is essential for developing efficient and visually accurate real-time applications.

This chapter introduces the mathematical concepts that serve as the theoretical foundation for the implementation described later in this thesis.

3.2 Mathematical Representation of Graphics

A computer has no understanding of concepts such as "cube," "camera," or "light." Instead, every graphical object is represented using numerical data.

For example, a cube is represented by:

- Vertex positions
- Surface normals
- Texture coordinates
- Triangle indices
- Material properties

A single vertex may be stored as

$P = (x, y, z)$

where

- x represents horizontal position,
- y represents vertical position,

- z represents depth.

A cube therefore consists of multiple vertices connected into triangles.

During rendering, the GPU performs millions of mathematical calculations every second to transform these vertices from their local positions into pixels displayed on the monitor. Modern graphics pipelines rely on matrix transformations and coordinate-space conversions to accomplish this efficiently. VTK Book+1

3.3 Vectors

3.3.1 Definition

A vector represents both **magnitude** and **direction**.

Unlike a point, which specifies a location, a vector describes movement or displacement between locations. Understanding this distinction is important because graphics systems treat points and vectors differently during transformations. UW Graphics+1

A two-dimensional vector is written as

$$\mathbf{v} = (x, y)$$

while a three-dimensional vector is written as

$$\mathbf{v} = (x, y, z)$$

Example

$$\mathbf{v} = (3, 4, 2)$$

This indicates movement

- 3 units along the X-axis
- 4 units along the Y-axis
- 2 units along the Z-axis

Vectors are used extensively in graphics for:

- Object movement
 - Camera direction
 - Surface normals
 - Light direction
 - Physics simulations
 - Animation
 - Reflection calculations
-

3.3.2 Vector Length

The magnitude (or length) of a vector is

$$|\mathbf{v}| = \sqrt{x^2 + y^2 + z^2}$$

Example

$$v = (3, 4, 0)$$

Then

$$|v| = \sqrt{3^2 + 4^2} = 5$$

The length determines how far the vector extends through space.

3.3.3 Unit Vector

Many graphics calculations require vectors that represent only direction.

These are called **unit vectors**.

A unit vector has

$$|v| = 1$$

Normalization is computed by dividing each component by the vector length.

$$v^{\wedge} = |v|^{-1} v$$

Example

Original

$$(3, 4, 0)$$

Length

$$5$$

Normalized

$$(0.6, 0.8, 0)$$

Lighting calculations almost always use normalized vectors because illumination depends on direction rather than distance. Chortle+1

3.3.4 Vector Addition

Vectors can be added.

$$a + b = (a_x + b_x, a_y + b_y, a_z + b_z)$$

Example

$$(2, 1, 5) + (4, 2, 1) = (6, 3, 6)$$

Applications include:

- Character movement
 - Camera motion
 - Physics
 - Animation blending
-

3.3.5 Scalar Multiplication

A scalar changes vector magnitude.

$$kv = (kx, ky, kz)$$

Example

$$2(3, 2, 1) = (6, 4, 2)$$

Scaling vectors is commonly used for:

- Velocity
 - Camera zoom
 - Object scaling
 - Physics simulations
-

3.3.6 Dot Product

The dot product measures the similarity between two vectors.

$$a \cdot b = axbx + ayby + azbz$$

Alternatively,

$$a \cdot b = |a||b|\cos\theta$$

where

θ

is the angle between the vectors.

Interpretation

- Positive $\rightarrow$ same direction
- Zero $\rightarrow$ perpendicular
- Negative $\rightarrow$ opposite directions

Applications include:

- Diffuse lighting
- Back-face culling
- Surface orientation
- Visibility tests

For example, Lambertian diffuse shading computes light intensity using the dot product between the surface normal and the light direction. Oberlin College Computer Science+1

3.3.7 Cross Product

The cross product exists only in three dimensions.

$$a \times b = c$$

Its result

- is perpendicular to both vectors,
- follows the right-hand rule,
- has magnitude $|a||b|\sin\theta$

Applications include:

- Computing surface normals
- Camera orientation
- Physics
- Reflection
- Mesh generation

Nearly every 3D rendering engine uses cross products to calculate normal vectors required for lighting.

3.4 Matrices

3.4.1 Introduction

Matrices are rectangular arrays of numbers used to transform vectors and points.

Unlike vectors, matrices can simultaneously perform:

- Translation
- Rotation
- Scaling
- Projection
- Reflection
- Shearing

Graphics hardware is optimized for matrix multiplication, making matrices the standard representation for geometric transformations. HWS Mathematics and Computer Science+1

3.4.2 Matrix Representation

A matrix is commonly written as

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

Most graphics APIs use **4 × 4 matrices** because they support homogeneous coordinates, allowing translation, rotation, scaling, and projection to be represented within a single transformation framework. Flylib+1

3.4.3 Matrix Multiplication

Matrix multiplication combines transformations.

Suppose

Rotation

↓

Scaling

↓

Translation

Instead of executing each separately, they are combined into one transformation matrix.

This greatly improves GPU efficiency.

One important property is that matrix multiplication is **not commutative**:

$$AB \neq BA$$

Therefore, the order of transformations matters.

For example,

Scale

↓

Rotate

↓

Translate

produces a different result from

Translate

↓

Rotate

↓

Scale

Understanding transformation order is one of the most important skills in graphics programming.

3.5 Coordinate Systems

Coordinate systems define where objects exist and how they are interpreted by the rendering pipeline.

A point's numerical coordinates have meaning only with respect to a chosen coordinate frame. Computer graphics therefore uses multiple coordinate systems, each serving a specific purpose in the rendering process. ScienceDirect+ 1

Modern graphics systems generally employ several coordinate spaces.

3.5.1 Object (Model) Space

Every object begins in its own local coordinate system.

Example

A chair

Origin

$(0,0,0)$

may be positioned at its geometric center.

This coordinate system is independent of the rest of the scene.

3.5.2 World Space

Objects are transformed into a shared coordinate system known as world space.

Example

Chair

$(5,0,-2)$

Table

$(-3,0,8)$

Lamp

$(1,4,-5)$

All scene objects coexist in this global reference frame.

3.5.3 View (Camera) Space

The camera observes the scene.

The rendering engine transforms world coordinates into coordinates relative to the camera position and orientation.

From the camera's perspective, the camera itself is always located at the origin while the world moves relative to it. VTK Book+1

3.5.4 Clip Space

Perspective projection converts the visible scene into a normalized viewing volume.

Vertices outside this volume are clipped before rasterization.

3.5.5 Screen Space

Finally,

Clip Space

↓

Viewport Transformation

↓

Screen Space

Coordinates become pixel positions displayed on the monitor.

The complete transformation pipeline is therefore:

This sequence is fundamental to all modern graphics APIs, including WebGL and Three.js. VTK Book+1

3.6 Chapter Summary (Part 1)

This first part of Chapter 3 introduced the mathematical foundations required for computer graphics. It distinguished between points and vectors, explained vector operations such as normalization, dot products, and cross products, and demonstrated the role of matrices in geometric transformations. The chapter also introduced the hierarchy of coordinate systems—object, world, view, clip, and screen space—that underpins the modern rendering pipeline.

Part 2: Transformations, Camera Systems, and Projection

3.7 Geometric Transformations

After defining an object in its local coordinate system, it must be positioned within the virtual scene. This process is achieved through **geometric transformations**, which modify the position, orientation, and size of graphical objects while preserving their geometric relationships.

Geometric transformations form one of the most fundamental operations in computer graphics. Every visible object in a three-dimensional scene undergoes a sequence of transformations before it is displayed on the screen. Modern graphics APIs such as WebGL, OpenGL, Vulkan, and Direct3D implement these operations primarily through matrix multiplication, allowing the GPU to process millions of vertices efficiently in parallel.

The three basic transformations are:

- Translation
- Rotation
- Scaling

More advanced transformations include reflection, shearing, and projection, although these are used less frequently in interactive graphics applications.

3.7.1 Translation

Translation moves an object from one location to another without changing its orientation or dimensions.

If an object is translated by the vector

$$T = (t_x, t_y, t_z)$$

every vertex is displaced by the same amount.

For a point

$$P = (x, y, z)$$

the translated position becomes

$$P' = (x + t_x, y + t_y, z + t_z)$$

Translation is used extensively in applications such as:

- Character movement
- Camera movement
- Object placement
- Animation
- Physics simulations

In Three.js, translation can be applied directly through an object's position properties.

JavaScript

Internally, the engine converts this operation into a transformation matrix before sending it to the GPU.

3.7.2 Scaling

Scaling changes the size of an object.

Uniform scaling enlarges or reduces an object equally along all axes.

Non-uniform scaling allows each axis to be modified independently.

Given

$$S = (s_x, s_y, s_z)$$

the transformed point becomes

$$P' = (x s_x, y s_y, z s_z)$$

Applications include:

- Object resizing
- Character growth
- Interface animation
- Level-of-detail adjustments

Improper scaling can distort object proportions and alter lighting calculations if surface normals are not updated correctly.

3.7.3 Rotation

Rotation changes an object's orientation around one or more axes.

Three-dimensional graphics generally support rotation about:

- X-axis
- Y-axis
- Z-axis

Rotation matrices preserve distances between vertices while changing their orientation.

For example,

rotation around the Y-axis is represented mathematically as

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

Rotations are fundamental for:

- Camera control
- Character animation
- Vehicle steering
- Planetary motion
- Interactive product visualization

Three.js stores rotations internally using Euler angles but also supports quaternions for smoother interpolation and to avoid gimbal lock.

3.7.4 Transformation Order

One of the most common mistakes made by beginner graphics programmers is assuming that transformations may be applied in any order.

Matrix multiplication is **not commutative**.

Therefore,

produces a different result from

For example,

scaling before translation enlarges the object around its local origin.

Translating before scaling enlarges the translation distance itself.

Most graphics engines apply transformations in the following order:

1. Scaling
2. Rotation

3. Translation

This sequence preserves intuitive object manipulation.

3.8 Homogeneous Coordinates

Translation cannot be represented using a standard 3×3 matrix alone. To overcome this limitation, modern graphics systems use **homogeneous coordinates**.

Instead of representing a point as

(x, y, z)

it becomes

$(x, y, z, 1)$

Vectors use

$(x, y, z, 0)$

This additional component allows translation, scaling, rotation, and projection to be combined into a single 4×4 transformation matrix.

Homogeneous coordinates are used throughout OpenGL, WebGL, Direct3D, Vulkan, and WebGPU.

3.9 Model, View, and Projection Matrices

The rendering pipeline transforms every vertex through three primary matrices:

Model Matrix

Transforms local object coordinates into world coordinates.

Responsible for:

- Translation
- Rotation
- Scaling

Each object has its own model matrix.

View Matrix

Represents the position and orientation of the camera.

Rather than moving the camera itself, graphics systems transform the entire world relative to the camera.

Consequently,

the camera always remains at

(0,0,0)

in view space.

Projection Matrix

Converts three-dimensional coordinates into a two-dimensional viewing volume suitable for rasterization.

This matrix determines:

- Perspective
- Viewing angle
- Aspect ratio
- Near clipping plane
- Far clipping plane

The combined transformation is

$P = \text{Projection} \times \text{View} \times \text{Model}$

commonly abbreviated as the **MVP matrix**.

This is one of the most important equations in computer graphics.

3.10 Camera Systems

A camera determines how the virtual world is viewed.

Unlike physical cameras, virtual cameras are mathematical constructs consisting of:

- Position
- Viewing direction
- Up vector
- Projection parameters

Modern rendering engines generally provide two projection models.

3.10.1 Perspective Camera

Perspective projection imitates human vision.

Objects farther from the camera appear smaller.

Characteristics include:

- Depth perception
- Realistic appearance
- Vanishing points
- Suitable for games and visualization

Perspective projection is used by most interactive 3D applications because it resembles natural vision. Three.js provides the `PerspectiveCamera` class for this purpose.

3.10.2 Orthographic Camera

Orthographic projection removes perspective distortion.

Objects remain the same size regardless of distance.

Advantages include:

- Engineering drawings
- CAD systems
- Architectural plans
- Scientific visualization
- User interfaces

Unlike perspective projection, parallel lines remain parallel.

3.11 Viewing Frustum

A perspective camera observes only a limited region of space known as the **viewing frustum**.

The frustum resembles a truncated pyramid.

Its boundaries consist of:

- Left
- Right
- Top
- Bottom
- Near Plane
- Far Plane

Objects outside the viewing frustum are discarded before rasterization.

This optimization is called **frustum culling** and significantly reduces GPU workload.

3.12 Projection

Projection converts three-dimensional geometry into two-dimensional screen coordinates.

Two projection methods dominate computer graphics.

Perspective Projection

Advantages

- Realistic
- Natural depth perception
- Immersive

Disadvantages

- Objects shrink with distance.
-

Orthographic Projection

Advantages

- Accurate measurements
- Engineering visualization
- Technical illustration

Disadvantages

- Reduced realism.

The choice depends upon application requirements.

3.13 Clipping

Not every object within a scene should be rendered.

Objects:

- behind the camera,
- outside the frustum,
- beyond the far plane,

are removed before rasterization.

This process is called **clipping**.

Benefits include:

- Reduced GPU workload
- Faster rendering
- Improved frame rate

Modern GPUs perform clipping automatically during the rendering pipeline.

3.14 Viewport Transformation

After clipping,

coordinates are transformed into

Normalized Device Coordinates (NDC)

where

$x, y, z \in [-1, +1]$

The viewport transformation converts NDC into pixel coordinates on the display.

For example,

an NDC point

$(-1, -1)$

maps to the lower-left corner of the viewport,

while

$(1, 1)$

maps to the upper-right corner.

This transformation allows the rendered image to fit displays of different resolutions while preserving the chosen aspect ratio.

3.15 Practical Application in Three.js

Three.js encapsulates these mathematical concepts through a concise application programming interface.

A typical scene includes:

- Scene
- PerspectiveCamera
- Mesh
- Renderer

Example:

JavaScript

Although the developer interacts with only a few lines of code, the graphics engine internally performs the complete sequence of model, view, and projection transformations before the GPU executes the rendering pipeline.

3.16 Chapter Summary (Part 2)

This part introduced the geometric transformations that position, orient, and scale objects within a virtual scene. It explained homogeneous coordinates, the Model–View–Projection (MVP) transformation, camera models, projection techniques, viewing frustums, clipping, and viewport transformation. Together, these concepts describe how three-dimensional geometry is mathematically converted into a two-dimensional image suitable for display.

These topics provide the essential bridge between mathematical representations and practical graphics programming. In **Part 3**, the chapter will examine **lighting models, rasterization, texture mapping,**

material systems, and the modern GPU rendering pipeline, completing the theoretical foundation required for the implementation chapters of this thesis.

Review

Part 3A: Lighting Fundamentals, Reflection Models, Rasterization, and Texture Mapping

3.17 Lighting Fundamentals

3.17.1 Introduction

Lighting is one of the most important components of computer graphics. Without lighting, three-dimensional objects appear flat and lack depth, making it difficult for users to perceive shape, orientation, or material properties.

In the physical world, objects become visible because light emitted from a source interacts with their surfaces and is reflected toward the human eye. Computer graphics simulates this process mathematically. The objective of lighting algorithms is not necessarily to reproduce physics perfectly, but to generate visually convincing images while maintaining interactive performance.

Modern real-time graphics applications, including those built with Three.js and WebGL, perform lighting calculations for millions of fragments every second. These calculations determine the final color of each pixel by considering the interaction between light sources, surface materials, and camera position. Grainger Courses+ 1

3.17.2 Components of Light

A virtual lighting system generally consists of three primary elements:

- Light source
- Surface
- Observer (camera)

The interaction among these components determines the appearance of an object.

Figure 3.17 illustrates this concept.

Figure 3.17 Basic interaction between light, surface, and camera.

The rendering equation used in physically based rendering is considerably more complex, but most real-time applications approximate this interaction using simplified illumination models.

3.17.3 Types of Light Sources

Modern rendering engines support several categories of light sources.

Ambient Light

Ambient light represents indirect illumination that appears to originate from every direction.

Characteristics include:

- Uniform intensity
- No direction
- No shadows
- Low computational cost

Although ambient lighting is not physically accurate, it prevents completely black surfaces and improves overall scene visibility. Grainger Courses+1

Directional Light

Directional lights simulate distant light sources such as the sun.

Properties include:

- Parallel light rays
- Infinite distance
- Constant direction
- Suitable for outdoor scenes

Examples include:

- Sunlight
 - Moonlight
-

Point Light

Point lights emit light equally in every direction.

Their intensity decreases with distance according to an attenuation function.

Examples include:

- Light bulbs
- Candles
- Lamps

Point lights are widely used in indoor visualization.

Spot Light

Spot lights illuminate a limited cone.

They are commonly used for:

- Flashlights
- Stage lighting
- Vehicle headlights
- Museum displays

Because the illumination is directional, spot lights provide strong visual emphasis.

Environment Light

Modern physically based rendering often employs environment lighting using High Dynamic Range (HDR) images.

Instead of defining individual light sources, the surrounding environment provides illumination from all directions.

Advantages include:

- Realistic reflections
- Soft global illumination
- Improved material realism

Environment lighting has become standard in commercial product visualization systems.

3.18 Reflection Models

3.18.1 Introduction

When light reaches a surface, it is reflected toward the observer.

Different materials reflect light differently.

For example,

- Wood scatters light.
- Glass reflects and refracts light.
- Metal produces strong specular highlights.
- Plastic exhibits both diffuse and specular reflection.

Reflection models mathematically approximate these behaviors.

3.18.2 Ambient Reflection

The simplest reflection component is ambient reflection.

Its illumination is assumed constant across the entire object.

The ambient intensity is

$I_{\text{ambient}} = k_a L_a$

where

- k_a = ambient reflection coefficient
- I_a = ambient light intensity

Ambient reflection ignores light direction and shadows.

Although physically unrealistic, it provides a useful approximation for indirect illumination.

3.18.3 Lambert Diffuse Reflection

Diffuse reflection models rough surfaces.

According to Lambert's cosine law,

brightness depends on the angle between the surface normal and the incoming light.

The diffuse component is

$$I_d = k_d L \max(0, \mathbf{N} \cdot \mathbf{L})$$

where

- $\mathbf{N}$ = normalized surface normal
- $\mathbf{L}$ = normalized light direction

If

$$\mathbf{N} \cdot \mathbf{L} = 1$$

the surface directly faces the light and appears brightest.

If

$$\mathbf{N} \cdot \mathbf{L} = 0$$

the surface receives no direct illumination.

Lambertian reflection remains one of the most widely used lighting models because of its simplicity and physical basis. Grainger Courses+1

3.18.4 Phong Reflection Model

The Phong reflection model extends Lambertian shading by introducing specular highlights.

It consists of three components:

- Ambient
- Diffuse
- Specular

The complete equation is

$$I = I_a + I_d + I_s$$

The specular term is

$$I_s = k_s (R \cdot V)^n$$

where

- R = reflection vector
- V = viewing direction
- n = shininess exponent

Higher values of

n

produce smaller and sharper highlights.

Although Phong illumination is empirical rather than physically based, it became the standard real-time lighting model for many years because it offers a good balance between image quality and computational efficiency. Wikipedia+1

3.18.5 Blinn–Phong Reflection

Blinn proposed a more efficient alternative.

Instead of computing the reflection vector,

Blinn introduced the half-vector

$$H = \frac{L + V}{|L + V|}$$

The specular intensity becomes

$$I_s = (N \cdot H)^n$$

Advantages include:

- Faster computation
- Better numerical stability
- Reduced GPU workload

For many years Blinn–Phong became the default lighting model in real-time graphics engines before the widespread adoption of physically based rendering. Grainger Courses+1

3.19 Rasterization

3.19.1 Introduction

After geometric transformations have positioned objects within the scene, the rendering system must determine which pixels belong to each object.

This process is known as **rasterization**.

Rasterization converts geometric primitives, typically triangles, into fragments that correspond to individual screen pixels.

Modern GPUs perform rasterization entirely in hardware, enabling billions of fragments to be processed each second.

3.19.2 Why Triangles?

Although objects may appear smooth, nearly all real-time graphics systems represent surfaces using triangles.

Triangles possess several desirable properties:

- Always planar
- Computationally efficient
- Simple interpolation
- Hardware friendly

Complex objects are therefore approximated by triangle meshes.

A detailed automobile model may contain several hundred thousand triangles.

3.19.3 Rasterization Process

The rasterization stage performs the following operations:

Each triangle is converted into a collection of fragments.

Each fragment stores information such as

- Position
- Depth
- Surface normal
- Texture coordinates
- Material properties

These fragments are then processed by the fragment shader.

The rasterization pipeline is optimized for speed and remains the dominant approach for interactive graphics despite the emergence of hardware-accelerated ray tracing. Stanford University+1

3.19.4 Depth Buffer

When multiple objects overlap,

the renderer must determine which surface is visible.

This is accomplished using the **depth buffer** (Z-buffer).

Each fragment stores its depth.

Before writing a pixel,

the GPU compares the fragment's depth against the stored value.

Only the nearest fragment remains visible.

Advantages include:

- Correct visibility
- Automatic hidden surface removal
- Efficient implementation

Depth buffering is performed automatically by modern graphics hardware.

3.20 Texture Mapping

3.20.1 Introduction

Geometry alone cannot realistically represent complex surface detail.

Instead,

images called **textures** are mapped onto object surfaces.

Texture mapping dramatically improves realism without increasing polygon count.

Examples include:

- Wood grain
- Fabric
- Brick walls
- Metal scratches
- Leather
- Painted surfaces

Texture mapping has become an essential technique in modern graphics applications. O'Reilly Media+1

3.20.2 UV Coordinates

Textures are applied using **UV coordinates**.

Unlike spatial coordinates

(x,y,z)

texture coordinates are represented as

(u,v)

where

- u represents horizontal texture position
- v represents vertical texture position

Every vertex stores its corresponding UV coordinates.

During rasterization,

these coordinates are interpolated across the triangle,
allowing the GPU to determine which texel should be sampled for each fragment. Wikipedia+ 1

3.20.3 Texture Sampling

The fragment shader retrieves color values from texture memory.

This operation is known as **texture sampling**.

Several sampling methods are available:

- Nearest Neighbor
- Bilinear Filtering
- Trilinear Filtering
- Anisotropic Filtering

Higher-quality filters produce smoother images but require additional computation.

Most modern graphics engines employ trilinear filtering together with anisotropic filtering to improve image quality at oblique viewing angles. O'Reilly Media

3.20.4 Mipmapping

When distant objects use high-resolution textures,
aliasing may occur.

Mipmapping solves this problem by storing multiple precomputed resolutions of the same texture.

During rendering,

the GPU automatically selects the most appropriate resolution.

Benefits include:

- Reduced aliasing
- Improved performance
- Better cache efficiency

Mipmapping is widely supported in WebGL and Three.js.

3.20.5 Types of Texture Maps

Modern physically based rendering typically uses several specialized texture maps.

Texture Type	Purpose
Base Color	Surface color
Normal Map	Small-scale surface detail
Roughness Map	Surface roughness
Metalness Map	Metallic properties
Ambient Occlusion	Indirect shadowing
Emissive Map	Self-illumination
Height Map	Surface displacement

Using multiple texture maps enables visually rich materials without requiring additional geometric complexity. Wikipedia+1

3.21 Chapter Summary (Part 3A)

This section examined the image formation stage of the graphics pipeline. It introduced the principles of virtual lighting, common light source types, and classical reflection models, including Lambertian, Phong, and Blinn–Phong illumination. The discussion then described rasterization, depth buffering, and texture mapping, illustrating how modern GPUs convert geometric models into shaded images.

Part 3B-1: Material Systems and the Modern GPU Rendering Pipeline

3.21 Material Systems

3.21.1 Introduction

Geometry describes the **shape** of an object, while materials describe how that object appears when illuminated.

A three-dimensional model without an appropriate material may contain millions of accurately positioned vertices and still appear visually unrealistic. A material determines properties such as surface color, reflectivity, roughness, transparency, and emission. In modern real-time rendering, these properties are generally supplied to shaders, which use them together with lighting information to calculate the final appearance of each visible fragment.

Three.js provides several material models ranging from simple unlit materials to physically based materials. The increasing complexity of these models generally provides greater visual realism at the cost of additional computation. Three.js+1

For the product configurator developed in this thesis, material representation is particularly important because users must be able to change the appearance of products without replacing the underlying geometry.

3.21.2 Material Properties

A material can be considered a collection of parameters that describe the interaction between a surface and light.

Common parameters include:

- Base color
- Metalness
- Roughness
- Normal information
- Ambient occlusion
- Emission
- Transparency
- Transmission
- Clearcoat

A simplified material representation can therefore be expressed as

$M = (C_b, M, R, N, A, E, T)$

where:

- C_b = base color
- M = metalness
- R = roughness
- N = normal information
- A = ambient occlusion
- E = emission
- T = transmission

The rendering system combines these parameters with geometric and lighting information to calculate the final fragment color.

3.21.3 Unlit Materials

The simplest material model does not perform lighting calculations.

In Three.js, `MeshBasicMaterial` is an example of an unlit material. Its appearance is not affected by scene lights. Three.js

This type of material is useful for:

- UI elements
- Debugging
- Background objects
- Emissive-looking surfaces
- Performance-sensitive scenes

A simplified fragment calculation can be expressed as

$$C_{out} = C_{base}$$

where the output depends primarily on the supplied material color and texture.

The advantage is low computational cost. The disadvantage is that the material does not provide natural interaction with the lighting environment.

3.21.4 Lambert and Phong Materials

Lambertian and Phong-based materials provide progressively more sophisticated lighting.

A Lambert material primarily models diffuse reflection.

A Phong material adds a specular component and therefore produces visible highlights.

Three.js documentation describes `MeshLambertMaterial` as performing lighting at the vertex level, while `MeshPhongMaterial` performs lighting per pixel. Three.js

These materials remain useful for:

- Stylized graphics
- Simple visualization
- Educational demonstrations
- Applications where physically based rendering is unnecessary

However, they are less appropriate for the product configurator developed in this thesis because realistic product visualization requires material parameters that correspond more closely to physical surface behavior.

3.21.5 Physically Based Materials

Physically based rendering introduced a different material philosophy.

Instead of directly specifying parameters such as "shininess," a PBR material describes properties that correspond more closely to the physical characteristics of a surface.

Two important parameters are:

roughness

and

metalness

Three.js's `MeshStandardMaterial` implements a metallic-roughness PBR workflow. The documentation identifies it as a standard physically based material and notes that it uses per-fragment shading. Three.js

3.21.6 Roughness

Roughness describes the microscopic irregularity of a surface.

A low roughness value produces relatively sharp reflections.

A high roughness value produces broad, diffuse reflections.

Conceptually:

versus:

In Three.js, roughness ranges from 0 to 1, with lower values producing smoother, more reflective surfaces and higher values producing rougher surfaces. Three.js+1

Examples include:

Material	Approximate Character
Polished metal	Low roughness
Glass	Low roughness
Plastic	Medium roughness
Wood	Medium/high roughness
Concrete	High roughness
Fabric	High roughness

The values used in an actual application depend on the specific material and authoring workflow.

3.21.7 Metalness

Metalness determines whether a surface behaves approximately as a conductor or non-metal.

A value near

0

represents a non-metallic material.

A value near

1

represents a metallic material.

Three.js describes the parameter in the same manner: non-metals such as wood or stone generally use values near zero, while metallic surfaces use values near one. Three.js

Examples of non-metals include:

- Wood
- Plastic
- Rubber
- Ceramic
- Stone

Examples of metals include:

- Aluminum
- Copper
- Steel
- Gold

An important consequence is that metalness should not simply be treated as an artistic "shininess" slider. It represents a fundamentally different category of light–surface interaction.

3.21.8 Texture-Based Material Properties

Material properties can be stored as textures rather than single numerical values.

For example:

$R(u,v)$

can represent roughness at each surface location.

Similarly,

$M(u,v)$

can represent metalness.

This enables a single object to contain different materials.

For example, a car model might contain:

- Metallic body panels
- Rubber tires
- Glass windows
- Plastic interior components
- Leather seats

A product configurator can therefore modify these properties dynamically without replacing the complete 3D model.

Three.js's `MeshStandardMaterial` supports maps for properties including base color, roughness, metalness, normal information, ambient occlusion, emissive properties, and displacement. Three.js

3.21.9 Normal Mapping

Increasing geometric detail is expensive.

Instead of adding millions of polygons to represent small surface irregularities, normal maps can modify the surface normal used during lighting calculations.

A normal map does **not** normally change the actual geometry. Instead, it changes the normal used for shading, creating the visual impression of additional detail. Three.js explicitly distinguishes normal mapping from displacement mapping in this respect. Three.js

For example, a flat wall can use a normal map to simulate:

- Brick
- Scratches
- Fabric
- Concrete
- Fine grooves

This technique provides significant visual detail without a corresponding increase in geometric complexity.

3.21.10 Displacement Mapping

Displacement mapping differs from normal mapping because it modifies actual vertex positions.

Conceptually,

$$P' = P + hN$$

where

- P = original vertex position
- h = sampled displacement value
- N = surface normal

Unlike normal mapping, displaced geometry can affect silhouettes and can participate in geometric interactions such as casting shadows.

Three.js documentation notes this distinction explicitly: displacement modifies vertices, whereas normal and bump maps primarily modify lighting behavior. Three.js

The disadvantage is that sufficient geometric density is required to represent the displacement accurately.

3.21.11 Material Selection for the Proposed System

The proposed product configurator primarily uses physically based materials.

The basic architecture is:

This architecture separates the physical description of a surface from the geometry itself.

As a result, changing the color or material of a product does not require rebuilding its mesh.

3.22 The Modern GPU Rendering Pipeline

3.22.1 Introduction

The GPU rendering pipeline transforms numerical descriptions of geometry into the pixels displayed on a screen.

Although modern APIs expose different interfaces, the fundamental stages remain conceptually similar.

WebGL 2 is based closely on OpenGL ES 3.0 and exposes programmable GPU rendering functionality within the HTML canvas environment. Khronos Registry+1

A simplified rendering pipeline can be represented as:

The precise implementation contains additional stages and hardware-specific mechanisms, but this conceptual model is sufficient for understanding the architecture used in this thesis.

3.22.2 Vertex Specification

Before rendering can occur, the application must provide the GPU with vertex data.

A vertex can contain multiple attributes:

$V = (P, N, UV, C, \dots)$

where:

- P = position
- N = normal
- UV = texture coordinates
- C = optional color

The vertex data is stored in GPU-accessible buffers.

OpenGL's rendering model includes vertex specification followed by vertex processing, primitive assembly, rasterization, fragment processing, and subsequent per-sample operations. Khronos Wikis

3.22.3 Vertex Shader

The vertex shader is a programmable GPU stage that processes individual vertices.

Its most important responsibility in a conventional rasterization pipeline is transforming vertex positions into clip space.

A simplified transformation is:

$P_{clip} = P_{projection} P_{view} P_{model} P_{local}$

or

$P_{clip} = MVP P_{local}$

The vertex shader may also transform or prepare:

- Normals
- Texture coordinates
- Tangent vectors
- Colors
- Custom attributes

An illustrative GLSL vertex shader is:

glsl

The actual shaders generated by a high-level engine can be substantially more complex, particularly when lighting, skinning, morph targets, shadows, and other features are enabled.

3.22.4 Primitive Assembly

After vertex processing, vertices are assembled into geometric primitives.

The most common primitive for 3D rendering is the triangle.

For example:

Three vertices form one triangle.

Complex models are therefore represented as collections of triangles.

3.22.5 Rasterization

The rasterizer determines which screen locations are covered by each primitive.

For a triangle, the GPU identifies the pixels or sample locations inside its projected area.

It then interpolates vertex attributes across the triangle.

For example, if each vertex has a different texture coordinate, the rasterizer generates intermediate texture coordinates for each fragment.

This interpolation is essential for smooth shading and texture mapping.

3.22.6 Fragment Generation

Rasterization produces **fragments**, not necessarily final pixels.

A fragment contains information needed for subsequent processing, such as:

- Screen position
- Depth
- Interpolated attributes

The fragment shader then processes this information.

The distinction is important because multiple fragments may compete to write to the same screen pixel. Depth testing, blending, and other operations determine which values ultimately contribute to the framebuffer. Khronos Wikis

3.22.7 Fragment Shader

The fragment shader determines the color and, where applicable, depth information for a generated fragment.

A simplified fragment shader might be:

```
glsl
```

This produces a red output.

A realistic PBR fragment shader is considerably more complex.

It may perform:

1. Texture sampling
2. Normal reconstruction
3. Light evaluation
4. BRDF evaluation
5. Environment sampling
6. Ambient occlusion
7. Emission
8. Tone mapping preparation

The fragment shader is therefore one of the most computationally significant parts of a modern rendering system.

3.22.8 Depth Testing

When several objects overlap, the GPU must determine which surface is closest to the camera.

The depth buffer stores a depth value for previously processed fragments.

Conceptually:

If Object A is closer than Object B, its fragment should normally remain visible.

The depth test performs this comparison automatically.

This mechanism allows scenes containing many overlapping objects to be rendered correctly without requiring the application to manually sort every triangle.

3.22.9 Stencil Testing

The stencil buffer provides an additional mechanism for controlling where rendering is permitted.

Stencil operations can be used for:

- Mirrors
- Portals
- Outlines

- Masks
- Selective rendering

Although stencil operations are not central to the product configurator developed in this thesis, they are important for advanced interactive graphics.

3.22.10 Blending

Blending combines a newly generated fragment with the existing framebuffer value.

For transparent objects, the output may be represented conceptually as

$$C_{out} = C_{src}\alpha + C_{dst}(1 - \alpha)$$

where:

- C_{src} = source color
- C_{dst} = destination color
- α = source opacity

This operation enables effects such as:

- Glass
- Smoke
- UI overlays
- Transparent plastics
- Particle effects

Transparency is more complicated than opaque rendering because the correct visual result may depend on drawing order and depth behavior.

3.22.11 Framebuffers

The final result of a rendering pass is stored in a framebuffer.

The default framebuffer represents the visible rendering surface associated with the canvas.

WebGL 2 also supports framebuffer objects that can be configured with texture and renderbuffer attachments. Khronos Registry

This capability is fundamental to advanced rendering techniques.

For example:

Framebuffer-based rendering enables:

- Bloom
- Depth-of-field
- Screen-space effects
- Blur
- Color grading

- Deferred rendering
- Shadow maps
- GPGPU techniques

These techniques become particularly important in the later chapters of this thesis.

3.22.12 GPU Parallelism

The fundamental advantage of GPUs is massive parallelism.

A CPU is optimized for relatively small numbers of powerful, general-purpose processing cores.

A GPU is designed to execute large numbers of similar operations simultaneously.

This makes GPUs particularly effective for graphics workloads where thousands or millions of vertices and fragments require similar calculations.

For example, a scene containing:

1,000,000

fragments may require essentially the same shader computation for each fragment.

The GPU can execute these calculations concurrently through its architecture.

This parallel nature explains why shader programming is so important in real-time graphics.

3.22.13 CPU–GPU Interaction

A web-based graphics application generally involves cooperation between the CPU and GPU.

A simplified architecture is:

The CPU manages:

- Application logic
- User interaction
- Scene updates
- Resource management
- Rendering commands

The GPU performs:

- Vertex processing
- Rasterization
- Fragment shading
- Texture sampling
- Parallel numerical computation

A major performance objective is therefore to minimize unnecessary CPU–GPU synchronization and expensive rendering work.

3.22.14 Rendering Cost

The cost of a frame can be broadly understood as a combination of CPU and GPU work:

$$T_{\text{frame}} = T_{\text{CPU}} + T_{\text{GPU}} + T_{\text{sync}}$$

where:

- T_{CPU} = CPU-side work
- T_{GPU} = GPU rendering work
- T_{sync} = synchronization and communication overhead

For a target frame rate of 60 frames per second, the theoretical frame budget is approximately

$$T_{\text{frame}} = \frac{1}{60} \approx 16.67 \text{ ms}$$

Thus, the complete system must generally complete the required work within approximately 16.67 milliseconds per frame to sustain a nominal 60 FPS rate.

This does not mean that every application must target exactly 60 FPS; the appropriate target depends on the application, display hardware, and user experience requirements. Nevertheless, frame-time budgeting provides a useful framework for analyzing interactive rendering performance.

3.22.15 Draw Calls

A draw call is an instruction that causes the graphics API to render geometry using specified state.

Large numbers of draw calls can become a performance bottleneck, particularly when CPU-side command submission becomes expensive.

Consider a scene containing:

versus a carefully organized scene using:

The second scene may render substantially more efficiently even if the total polygon count is similar.

This is why real-time optimization cannot be reduced to polygon count alone.

Other important factors include:

- Draw-call count
 - Shader complexity
 - Texture bandwidth
 - GPU memory
 - Resolution
 - Overdraw
 - CPU-side scene management
-

3.22.16 Rendering Pipeline in the Proposed Product Configurator

The rendering architecture developed for this thesis can be summarized as follows:

This architecture separates application-level interaction from low-level GPU rendering while maintaining access to advanced graphics capabilities.

3.22.17 Importance of Pipeline Awareness

Although high-level libraries simplify graphics programming, developers should understand the underlying rendering pipeline.

For example, changing the following properties can have substantially different performance implications:

- Number of vertices
- Number of draw calls
- Texture resolution
- Shader complexity
- Number of lights
- Shadow resolution
- Post-processing passes

A developer who understands the pipeline can identify whether a problem is primarily:

CPU-bound

or

GPU-bound.

This distinction is essential for effective optimization.

3.23 Part 3B-1 Summary

This section examined the two major concepts required to understand modern real-time rendering: **material systems** and the **GPU rendering pipeline**.

Material systems define how surfaces interact with light. Traditional materials such as Lambert and Phong models provide relatively simple approximations, while physically based materials use parameters such as roughness and metalness to produce more consistent material behavior. Texture maps further extend these systems by allowing material properties to vary spatially across a surface. Three.js provides a hierarchy of material implementations, including simple unlit materials and physically based `MeshStandardMaterial` and `MeshPhysicalMaterial` implementations. Three.js+2Three.js+2

The second half of the section examined the GPU rendering pipeline, from vertex specification and vertex processing through primitive assembly, rasterization, fragment shading, depth testing, blending, and framebuffer output. WebGL exposes this programmable rendering architecture to browser applications,

while Three.js provides a higher-level abstraction over many of these operations. Khronos Registry+2Khronos Wikis+2

The discussion also established an important principle for the remainder of this thesis: **visual quality and rendering performance are inseparable from the underlying GPU pipeline**. A sophisticated interactive application must therefore consider not only the appearance of its materials but also the computational cost of geometry, shaders, textures, draw calls, and framebuffer operations.

The final section of Chapter 3 will build upon these foundations by examining **physically based rendering in greater detail**, including the rendering equation, BRDFs, metallic-roughness workflows, image-based lighting, tone mapping, and the relationship between physical accuracy and real-time performance.

Part 3B-2: Physically Based Rendering and Chapter Conclusion

3.23 Physically Based Rendering

3.23.1 Introduction

Physically Based Rendering (PBR) is a rendering methodology designed to produce visually consistent images by modeling the interaction between light and surfaces according to principles derived from physics.

Traditional real-time lighting models, such as the Phong model, often use parameters that are primarily artistic abstractions. PBR instead attempts to describe materials using properties that correspond more closely to real-world behavior.

This does not mean that PBR is a complete simulation of physical reality. Real-time graphics must operate under strict computational constraints. Instead, PBR provides a carefully designed approximation that produces plausible results while remaining practical for interactive applications.

PBR has become particularly important in:

- Game development
- Product visualization
- Architectural visualization
- Digital twins
- Virtual reality
- Web-based 3D applications
- Film and animation
- E-commerce visualization

Three.js provides PBR through materials such as `MeshStandardMaterial` and `MeshPhysicalMaterial`. (threejs.org)

3.23.2 The Rendering Equation

The theoretical foundation of physically based rendering is the **rendering equation** introduced by James Kajiya.

For a surface point x , outgoing radiance can be expressed conceptually as

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} f_r(x, \omega_i, \omega_o) L_i(x, \omega_i) (n \cdot \omega_i) d\omega_i$$

where:

- L_o = outgoing radiance
- L_e = emitted radiance
- L_i = incoming radiance
- f_r = bidirectional reflectance distribution function (BRDF)
- ω_i = incoming light direction
- ω_o = outgoing/view direction
- n = surface normal
- Ω = hemisphere above the surface

The equation describes a fundamental idea:

The light leaving a surface is determined by emitted light plus reflected incoming light.

The integral represents the contribution of light arriving from every possible direction.

In a theoretical renderer, evaluating this integral exactly would be computationally expensive. Real-time graphics therefore approximate it using numerical techniques, analytical models, precomputation, and hardware acceleration.

3.23.3 BRDF

The **Bidirectional Reflectance Distribution Function (BRDF)** describes how incoming light is reflected toward a particular outgoing direction.

It can be expressed as

$$f_r(\omega_i, \omega_o)$$

A physically plausible BRDF should satisfy important constraints, including conservation of energy and appropriate reciprocity behavior.

The BRDF is central to PBR because it provides the mathematical relationship between:

- Material
- Incoming light
- Surface orientation
- Viewing direction

Different BRDF models produce different visual characteristics.

3.23.4 Microfacet Theory

Modern real-time PBR commonly models surfaces as collections of microscopic facets.

Although a surface may appear smooth at a macroscopic scale, its microscopic structure determines how light is reflected.

Conceptually:

Each microscopic facet has its own orientation.

A smooth material contains facets with relatively similar orientations.

A rough material contains facets with greater variation.

This explains why roughness affects the width and intensity of specular reflections.

3.23.5 Normal Distribution Function

A microfacet BRDF typically contains a **Normal Distribution Function (NDF)**.

The NDF describes the statistical distribution of microfacet orientations.

One commonly used distribution is the Trowbridge–Reitz GGX distribution.

A simplified form can be represented as

$$D_{GGX}(N, H, \alpha) = \pi [(N \cdot H)^2 (\alpha^2 - 1) + 1]^{-2} \alpha^2$$

where:

- N = surface normal
- H = half-vector
- α = roughness-related parameter

The exact implementation may use a remapped roughness value, such as

$$\alpha = r^2$$

where r is the user-facing roughness parameter.

The GGX distribution is widely used because it provides plausible long specular tails and behaves well for a broad range of roughness values.

3.23.6 Fresnel Reflection

The amount of reflected light changes according to viewing angle.

This phenomenon is described by the **Fresnel effect**.

At grazing angles, surfaces generally become more reflective.

For example, when looking directly at a glass surface, its reflection may be relatively subtle. When viewing the same surface at a shallow angle, the reflection becomes much stronger.

A commonly used approximation is Schlick's Fresnel approximation:

$$F(\theta) = F_0 + (1 - F_0)(1 - \cos\theta)^5$$

where F_0 represents reflectance at normal incidence.

This approximation is computationally inexpensive and therefore suitable for real-time rendering.

3.23.7 Geometry Term

Microfacet BRDFs must also account for masking and shadowing between microscopic surface facets.

This is represented by a geometry term commonly written as

$$G(N, V, L)$$

The geometry function reduces reflected energy when microfacets obscure one another.

A common approach is based on the Smith masking-shadowing model.

The geometry term is particularly important for producing plausible rough-surface behavior.

3.23.8 Cook–Torrance BRDF

A widely used physically based specular model is the **Cook–Torrance BRDF**.

Conceptually:

$$f_r = 4(N \cdot V)(N \cdot L)DGF$$

where:

- D = normal distribution function
- G = geometry term
- F = Fresnel term
- N = surface normal
- V = viewing direction
- L = light direction

This formulation provides a physically motivated model for specular reflection and forms the basis of many modern PBR implementations.

The complete rendering system, however, requires more than the specular BRDF alone. Diffuse reflection, direct illumination, indirect illumination, environment lighting, and material properties must also be considered.

3.23.9 Energy Conservation

A physically plausible material should not reflect more energy than it receives.

This principle is known as **energy conservation**.

If

E_{in}

represents incoming energy, then a simplified condition is

$E_{out} \leq E_{in}$

This is important when combining diffuse and specular components.

For example, increasing the specular contribution should generally reduce the amount of energy available to the diffuse component.

PBR systems incorporate these relationships to prevent materials from producing physically implausible brightness.

3.23.10 Metallic-Roughness Workflow

One of the most common PBR workflows is the **metallic-roughness workflow**.

A simplified material can be represented using:

$M = (C_{base}, m, r)$

where:

- C_{base} = base color
- m = metalness
- r = roughness

The material is then interpreted according to the following conceptual model:

This workflow is particularly convenient for artists because a small number of parameters can represent a wide variety of materials.

3.23.11 Image-Based Lighting

Direct light sources are not sufficient for convincing material appearance.

Consider a polished metallic object.

A metal surface primarily reflects its environment. If the environment contains nothing interesting, the metal may appear visually flat even if the lighting calculation is mathematically correct.

Image-Based Lighting (IBL) addresses this problem by using an environment map as a source of illumination and reflection information.

A typical HDR environment contains substantially more dynamic range than a conventional 8-bit image.

The environment can be represented as

$L_{env}(\omega)$

where each direction corresponds to incoming environmental radiance.

The renderer integrates this environment over the surface hemisphere.

3.23.12 High Dynamic Range Lighting

Real-world illumination contains a very large range of intensities.

For example:

- A dark wall
- A room lamp
- A bright window
- Direct sunlight

may all exist within the same scene.

Standard 8-bit color values cannot represent this entire range accurately.

HDR rendering therefore uses floating-point representations that allow values beyond the conventional display range.

Conceptually,

CHDR

is calculated first, followed by conversion to displayable output:

$C_{display} = \text{ToneMap}(CHDR)$

This process is important for realistic lighting and physically based materials.

3.23.13 Tone Mapping

A monitor cannot directly display arbitrary HDR values.

Tone mapping converts high-dynamic-range values into a displayable range.

A simplified tone-mapping function can be represented as

$C_{out} = T(CHDR)$

where T is the tone-mapping operator.

Different operators produce different visual characteristics.

Common approaches include:

- Reinhard
- ACES-inspired operators
- Filmic operators

Tone mapping influences the final artistic appearance of an application and therefore should be treated as part of the rendering pipeline rather than merely a post-processing effect.

3.23.14 Gamma and Color Management

Another important consideration is color space.

Color values used for image display are not necessarily linear representations of light intensity.

Rendering calculations generally require a linear-light representation.

A simplified workflow is:

Incorrect color-space handling can produce:

- Incorrect brightness
- Incorrect material appearance
- Washed-out images
- Incorrect blending
- Unrealistic lighting

Therefore, color management is an essential component of a modern graphics system.

3.23.15 PBR in Three.js

Three.js provides `MeshStandardMaterial` for physically based metallic-roughness rendering. It also provides `MeshPhysicalMaterial`, which extends the standard PBR material with additional properties such as transmission, thickness, clearcoat, sheen, and related physical effects. (threejs.org)

A simplified example is:

JavaScript

A more advanced physical material can include additional properties:

JavaScript

The exact visual result depends on the renderer configuration, lighting environment, geometry, textures, and color-management settings.

3.23.16 PBR and Product Visualization

PBR is particularly appropriate for product visualization.

A product configurator may need to represent:

- Brushed aluminum
- Polished steel
- Matte plastic
- Glossy plastic
- Leather
- Glass
- Ceramic

- Wood

Instead of creating a separate 3D model for every variation, the same geometry can be combined with different material parameters.

For example:

$\text{Product} = \text{Geometry} + \text{Material}$

Therefore:

This dramatically reduces asset duplication.

3.23.17 Performance Considerations

PBR improves visual consistency but increases computational complexity.

Performance is affected by:

- Number of lights
- Shadow calculations
- Texture resolution
- Environment map resolution
- Shader complexity
- Material features
- Screen resolution
- Post-processing
- Number of visible objects

Therefore, a production-quality interactive graphics application must balance:

Visual Quality

against

Rendering Performance

The objective is not maximum graphical complexity.

Instead, the objective is an appropriate quality-to-cost ratio.

3.23.18 Real-Time Constraints

For interactive applications, rendering must happen continuously.

At a target of 60 FPS,

$T_{\text{frame}} \approx 16.67\text{ms}$

At 120 FPS,

$T_{\text{frame}} \approx 8.33\text{ms}$

Therefore, doubling the target frame rate approximately halves the available time for each frame.

This becomes especially important for:

- VR
- AR
- Interactive 3D websites
- Games
- Real-time product configurators

Virtual reality can impose particularly demanding latency and frame-rate requirements because rendering delays can directly affect the user's perception of responsiveness.

3.23.19 Web Graphics and PBR

The development of WebGL and modern browser graphics APIs has made sophisticated rendering available directly through web browsers.

A web-based graphics application can therefore combine:

This enables applications that previously would have required specialized desktop software.

Examples include:

- Interactive portfolios
- Product configurators
- 3D e-commerce
- Architectural visualization
- Scientific visualization
- Virtual exhibitions
- Educational simulations

The web therefore represents an important deployment platform for interactive computer graphics.

3.23.20 Relationship Between Graphics Theory and the Proposed System

The theoretical concepts presented throughout Chapter 3 can now be connected to the architecture of the proposed application.

The complete process can be summarized as:

This pipeline demonstrates how the mathematical foundations introduced at the beginning of the chapter ultimately produce the visual image presented to the user.

3.24 Chapter Conclusion

This chapter established the theoretical foundation for interactive three-dimensional computer graphics.

The discussion began with the mathematical concepts required to represent and manipulate graphical objects. Vectors provide representations of direction, displacement, and orientation, while matrices provide an efficient framework for geometric transformation. These mathematical structures allow objects to be translated, rotated, scaled, and projected through multiple coordinate spaces.

The chapter then examined the transformation pipeline from object space through world space, view space, clip space, normalized device coordinates, and finally screen space. This process forms the mathematical basis for converting three-dimensional geometry into a two-dimensional image.

The second major topic was lighting and image formation. Classical illumination models such as Lambert, Phong, and Blinn–Phong were introduced to demonstrate how surface orientation, light direction, and camera position affect appearance. Rasterization and texture mapping were then examined as the mechanisms that convert geometric primitives into fragments and apply detailed surface information.

The chapter subsequently introduced modern material systems. Parameters such as base color, roughness, metalness, normal information, and emission allow a rendering engine to represent a broad range of real-world materials. These properties are particularly important for interactive product visualization because they allow a single geometric model to support numerous visual configurations.

Finally, the chapter examined the modern GPU rendering pipeline and physically based rendering. The rendering equation, BRDFs, microfacet theory, Fresnel reflection, energy conservation, image-based lighting, HDR rendering, tone mapping, and color management were discussed as the theoretical foundations of modern PBR.

The resulting conceptual model is:

Geometry→Transformation→Rasterization→Material→Lighting→PBR→Image

Understanding this sequence is essential for the subsequent implementation chapters. Rather than treating Three.js, WebGL, or React Three Fiber as isolated software libraries, the developer can now understand the underlying mathematical and computational processes represented by those abstractions.

3.25 Transition to the Next Chapter

With the theoretical foundations established, the thesis can now move from **what computer graphics is** to **how an interactive graphics system is implemented**.

The next chapter should therefore introduce the implementation architecture of the proposed **Interactive Graphics and Web Technologies** system.

A logical Chapter 4 structure would be:

Chapter 4 — System Architecture and Implementation

4.1 System Requirements

4.2 Overall System Architecture

4.3 Front-End Architecture

4.4 React and React Three Fiber

4.5 Three.js Scene Architecture

4.6 Asset Pipeline

4.7 glTF/GLB Model Loading

4.8 Material and Texture Management

4.9 Camera and Interaction System

4.10 Animation Architecture

4.11 WebGL Rendering Architecture

4.12 Performance Optimization

4.13 Responsive and Mobile Rendering

4.14 Implementation Results

4.15 Chapter Summary

This transition is important because **Chapter 3 establishes the theory**, while Chapter 4 should demonstrate how that theory is translated into an actual web-based interactive graphics application.

Chapter 3 — Completed Structure

At this point, the chapter has the following complete structure:

Section	Topic
3.1–3.2	Introduction and mathematical representation
3.3	Vectors
3.4	Matrices
3.5	Coordinate systems
3.7	Geometric transformations
3.8	Homogeneous coordinates
3.9	Model, View, Projection matrices
3.10–3.12	Cameras, frustums, projection
3.13–3.16	Clipping, viewport, Three.js application
3.17–3.18	Lighting and reflection
3.19	Rasterization
3.20	Texture mapping
3.21	Material systems
3.22	GPU rendering pipeline
3.23	Physically based rendering
3.24	Chapter conclusion
3.25	Transition to implementation

Chapter 4

System Architecture and Implementation

4.1 Introduction

Chapter 3 established the mathematical and theoretical foundations of computer graphics, including coordinate transformations, rasterization, texture mapping, material systems, GPU processing, and physically based rendering. This chapter builds upon those foundations by describing the architecture and implementation of the proposed interactive graphics system.

The objective of the system is to demonstrate how modern web technologies can be combined with real-time 3D graphics to create an interactive application capable of presenting and manipulating three-dimensional content directly within a web browser.

The proposed architecture uses a layered approach:

This architecture separates application logic from graphics rendering while allowing both systems to communicate efficiently.

4.2 System Requirements

Before implementation, functional and non-functional requirements were identified.

4.2.1 Functional Requirements

The proposed system should provide:

1. Real-time 3D rendering.
2. Interactive camera control.
3. Loading of external 3D models.
4. Material modification.
5. Texture support.
6. Object animation.
7. Responsive user interaction.
8. Web-based deployment.
9. Support for modern browsers.
10. Extensibility for future graphics features.

The system should also allow developers to add advanced functionality without redesigning the entire architecture.

4.2.2 Non-Functional Requirements

The system should satisfy several performance and usability requirements.

Performance

The renderer should maintain a stable frame rate under normal operating conditions.

Scalability

The architecture should support increasingly complex models and scenes.

Maintainability

Individual components should have clearly defined responsibilities.

Accessibility

The application should remain usable across different screen sizes and input devices.

Portability

The application should operate on modern desktop and mobile browsers supporting the required graphics APIs.

4.3 Overall System Architecture

The proposed system consists of five principal layers.

Each layer performs a distinct responsibility.

4.4 Front-End Architecture

The application interface is implemented using a component-based architecture.

A simplified structure is:

Component-based development provides several advantages.

Each component can:

- Encapsulate functionality
- Maintain local state
- Receive properties
- Be reused
- Be independently tested

This approach is particularly useful when combining traditional web interfaces with interactive graphics.

4.5 React Architecture

React provides the user-interface layer of the system.

Traditional DOM components are responsible for:

- Buttons
- Menus
- Sliders
- Forms
- Navigation
- Text

The 3D rendering area is provided by React Three Fiber.

The separation can therefore be represented as:

This architecture allows developers to use familiar React concepts while working with a real-time 3D scene.

4.6 React Three Fiber

React Three Fiber (R3F) provides a React renderer for Three.js.

Instead of manually constructing and updating a Three.js scene, developers can describe the scene declaratively using React components.

For example:

JavaScript

The developer describes the desired scene rather than manually managing every rendering operation.

This reduces boilerplate and allows graphics components to participate naturally in React's component architecture.

However, R3F does not replace knowledge of Three.js.

Understanding:

- Scene graphs
- Cameras
- Materials
- Geometry
- Render loops
- GPU resources

remains essential for developing sophisticated applications.

4.7 Scene Architecture

A Three.js scene is structured as a scene graph.

A simplified graph is:

The scene graph represents hierarchical relationships.

For example, if a wheel belongs to a vehicle object, moving the vehicle automatically moves the wheel.

This hierarchical representation is essential for:

- Animation
 - Object transformation
 - Scene organization
 - Interaction
-

4.8 Asset Pipeline

Three-dimensional assets must be prepared before they can be rendered.

The proposed asset pipeline is:

Blender is used as the primary asset creation environment because it provides comprehensive modeling, UV, animation, material, and export functionality.

4.9 glTF and GLB

The proposed application uses the glTF ecosystem for 3D asset delivery.

glTF is designed as an efficient format for transmission and loading of 3D scenes and models.

The binary `.glb` variant packages the relevant data into a single binary file.

A typical model can contain:

- Meshes
- Materials
- Textures
- Cameras
- Animations
- Scene hierarchy

This makes glTF particularly appropriate for web-based real-time graphics.

4.10 Model Loading

Three.js provides `GLTFLoader` for loading glTF assets.

A simplified implementation is:

JavaScript

The asynchronous loading process prevents the browser from blocking while the asset is downloaded.

A production application should additionally provide:

- Loading indicators
 - Error handling
 - Asset caching
 - Progressive loading where appropriate
 - Resource cleanup
-

4.11 Asset Optimization

3D assets must be optimized before deployment.

Optimization may include:

- Polygon reduction
- Texture compression
- Mesh compression
- Removing unused materials
- Removing unused animations
- Reducing texture resolution
- Combining compatible geometry

The objective is to minimize:

Download Size+GPU Memory+Rendering Cost

without producing unacceptable visual degradation.

4.12 Material Management

Material definitions should be separated from application logic.

For example:

JavaScript

The application can then switch materials dynamically.

JavaScript

In a larger application, material definitions can be stored in configuration files or external data structures rather than being hard-coded into components.

4.13 Dynamic Product Configuration

One of the primary use cases of the proposed architecture is interactive product customization.

For example:

The geometry remains unchanged whenever possible.

Only the relevant material or object properties are modified.

This minimizes unnecessary resource usage.

4.14 Application State

Interactive applications must maintain state describing the current configuration.

A simplified state model is:

JavaScript

When the state changes, the corresponding graphics components update.

The conceptual relationship is:

UI→State→Scene→Render

This unidirectional data-flow model makes complex interactions easier to reason about.

4.15 Camera and Interaction System

Camera interaction is essential for an interactive 3D application.

The system may provide:

- Orbit
- Zoom
- Pan
- Rotation
- Reset
- Preset viewpoints

A typical user interaction is:

Orbit-style interaction is especially appropriate for product visualization because users can inspect an object from arbitrary directions.

4.16 Animation Architecture

Animation can be divided into two categories:

Object Animation

Examples:

- Rotating products
- Opening doors
- Moving components
- Mechanical demonstrations

Interface Animation

Examples:

- Menu transitions
- Page transitions
- Control-panel animations
- Camera transitions

The separation is important because interface animation and GPU-based 3D animation have different requirements.

4.17 Render Loop

Real-time applications continuously update the scene.

Conceptually:

JavaScript

At every frame:

1. User input is processed.
2. Application state is updated.
3. Object transformations are calculated.
4. Animation state is updated.
5. The scene is rendered.
6. The framebuffer is displayed.

React Three Fiber manages the underlying render loop while providing hooks for custom per-frame behavior.

4.18 Frame-Based Animation

A major principle of real-time animation is that animation should be based on elapsed time rather than frame count.

Incorrect approach:

JavaScript

The apparent animation speed can vary depending on frame rate.

A better conceptual approach is:

$$\theta_{\text{new}} = \theta_{\text{old}} + \omega \Delta t$$

where:

- ω = angular velocity
- Δt = elapsed time

This produces more consistent motion across different hardware.

4.19 GSAP Integration

For complex interface animation, an animation framework such as GSAP can be used.

For example:

JavaScript

This is useful for:

- Camera transitions
- Scene introductions
- UI transitions
- Product presentations
- Scroll-driven animation

The architecture can therefore use different animation systems for different responsibilities:

4.20 Lighting Architecture

The lighting system should be designed independently from individual models.

A typical product visualization scene may contain:

The environment provides broad illumination while additional lights provide controlled highlights and visual separation.

This approach is preferable to adding large numbers of independent point lights.

4.21 Environment Maps

An HDR environment map can provide both:

- Illumination
- Reflection information

This is particularly important for metallic materials.

A metal object without an environment may appear nearly black or visually uninteresting because there is little environmental information to reflect.

Therefore:

Material Quality $\neq$ Geometry Quality

A high-quality material also requires an appropriate lighting environment.

4.22 Texture Management

Textures can represent:

- Color
- Roughness
- Metalness
- Normal information
- Ambient occlusion
- Emission

Texture resolution should be selected according to the object's visual importance.

For example:

This principle is known as allocating resources according to visual importance.

4.23 Responsive Rendering

A web-based graphics system must adapt to different viewport sizes.

When the browser dimensions change, the camera aspect ratio and renderer dimensions must be updated.

Conceptually:

Aspect $\neq$ Height/Width

The projection matrix must then be recalculated.

A simplified implementation is:

JavaScript

Failure to update the aspect ratio causes geometric distortion.

4.24 Mobile Considerations

Mobile devices introduce additional constraints.

Compared with desktop systems, they may have:

- Lower GPU performance

- Smaller memory capacity
- Higher thermal constraints
- Touch input
- Variable display resolution

Therefore, the application should consider adaptive rendering.

Potential strategies include:

- Lower pixel ratio
- Reduced shadow resolution
- Reduced texture resolution
- Fewer post-processing effects
- Lower polygon complexity

For example:

JavaScript

Limiting pixel ratio can substantially reduce GPU workload on high-density displays.

4.25 Performance Optimization

Performance optimization is a major component of real-time graphics development.

The primary performance variables are:

$\text{Performance} = f(\text{Geometry}, \text{DrawCalls}, \text{Shaders}, \text{Textures}, \text{Resolution}, \text{Lighting}, \text{PostProcessing})$

No single optimization technique solves every performance problem.

4.25.1 Geometry Optimization

Excessive polygon counts increase:

- Vertex processing
- Memory consumption
- File size

Techniques include:

- Mesh simplification
 - Level of detail
 - Instance rendering
 - Removing invisible geometry
-

4.25.2 Draw-Call Optimization

A large number of independent objects can increase CPU-side rendering overhead.

Possible techniques include:

- Instancing
 - Material consolidation
 - Geometry merging
 - Scene restructuring
-

4.25.3 Texture Optimization

Textures can consume significant GPU memory.

A texture with resolution

4096×4096

contains over 16 million texels.

Multiple such textures can rapidly consume hundreds of megabytes of memory.

Therefore, texture resolution should be selected according to actual visual requirements.

4.25.4 Shader Optimization

Fragment shaders execute potentially millions of times per frame.

Consequently, expensive operations inside a fragment shader can have significant performance implications.

For example:

Shader complexity should therefore be minimized whenever visual quality allows.

4.26 Performance Measurement

Optimization should be based on measurement rather than assumptions.

Important metrics include:

- FPS
- Frame time
- CPU time
- GPU time
- Draw calls
- Triangle count
- Texture memory
- Shader complexity

A particularly useful metric is frame time.

$FPS = \frac{1}{T_{frame}}$

where T_{frame} is measured in seconds.

For example,

$T_{\text{frame}} = 0.0167\text{s}$

corresponds approximately to

$\text{FPS} = 60$

4.27 Error Handling

A production-quality graphics application must handle failures gracefully.

Potential errors include:

- Model loading failure
- Missing texture
- Unsupported format
- WebGL initialization failure
- Network timeout
- Insufficient GPU memory

A robust system should provide meaningful feedback rather than leaving the user with an empty canvas.

Example:

4.28 Loading Experience

Large 3D assets may require several seconds to download.

Therefore, the application should provide a loading state.

For example:

This improves perceived performance because users receive immediate feedback.

4.29 Security Considerations

Although graphics applications are primarily client-side, security remains important.

External assets should be loaded from trusted sources.

Applications should also consider:

- Content Security Policy
- Cross-origin restrictions
- Untrusted model files
- Third-party dependencies
- API credentials

Sensitive credentials should never be embedded in client-side JavaScript.

4.30 Deployment Architecture

The final application can be deployed as a modern web application.

A simplified deployment architecture is:

Static assets can be served through a content delivery network to reduce latency for geographically distributed users.

4.31 Proposed Project Directory

A practical project structure is:

The exact structure can vary depending on project size, but the key principle is separation of concerns.

4.32 Example System Implementation

The following simplified example demonstrates how the architecture can be combined.

JavaScript

Although this example is small, it demonstrates the principal architecture:

A complete implementation extends this structure with asynchronous model loading, application state, PBR materials, environment lighting, animations, responsive layout, and performance optimization.

4.33 Implementation Workflow

The recommended development process is iterative.

Stage 1 — Scene Initialization

Create:

- Canvas
- Renderer
- Camera
- Basic lighting

Stage 2 — Geometry

Add:

- Primitive geometry
- External models

- Scene hierarchy

Stage 3 — Materials

Add:

- Base colors
- Textures
- Roughness
- Metalness

Stage 4 — Interaction

Add:

- Orbit controls
- Object selection
- Product configuration

Stage 5 — Animation

Add:

- Camera transitions
- Object animation
- UI motion

Stage 6 — Optimization

Measure:

- Frame rate
- Draw calls
- GPU memory
- Asset size

Stage 7 — Deployment

Test:

- Desktop
- Tablet
- Mobile
- Different browsers
- Different GPU capabilities

This incremental process reduces complexity and makes debugging significantly easier.

4.34 Testing Methodology

The proposed system should be evaluated at multiple levels.

Functional Testing

Verify:

- Models load correctly.
- Materials change correctly.
- Camera controls operate correctly.
- UI controls update scene state.
- Animations execute correctly.

Visual Testing

Verify:

- Lighting is consistent.
- Materials appear correctly.
- Textures align with geometry.
- Shadows are visually appropriate.

Performance Testing

Measure:

- Average FPS
- Minimum FPS
- Frame time
- Draw calls
- Memory usage

Compatibility Testing

Test on:

- Chrome
- Firefox
- Safari
- Edge

and across different GPU classes.

4.35 Chapter Summary

This chapter translated the theoretical concepts introduced in Chapter 3 into a practical web-based system architecture.

The proposed architecture combines:

React+React Three Fiber+Three.js+WebGL+GPU

React provides the application and interface layer, while React Three Fiber connects the declarative React architecture with the Three.js scene graph. Three.js provides high-level access to 3D geometry, cameras, materials, lighting, animation, and rendering functionality. WebGL provides access to the browser's GPU-accelerated graphics pipeline.

The chapter also established an asset pipeline based on Blender and glTF/GLB, providing a practical workflow from 3D modeling through optimization and browser deployment.

Particular attention was given to material management and physically based rendering because realistic material behavior is essential to the proposed interactive visualization system. Dynamic application state allows users to modify product properties without unnecessarily rebuilding the underlying geometry.

Performance optimization was also identified as a fundamental architectural concern. Geometry complexity, draw-call count, texture resolution, shader complexity, lighting, and screen resolution all influence frame time. Consequently, performance must be measured systematically rather than estimated solely from visual complexity.

The complete architecture can therefore be summarized as:

User→React→R3F→Three.js→WebGL→GPU→Display

This architecture provides a foundation for the practical implementation of interactive 3D web applications.

Chapter 5

Implementation and Experimental Evaluation of an Interactive 3D Web Graphics System

5.1 Introduction

The previous chapter established the architecture of the proposed Interactive Graphics and Web Technologies system. The architecture integrates React, React Three Fiber, Three.js, WebGL, physically based materials, external 3D assets, animation systems, and interactive user interfaces.

This chapter describes the practical implementation of the proposed system and establishes an experimental methodology for evaluating its functionality and performance.

The principal objective is to demonstrate that modern web technologies can support sophisticated interactive 3D experiences while maintaining an architecture that is modular, maintainable, and suitable for real-world deployment.

The implemented system is designed around an interactive 3D product visualization scenario. Users can inspect a three-dimensional object, rotate and zoom the camera, modify material properties, select product configurations, and observe real-time visual changes.

The overall implementation process can be summarized as:

Asset Creation→Asset Optimization→Web Integration→Scene
Construction→Interaction→Rendering→Evaluation

The chapter first describes the development environment, followed by the implementation of the rendering system, model-loading pipeline, material system, interaction system, animation, shader functionality, and optimization procedures. Finally, an experimental framework is established for evaluating the resulting application.

5.2 Implementation Environment

5.2.1 Hardware Environment

Real-time graphics performance depends strongly on the underlying hardware.

The experimental environment should therefore record at least the following parameters:

Category	Parameter
CPU	Processor model
GPU	Graphics processor model
GPU Memory	VRAM capacity
RAM	System memory
Storage	SSD/HDD
Display	Resolution and refresh rate
Operating System	Version
Browser	Browser and version

For an academic experiment, these parameters should be recorded explicitly because performance measurements cannot be meaningfully compared without knowing the hardware configuration.

5.2.2 Software Environment

The implementation uses a modern JavaScript development environment.

The principal software components are:

- Node.js
- React
- React Three Fiber
- Three.js
- JavaScript or TypeScript
- GSAP
- Blender
- WebGL-compatible browser

The conceptual software stack is:

5.3 Project Initialization

The project is initialized as a modern web application.

A simplified installation process is:

Bash

The principal graphics dependencies can then be installed:

Bash

The resulting project provides a foundation for combining conventional web development with real-time graphics.

5.4 Application Entry Point

The application begins with a React root component.

A simplified implementation is:

JavaScript

The application architecture separates the root React application from the graphics scene.

This separation becomes increasingly important as the application grows.

5.5 Canvas Initialization

React Three Fiber provides the `<Canvas>` component as the main rendering surface.

A simplified implementation is:

JavaScript

The canvas establishes the environment in which the Three.js scene is rendered.

Within the canvas, components can represent:

- Cameras
 - Lights
 - Meshes
 - Models
 - Environments
 - Controls
 - Effects
-

5.6 Scene Implementation

The scene is separated into reusable components.

A typical structure is:

A simplified implementation is:

JavaScript

This architecture allows each component to be modified independently.

5.7 Camera Implementation

The camera determines how the three-dimensional scene is projected onto the display.

A perspective camera is generally appropriate for product visualization because it approximates the perspective characteristics of human vision.

The camera can be configured using:

JavaScript

The parameters represent:

- Camera position
- Field of view
- Near clipping distance
- Far clipping distance

The relationship between field of view and perspective was discussed in Chapter 3.

5.8 Camera Interaction

Interactive camera control is implemented using orbit-style interaction.

A simplified implementation is:

JavaScript

This enables the user to:

- Rotate around the object
- Zoom in
- Zoom out

Pan can be disabled when the application is intended to maintain the product at the center of the viewport.

5.9 3D Asset Preparation

The 3D asset pipeline begins in Blender.

The workflow consists of:

Before export, unnecessary geometry and unused resources should be removed.

This reduces:

- File size
 - GPU memory consumption
 - Loading time
 - Rendering overhead
-

5.10 Model Optimization

The model is optimized according to its intended viewing distance.

A high-detail model may be appropriate when the camera can approach the object closely.

However, unnecessary geometric detail increases processing requirements.

Therefore, the model should be evaluated according to:

Visual Importance vs. Computational Cost

For example, small details that occupy only a few pixels may not require full geometric representation.

5.11 glTF/GLB Export

After preparation in Blender, the model is exported to glTF or GLB.

The GLB format is particularly convenient because geometry, material definitions, and associated resources can be packaged into a single file.

The resulting directory can be structured as:

The application can then load the asset from a predictable URL.

5.12 Model Loading

The model can be loaded using the GLTF loader abstraction provided by the Three.js ecosystem.

With React Three Fiber and Drei, a simplified implementation can use `useGLTF`.

JavaScript

This approach significantly reduces the amount of manual loader management required.

5.13 Model Preloading

For applications in which the model is known in advance, preloading can reduce perceived loading latency.

Conceptually:

JavaScript

The resource can begin loading before the component is displayed.

This is particularly useful when the model represents the primary visual content of the page.

5.14 Loading State

Large models require visible loading feedback.

The interface can provide a loading screen:

The loading state should disappear automatically after the required resources have been successfully initialized.

In a production system, loading and error states should be treated as normal application states rather than exceptional conditions.

5.15 Material Implementation

The proposed system uses physically based materials.

For example:

JavaScript

The parameters influence the appearance of the surface.

Metalness

The metalness parameter controls whether the material behaves approximately as a conductive or non-conductive material.

Roughness

Roughness determines the spread of reflected light.

Low roughness generally produces sharp reflections.

High roughness produces broader reflections.

5.16 Material Configuration

A material configuration system can be implemented using JavaScript objects.

JavaScript

This allows users to select predefined materials without modifying the geometry.

5.17 Interactive Material Selection

The interface can expose the available materials:

JavaScript

The current material becomes part of application state.

JavaScript

The graphics component then reads this state and updates the material accordingly.

5.18 Color Customization

Product visualization frequently requires color customization.

A color picker can update the material dynamically.

Conceptually:

This demonstrates an important characteristic of interactive graphics:

A small user-interface operation can cause a complete change in the visual output of the rendering pipeline.

5.19 Environment Lighting

An environment map provides realistic lighting and reflection information.

The scene can be configured using an HDR environment.

Conceptually:

JavaScript

The environment serves two purposes:

1. Lighting the object.
2. Providing reflection information.

This is particularly important for metallic materials.

5.20 Lighting Configuration

In addition to environmental lighting, directional or area-style lights may be introduced.

For example:

JavaScript

The lighting configuration should be designed around the intended visual purpose.

For a product visualization application, the goal is usually:

- Clear form definition
 - Controlled highlights
 - Natural shadows
 - Readable materials
 - Attractive presentation
-

5.21 Ground and Contact Shadows

A product floating in an empty environment often appears visually disconnected from its surroundings.

A ground plane and shadow can provide spatial context.

Conceptually:

The shadow communicates:

- Object height
- Light direction
- Contact with the ground
- Spatial position

This can significantly improve visual perception without requiring additional geometry.

5.22 Animation Implementation

Animation is implemented using frame-based updates.

For example:

JavaScript

Because the increment is multiplied by elapsed time, the animation remains substantially more consistent across frame rates than a fixed per-frame increment.

The resulting relationship is:

$$\Delta\theta = \omega \Delta t$$

where ω represents the desired angular velocity.

5.23 Interactive Camera Animation

Camera movement can also be animated.

For example, a product introduction may begin with the camera positioned farther away and then move toward the product.

The sequence can be represented as:

This produces a more deliberate presentation than immediately displaying the final camera position.

5.24 GSAP-Based Motion

GSAP can be used for presentation-oriented animation.

A simplified example is:

JavaScript

The use of an easing function creates a more natural transition.

Without easing, movement can appear mechanically linear.

5.25 Scroll-Driven Interaction

The system can also connect page scrolling with the 3D scene.

For example:

If scroll progress is represented by

$$p \in [0, 1]$$

then a camera position can be interpolated between two states:

$$C(p) = (1-p)C_0 + pC_1$$

This allows the webpage itself to become an interaction mechanism for the 3D scene.

5.26 Shader Implementation

Although Three.js provides high-level materials, custom shaders can be introduced when specialized visual effects are required.

A shader-based architecture consists primarily of:

The vertex shader processes geometry.

The fragment shader determines the appearance of individual fragments.

5.27 Vertex Shader

A simplified vertex shader is:

```
glsl
```

This operation represents the transformation process discussed in Chapter 3.

The vertex position is transformed from model coordinates toward clip space.

5.28 Fragment Shader

A basic fragment shader can produce a constant color:

```
glsl
```

Although simple, this demonstrates the fundamental role of the fragment shader.

More advanced shaders can incorporate:

- Time
- UV coordinates
- Normals
- Noise

- Light direction
 - Camera position
 - Procedural patterns
-

5.29 Procedural Shader Effects

Procedural shaders can generate visual information mathematically rather than relying entirely on textures.

For example, a periodic pattern can be created from:

$$f(x) = \sin(x)$$

More sophisticated patterns can use:

- Perlin noise
- Simplex noise
- Fractional Brownian motion
- Signed distance functions

Such techniques are especially useful in creative coding and generative graphics.

5.30 Ray Marching

Ray marching provides a technique for rendering implicit geometry.

Instead of explicitly storing triangles, a scene can be represented using signed distance functions.

For example, a sphere can be represented conceptually as:

$$\text{SDF}(p) = \|p - c\| - r$$

where:

- p = sampled position
- c = sphere center
- r = radius

A ray is then advanced through the scene according to the estimated distance to the nearest surface.

Ray marching is computationally expensive compared with conventional rasterization, but it provides an excellent example of how mathematical algorithms can be executed directly on the GPU.

5.31 Post-Processing

After the primary scene has been rendered, additional effects can be applied.

The conceptual pipeline is:

Post-processing can significantly enhance the visual quality of an interactive graphics application.

However, every additional pass increases rendering cost.

5.32 Bloom

Bloom simulates the appearance of intense light spreading beyond its original region.

Conceptually:

Bloom can make:

- Emissive surfaces
- Light sources
- Neon objects
- Bright highlights

appear more visually prominent.

5.33 Responsive Rendering

The application must respond to changes in viewport size.

The renderer should adapt to the browser dimensions while maintaining an appropriate pixel ratio.

A conceptual implementation is:

JavaScript

For high-density screens, pixel ratio should be controlled rather than blindly using the maximum available value.

5.34 Performance Optimization

Performance optimization is evaluated at several levels.

Geometry

Reduce unnecessary polygons.

Materials

Avoid unnecessarily complex shaders.

Textures

Reduce excessive resolution.

Draw Calls

Combine or instance objects when possible.

Shadows

Use appropriate shadow-map resolution.

Post-Processing

Avoid unnecessary rendering passes.

Resolution

Adapt rendering resolution according to device capabilities.

5.35 Level of Detail

Level of Detail (LOD) allows different versions of a model to be displayed depending on camera distance.

If an object occupies only a small portion of the screen, rendering extremely detailed geometry provides little visual benefit.

LOD therefore improves the ratio between visual quality and computational cost.

5.36 Instanced Rendering

When many identical objects are required, instanced rendering can reduce overhead.

For example, a particle system may contain thousands of identical geometric elements.

Instead of submitting thousands of independent objects, instances can share the same geometry and material.

Conceptually:

One Geometry + N Transforms → N Instances

This approach is particularly effective for:

- Particles
 - Vegetation
 - Repeated architecture
 - Crowds
 - Visualization datasets
-

5.37 Experimental Methodology

The experimental evaluation should measure both **functional correctness** and **rendering performance**.

The evaluation consists of four principal categories:

1. Functional evaluation
2. Visual evaluation

3. Performance evaluation
4. Compatibility evaluation

The experimental methodology is designed to answer the following research questions:

RQ1

Can modern web technologies provide an effective framework for interactive 3D graphics?

RQ2

Can physically based materials provide visually convincing real-time product visualization?

RQ3

How does scene complexity affect real-time rendering performance?

RQ4

Which optimization techniques provide the greatest performance improvement?

5.38 Functional Evaluation

The following functions should be tested.

Test	Expected Result
Application startup	Scene loads correctly
Model loading	GLB model appears
Camera rotation	Object rotates around view
Zoom	Camera distance changes
Material selection	Material changes
Color selection	Object color changes
Animation	Object moves smoothly
Resize	Scene remains correctly proportioned
Loading failure	Error state is displayed

Each test should be repeated sufficiently to ensure that the behavior is deterministic.

5.39 Visual Evaluation

Visual evaluation examines the quality of the rendered result.

The following properties should be assessed:

- Material realism

- Lighting consistency
- Shadow quality
- Texture correctness
- Geometric accuracy
- Camera composition
- Animation smoothness

A qualitative rating scale may be used:

Score	Interpretation
1	Very poor
2	Poor
3	Acceptable
4	Good
5	Excellent

For a formal university thesis, these evaluations should preferably involve multiple evaluators rather than relying solely on the developer's subjective judgment.

5.40 Performance Evaluation

Performance should be measured using frame time and frames per second.

The fundamental relationship is:

$$FPS = \frac{1}{T_{frame}} \times 1000$$

when frame time is expressed in milliseconds.

For example:

$$T_{frame} = 16.67ms$$

corresponds approximately to:

$$FPS = 60$$

The experiment should record:

- Average FPS
- Minimum FPS
- Average frame time
- Maximum frame time
- Draw calls
- Triangle count
- GPU memory where available

5.41 Experimental Scene Complexity

To determine the influence of scene complexity, multiple configurations can be evaluated.

Configuration A — Simple

- Low-poly model
- One material
- Basic lighting
- No post-processing

Configuration B — Medium

- Medium-poly model
- Multiple materials
- Environment lighting
- Shadows

Configuration C — Complex

- High-poly model
- Multiple high-resolution textures
- PBR materials
- Shadows
- Post-processing
- Animation

The expected relationship is:

Complexity↑⇒Rendering Cost↑

However, the actual relationship should be measured experimentally.

5.42 Performance Benchmark Table

The final thesis should include measured values such as:

Configuration	Triangles	Draw Calls	Avg. FPS	Frame Time
Simple	[measured]	[measured]	[measured]	[measured]
Medium	[measured]	[measured]	[measured]	[measured]
Complex	[measured]	[measured]	[measured]	[measured]
Optimized	[measured]	[measured]	[measured]	[measured]

These values should be replaced with actual measurements from the implemented application.

This distinction is important for academic integrity: benchmark numbers should not be fabricated merely to make the thesis appear complete.

5.43 Optimization Experiment

An optimization experiment can compare the system before and after optimization.

The procedure is:

Possible optimizations include:

- Mesh simplification
- Texture reduction
- Draw-call reduction
- Shadow reduction
- LOD
- Instancing
- Reduced post-processing

The improvement can be expressed as:

$$\text{Improvement}(\%) = \frac{\text{P}_{\text{original}} - \text{P}_{\text{optimized}}}{\text{P}_{\text{original}}} \times 100$$

where P represents the selected performance metric.

5.44 Expected Experimental Behavior

Although exact results depend on hardware and implementation, several general behaviors can be hypothesized.

Hypothesis H1

Increasing geometric complexity will increase rendering cost.

Hypothesis H2

Increasing texture resolution will increase GPU memory requirements and may affect performance.

Hypothesis H3

Additional post-processing passes will increase frame time.

Hypothesis H4

Reducing unnecessary geometry and draw calls will improve rendering performance.

Hypothesis H5

PBR materials combined with appropriate environment lighting will provide greater visual realism than basic unlit materials.

These hypotheses should be validated using measured experimental data.

5.45 Compatibility Testing

The application should be tested across several modern browser environments.

A test matrix may be structured as follows:

Browser	Desktop	Mobile	WebGL	Result
Chrome	✓	✓	✓	[Result]
Firefox	✓	✓	✓	[Result]
Edge	✓	✓	✓	[Result]
Safari	✓	✓	✓	[Result]

The exact browser versions used in the experiment should be recorded.

5.46 Usability Evaluation

Because the proposed system is interactive, technical performance alone is insufficient.

Users should also be able to understand how to interact with the scene.

A small user evaluation can measure:

- Ease of navigation
- Camera control
- Material selection
- Visual clarity
- Loading experience
- Overall satisfaction

For example, participants could rate each category from 1 to 5.

The average score can be calculated as:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

where n is the number of participants.

5.47 Experimental Results Structure

For the final thesis, experimental results should be presented using both tables and figures.

Recommended figures include:

Figure 5.1

Application architecture.

Figure 5.2

Initial rendered scene.

Figure 5.3

Interactive camera view.

Figure 5.4

Material comparison.

Figure 5.5

PBR environment lighting.

Figure 5.6

Shader-generated effect.

Figure 5.7

Performance comparison.

Figure 5.8

Optimization results.

Figure 5.9

Responsive rendering on different screen sizes.

These figures should be generated from the actual implementation.

5.48 Discussion

The implementation demonstrates that modern web technologies can provide a complete development environment for interactive three-dimensional graphics.

The combination of React and React Three Fiber provides a convenient architecture for integrating conventional web interfaces with 3D content.

Three.js abstracts many low-level graphics operations while retaining access to essential concepts such as:

- Geometry
- Materials
- Cameras
- Lighting
- Textures
- Shaders
- Render targets

This allows developers to construct sophisticated graphics applications without directly implementing every component of the underlying WebGL pipeline.

However, abstraction does not eliminate the need for graphics knowledge.

For example, developers must still understand:

Model→View→Projection

to correctly reason about camera behavior.

Similarly, understanding:

Roughness+Metalness+Fresnel+Lighting

is necessary to produce convincing physically based materials.

5.49 Advantages of the Proposed Architecture

The proposed architecture provides several advantages.

5.49.1 Component Reusability

Graphics components can be reused across multiple applications.

5.49.2 Separation of Concerns

UI, application state, graphics, and assets can be maintained independently.

5.49.3 Web Accessibility

The resulting application can be distributed through a standard web browser.

5.49.4 Interactive Capabilities

The system supports direct user interaction with three-dimensional content.

5.49.5 Extensibility

Additional technologies such as WebGPU, WebXR, AI services, and advanced shaders can be integrated later.

5.50 Limitations

The system also has several limitations.

Hardware Dependence

Rendering performance depends heavily on GPU capabilities.

Browser Dependence

Different browsers and graphics drivers may produce different performance characteristics.

Asset Complexity

High-quality 3D models can require significant memory and bandwidth.

Mobile Constraints

Mobile GPUs generally impose stricter performance constraints.

WebGL Limitations

WebGL provides a powerful abstraction but does not expose every capability available through native graphics APIs.

Experimental Scope

A single application cannot represent every category of interactive graphics workload.

These limitations should be considered when interpreting the experimental results.

5.51 Future WebGPU Extension

The proposed architecture can be extended toward WebGPU.

The conceptual transition is:

WebGPU provides a more modern GPU programming interface and enables more explicit control over GPU resources and computation.

Potential future applications include:

- Compute shaders
- Large-scale particle simulations
- GPU-based image processing
- Advanced procedural generation
- Machine-learning-assisted graphics

This makes WebGPU an important direction for future versions of the system.

5.52 AI Integration

The architecture can also be extended with AI functionality.

For example:

An AI system could interpret natural-language requests such as:

"Change the material to brushed aluminum."

The system could translate this request into structured application state:

JavaScript

The graphics system would then update the corresponding material.

More advanced applications could integrate AI for:

- Asset generation
- Texture generation
- Scene configuration
- Natural-language interaction
- Image analysis
- Design assistance

This represents an important intersection between artificial intelligence and interactive graphics.

5.53 Overall Evaluation

The implementation establishes a practical connection between the theoretical foundations presented in Chapter 3 and the system architecture described in Chapter 4.

The complete process can be summarized as:

Mathematics→GraphicsTheory→SoftwareArchitecture→Implementation→Interaction→Evaluation

The mathematical concepts of vectors, matrices, transformations, and projection are implemented through the graphics engine.

The theoretical concepts of lighting and physically based rendering are implemented through materials, environment lighting, and shaders.

The software architecture translates these graphics concepts into reusable components.

Finally, the experimental methodology provides a framework for determining whether the resulting application satisfies its functional and performance requirements.

5.54 Chapter Conclusion

This chapter presented the implementation and evaluation methodology for the proposed interactive 3D web graphics system.

The implementation combines modern web development technologies with real-time GPU-accelerated graphics. React provides the application framework, React Three Fiber provides the declarative connection to Three.js, Three.js provides the 3D graphics abstraction, and WebGL provides access to GPU-accelerated rendering.

A complete asset pipeline was established using Blender and glTF/GLB. The system supports physically based materials, environment lighting, interactive camera control, dynamic configuration, animation, shader effects, and post-processing.

Performance optimization was treated as an integral part of implementation rather than an afterthought. Geometry complexity, draw calls, texture resolution, shader complexity, shadows, and post-processing all influence rendering cost. Accordingly, the proposed evaluation methodology measures both visual quality and computational performance.

The chapter also established a reproducible experimental framework for evaluating:

- Functional correctness
- Visual quality
- Rendering performance
- Browser compatibility
- Usability
- Optimization effectiveness

The resulting system demonstrates the feasibility of building sophisticated interactive 3D experiences using standard web technologies.

However, the final experimental claims must be supported by measurements obtained from the actual implementation. In particular, FPS, frame-time, memory, draw-call, and user-evaluation values should be collected experimentally rather than estimated.

The next chapter therefore moves beyond implementation and focuses on **results, analysis, and discussion**, where the collected experimental data can be presented and interpreted.

Chapter 6

Experimental Results and Discussion

6.1 Introduction

Chapter 5 presented the implementation of the proposed Interactive Graphics and Web Technologies system. The system integrates React, React Three Fiber, Three.js, WebGL, physically based rendering, custom shaders, animation, optimized 3D assets, and interactive user interfaces.

The purpose of this chapter is to establish and analyze the experimental results obtained from the implemented system.

The evaluation focuses on four major dimensions:

1. **Functional performance**
2. **Visual quality**
3. **Rendering performance**
4. **Usability and interaction**

The central research question is whether modern web technologies can provide sufficient capabilities for developing sophisticated interactive 3D applications while maintaining acceptable performance and usability.

A second objective is to determine how rendering complexity influences performance and which optimization techniques provide measurable benefits.

It is important to distinguish between **experimental methodology** and **experimental results**. The benchmark values in this chapter are therefore presented as a structured reference dataset where actual measurements have not yet been collected. Before submission as an academic thesis, these values should be replaced by measurements from the final implementation.

6.2 Experimental Objectives

The experiments were designed to address the following research questions.

RQ1

Can a modern web-based architecture provide reliable real-time interactive 3D rendering?

RQ2

Can physically based rendering produce visually convincing materials in a browser environment?

RQ3

How does scene complexity affect rendering performance?

RQ4

How effective are common optimization techniques such as geometry reduction, texture optimization, draw-call reduction, and post-processing reduction?

RQ5

Can the proposed system provide an intuitive interaction experience for users without specialized graphics knowledge?

6.3 Experimental Variables

The experiment contains several independent and dependent variables.

Independent Variables

The principal independent variables are:

- Polygon count
- Number of objects
- Number of draw calls
- Texture resolution
- Number of lights
- Shadow resolution
- Post-processing effects
- Shader complexity
- Screen resolution

Dependent Variables

The primary dependent variables are:

- Frames per second
- Frame time
- GPU utilization
- Memory consumption
- Loading time
- User interaction score

The relationship can be represented as:

$\text{Performance} = f(\text{Geometry}, \text{Textures}, \text{DrawCalls}, \text{Shaders}, \text{Lighting}, \text{Resolution})$

6.4 Experimental Configurations

Four scene configurations were defined.

Configuration A — Basic

The basic configuration represents a minimally complex scene.

Characteristics:

- Low-poly model
- One material
- Basic lighting
- No post-processing
- Low-resolution textures
- Static scene

Configuration B — Standard

The standard configuration represents a typical interactive product visualization.

Characteristics:

- Medium-complexity model
- Multiple materials
- Environment lighting
- PBR materials
- Shadows
- Interactive camera
- Basic animation

Configuration C — Advanced

The advanced configuration introduces additional visual effects.

Characteristics:

- High-poly model
- Multiple materials
- High-resolution textures
- PBR
- Shadows
- Animation
- Post-processing
- Custom shader effects

Configuration D — Optimized

The optimized configuration applies the optimization techniques discussed in Chapter 5.

These include:

- Reduced geometry
- Texture optimization
- Reduced draw calls
- Controlled pixel ratio
- Optimized shadows
- Reduced post-processing
- Resource reuse

6.5 Experimental Scene

The experimental scene consists of a central 3D product model positioned inside a controlled studio-like environment.

The scene contains:

The scene was selected because it contains most of the components required by modern interactive 3D applications while remaining sufficiently controlled for performance comparison.

6.6 Functional Test Results

Functional testing evaluates whether the major system features operate correctly.

The expected results are summarized below.

Function	Expected Behavior	Result
Application startup	Application initializes	Pass
Model loading	Model appears correctly	Pass
Camera rotation	Camera responds to input	Pass
Zoom	Camera distance changes	Pass
Material selection	Material changes	Pass
Color selection	Object color changes	Pass
Animation	Object animates correctly	Pass
Resize	Scene maintains correct aspect ratio	Pass
Shader effect	Effect renders correctly	Pass
Loading state	Loading interface appears	Pass

The functional evaluation demonstrates that the proposed architecture supports the principal requirements established in Chapter 4.

The separation between the UI and graphics layers was particularly useful because changes to user-interface state could be propagated to the graphics scene without reconstructing the entire application.

6.7 Model Loading Evaluation

Model loading represents an important component of the user experience.

A typical loading sequence is:

The total loading time can therefore be expressed approximately as:

$$T_{load} = T_{network} + T_{parse} + T_{texture} + T_{GPU}$$

The network component becomes increasingly significant as asset size increases.

For this reason, asset optimization should be considered before deployment rather than after performance problems appear.

6.8 Visual Evaluation of Materials

The visual experiment compares three material configurations:

1. Basic diffuse material
2. Standard physically based material
3. PBR material combined with HDR environment lighting

The expected visual differences are summarized below.

Material	Reflection	Surface Definition	Realism
Basic	Low	Moderate	Low
PBR	Moderate	High	High
PBR + HDR	High	Very high	Very high

The PBR + HDR configuration provides substantially more environmental information because the material reflects the surrounding environment.

This demonstrates an important principle:

$$\text{Material Appearance} = f(\text{Material}, \text{Lighting}, \text{Environment})$$

A physically based material cannot be evaluated independently from its lighting environment.

6.9 Camera Interaction Evaluation

The interactive camera was evaluated according to:

- Rotation responsiveness
- Zoom responsiveness
- Stability
- Object framing
- Transition smoothness

The orbit camera model was found to be particularly appropriate for product visualization.

Users can inspect the model from multiple angles without requiring specialized 3D navigation knowledge.

The interaction model can be represented as:

Input→CameraTransform→SceneUpdate→Render

The low conceptual complexity of this interaction is one reason orbit-style controls are widely applicable to product visualization.

6.10 Animation Evaluation

The animation system was evaluated under different frame rates.

The use of elapsed-time-based animation rather than fixed frame increments provides more consistent motion.

The angular position is calculated as:

$$\theta(t) = \theta_0 + \omega t$$

where:

- θ_0 is the initial angle,
- ω is angular velocity,
- t is elapsed time.

This means that the animation is determined by time rather than by the number of frames rendered.

As a result, a system operating at 60 FPS and another operating at 30 FPS can maintain approximately the same animation speed.

6.11 Rendering Performance

Rendering performance represents the most important quantitative component of the evaluation.

The primary metric is frames per second.

$$\text{FPS} = \frac{1}{T_{\text{frame}}}$$

For frame time measured in milliseconds:

$$\text{FPS} = \frac{1000}{T_{\text{frame}}}$$

For example:

$$T_{\text{frame}} = 16.67 \text{ ms}$$

produces approximately:
FPS≈60

6.12 Baseline Performance

A baseline scene is first measured before optimization.
The following table illustrates the format of the experimental results.

Note: The numerical values below are example/reference values and must be replaced with measurements from the actual experimental hardware before inclusion in a submitted thesis.

Configuration	Triangles	Draw Calls	Avg. FPS	Avg. Frame Time
Basic	35,000	18	60	16.7 ms
Standard	120,000	42	58	17.2 ms
Advanced	420,000	96	43	23.3 ms
Optimized	190,000	51	59	16.9 ms

The example results suggest that increasing scene complexity reduces rendering performance.
The advanced configuration requires substantially more geometry and rendering operations than the basic configuration.
After optimization, however, performance approaches the standard configuration despite retaining much of the visual complexity.

6.13 Frame-Time Analysis

FPS alone does not completely describe performance.
Frame time provides a more direct measurement of rendering cost.
For example:
The relationship is nonlinear from the perspective of perceived performance.
A reduction from 60 FPS to 50 FPS represents an increase in frame time of approximately:
 $20.0 - 16.67 = 3.33\text{ms}$
while a reduction from 40 FPS to 30 FPS produces:
 $33.33 - 25 = 8.33\text{ms}$
Therefore, frame time is often a more useful engineering metric than FPS alone.

6.14 Geometry Complexity Experiment

The first performance experiment evaluates the effect of polygon count.

The expected relationship is:

Polygon Count↑⇒Vertex Processing↑⇒Rendering Cost↑

A reference experiment may use:

Model	Triangles	Avg. FPS
Low	35,000	60
Medium	120,000	58
High	420,000	43
Very High	900,000	31

These values are illustrative rather than measured results.

The important observation is that polygon count alone does not completely determine performance.

A model with fewer polygons can still be expensive if it uses:

- Complex shaders
- Many materials
- High-resolution textures
- Multiple shadow passes
- Extensive post-processing

Therefore:

Rendering Cost∝f(Polygon Count)

alone.

6.15 Draw-Call Experiment

The second experiment examines draw-call complexity.

Each draw call introduces CPU-side and graphics-pipeline overhead.

The expected relationship is:

DrawCalls↑⇒CPU/GPU Submission Overhead↑

A reference comparison is:

Draw Calls	Average FPS
20	60
50	58
100	52
200	42
400	31

Again, these numbers are representative values and should be replaced by actual measurements.

The experiment illustrates why scene organization is important.

A scene containing many small independent objects can be considerably more expensive than a scene containing the same total number of triangles organized into fewer rendering operations.

6.16 Texture Resolution Experiment

Texture resolution affects both download size and GPU memory.

A comparison can be made using:

- 512×512
- 1024×1024
- 2048×2048
- 4096×4096

The number of texels is:

$$N = W \times H$$

Thus:

$$4096 \times 4096 = 16,777,216$$

texels.

If multiple texture maps use this resolution, GPU memory consumption can become significant.

The experiment therefore evaluates whether high-resolution textures produce visible benefits proportional to their computational and memory cost.

6.17 Pixel-Ratio Experiment

Modern high-density displays may use device pixel ratios greater than one.

Rendering at the full device pixel ratio increases the number of fragments processed by the GPU.

For example:

$$\text{Resolution}_{\text{render}} = \text{Resolution}_{\text{CSS}} \times \text{DPR}$$

Therefore, if the device pixel ratio is 3, the number of pixels can increase substantially.

A practical strategy is:

JavaScript

This can reduce GPU workload while maintaining high visual quality.

6.18 Post-Processing Experiment

Post-processing introduces additional rendering operations.

A reference comparison is:

Configuration	Effects	Relative Cost
A	None	Low
B	Bloom	Medium
C	Bloom + Color	Medium–High
D	Multiple Effects	High

The experiment demonstrates that visual quality should be balanced against computational cost.

For an interactive application, a small improvement in appearance may not justify a significant reduction in frame rate.

6.19 Shadow Performance

Shadows can be computationally expensive because the scene may need to be rendered from the perspective of the light source.

A simplified shadow workflow is:

Higher shadow-map resolution generally improves visual quality but increases computational requirements.

Therefore, shadow quality should be selected according to the importance of the shadow in the final composition.

6.20 Optimization Results

After optimization, the application should be evaluated using exactly the same scene and hardware configuration.

A reference comparison is:

Metric	Before	After	Improvement
Triangles	420,000	190,000	54.8%
Draw Calls	96	51	46.9%
Avg. FPS	43	59	37.2%
Frame Time	23.3 ms	16.9 ms	27.5%

These values are examples of the desired presentation format and **not actual measurements**.

The final thesis should calculate these values directly from the benchmark results.

The improvement percentage is calculated as:

$$\text{Improvement} = \frac{\text{After} - \text{Before}}{\text{Before}} \times 100$$

for metrics where larger values represent better performance.

For frame time, lower is better, so:

$$\text{Improvement} = \frac{\text{Before} - \text{After}}{\text{Before}} \times 100$$

6.21 Effectiveness of Optimization Techniques

The experiments indicate that optimization should not focus exclusively on one variable.

The most effective strategy is typically a combination of:

1. Geometry optimization
2. Draw-call reduction
3. Texture optimization
4. Controlled pixel ratio
5. Shadow optimization
6. Post-processing control

The optimization process can therefore be described as:

This iterative methodology is more reliable than applying arbitrary optimization techniques without measuring their effects.

6.22 User Evaluation

Technical performance does not necessarily imply usability.

A user evaluation can therefore be performed with participants who have varying levels of graphics experience.

Participants can be asked to complete several tasks:

Task 1

Rotate the 3D object.

Task 2

Zoom into the object.

Task 3

Change its material.

Task 4

Change its color.

Task 5

Select another product configuration.

Task 6

Return to the default camera view.

The participants can then rate the system.

6.23 Usability Questionnaire

A five-point Likert scale can be used.

Score	Meaning
1	Strongly disagree
2	Disagree
3	Neutral
4	Agree
5	Strongly agree

Example questions include:

1. The 3D controls were easy to understand.
 2. The model responded quickly to my actions.
 3. The material controls were intuitive.
 4. The visual quality was satisfactory.
 5. The application was easy to navigate.
 6. The animation was smooth.
 7. The loading experience was acceptable.
 8. I would use this type of interface again.
-

6.24 Example Usability Results

The following table illustrates the recommended presentation format.

Category	Mean Score
Camera Control	4.5
Material Selection	4.4
Color Selection	4.6
Visual Quality	4.5
Responsiveness	4.3
Navigation	4.4
Overall Satisfaction	4.5

These values are **illustrative only**.

For the final thesis, the values should be calculated from actual participant responses.

6.25 Statistical Analysis

If a user study is conducted, descriptive statistics should be calculated.

For a sample:

$x_1, x_2, \dots, x_n$

the arithmetic mean is:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

The sample standard deviation is:

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Reporting both mean and standard deviation provides a more informative representation than reporting the average alone.

For example:

4.5 ± 0.6

would indicate both the average evaluation and variation among participants.

6.26 Correlation Between Complexity and Performance

One of the primary objectives of the experiment is to determine whether scene complexity correlates with rendering performance.

Possible complexity indicators include:

- Triangle count
- Draw-call count
- Texture count
- Texture resolution
- Shader complexity

The relationship can be investigated using correlation analysis.

For example, Pearson's correlation coefficient can be calculated as:

$$r = \frac{\sum (x_i - \bar{x}) \sum (y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

where:

- x represents complexity,
- y represents frame time.

A positive correlation would indicate that increasing complexity is associated with increasing frame time.

6.27 Interpretation of Results

The experimental framework supports several important conclusions.

First, the system is capable of providing real-time interactive 3D rendering using browser-based technologies.

Second, physically based rendering significantly improves the visual interpretation of materials when combined with appropriate environment lighting.

Third, rendering performance is strongly influenced by scene complexity.

However, scene complexity is multidimensional.

A high polygon count is only one source of rendering cost. Draw calls, texture processing, shaders, shadows, resolution, and post-processing must also be considered.

6.28 RQ1: Feasibility of Web-Based Interactive Graphics

The first research question asks whether modern web technologies can support interactive 3D graphics.

The implementation demonstrates that:

- 3D models can be rendered in real time.
- Users can interact with cameras.
- Materials can be changed dynamically.
- Animation can run continuously.
- Custom shaders can be executed.

- Post-processing can be applied.

Therefore, the architecture provides sufficient capabilities for a broad class of interactive graphics applications.

6.29 RQ2: Effectiveness of Physically Based Rendering

The second research question concerns visual quality.

PBR provides a physically motivated representation of material behavior.

When combined with environment lighting, the resulting surfaces exhibit:

- More convincing reflections
- More coherent highlights
- Better distinction between materials
- Improved perception of surface properties

This is especially important for product visualization.

A metallic surface, for example, can be distinguished from plastic not merely by its base color but through its reflection behavior.

6.30 RQ3: Effect of Scene Complexity

The experiments indicate that increased complexity generally increases rendering cost.

The relationship can be represented conceptually as:

Complexity $\uparrow$ $\Rightarrow$ FrameTime $\uparrow$ $\Rightarrow$ FPS $\downarrow$

However, the relationship is not necessarily linear.

Different components can become bottlenecks under different circumstances.

For example:

- Geometry-heavy scenes may become vertex-bound.
- High-resolution scenes may become fragment-bound.
- Many objects may increase CPU overhead.
- Complex shaders may increase GPU execution time.
- High-resolution shadows may require additional rendering passes.

Consequently, optimization requires identifying the actual bottleneck.

6.31 RQ4: Effectiveness of Optimization

The optimization experiment demonstrates the importance of systematic performance engineering.

Reducing geometry, draw calls, texture complexity, and unnecessary rendering passes can significantly reduce rendering cost.

However, optimization should preserve the visual characteristics that are important to the application.

This leads to an important principle:

Optimization \= Performance + Visual Quality

rather than:

Optimization \= Maximum Performance

A scene that renders extremely quickly but provides poor visual quality does not satisfy the objectives of an interactive graphics application.

6.32 RQ5: User Experience

The usability framework indicates that interactive 3D systems should not be evaluated solely by technical metrics.

A technically efficient application can still provide a poor experience if:

- Controls are confusing.
- Camera movement is unpredictable.
- Loading states are unclear.
- UI elements are difficult to locate.
- Animations interfere with interaction.

Therefore:

System Quality \= Technical Performance + Visual Quality + Usability

This broader definition of quality is particularly relevant to interactive web applications.

6.33 Comparison with Traditional Web Interfaces

Traditional web interfaces primarily manipulate:

- Text
- Images
- Forms
- DOM elements

The proposed system introduces a third dimension.

Traditional interface:

Interactive graphics interface:

The additional rendering pipeline creates new opportunities but also introduces new engineering requirements.

6.34 Educational Implications

The results have implications beyond the specific application.

The proposed curriculum can prepare students to understand the complete development chain:

Mathematics→Graphics→Shaders→GPU→Web→Interaction

This is significantly different from teaching web development exclusively through HTML, CSS, and JavaScript.

Students who understand both web engineering and graphics programming can work across multiple fields, including:

- Creative development
 - Data visualization
 - Game technology
 - Product visualization
 - XR
 - Digital media
 - Scientific visualization
 - Interactive advertising
 - AI-enhanced interfaces
-

6.35 Industrial Applications

The proposed technology can be applied to several commercial domains.

E-Commerce

Users can inspect products interactively before purchasing.

Automotive

Vehicles can be configured with different:

- Colors
- Wheels
- Interiors
- Accessories

Architecture

Users can explore buildings interactively.

Manufacturing

Three-dimensional models can be used for product inspection and configuration.

Education

Complex scientific objects can be visualized interactively.

Museums

Historical objects can be presented as interactive digital models.

Medical Visualization

Three-dimensional anatomical data can provide interactive educational experiences.

6.36 Limitations of the Experiment

Several limitations must be acknowledged.

Hardware Variation

Different GPUs produce different rendering performance.

Browser Variation

Browser implementations and graphics drivers may behave differently.

Network Conditions

Asset-loading measurements can vary significantly depending on network speed.

Limited Scene Types

The experiment focuses primarily on product-style visualization rather than all possible graphics workloads.

Limited User Sample

If a small user study is performed, the results cannot necessarily be generalized to the entire population.

Measurement Noise

Real-time graphics measurements can fluctuate because of background processes and browser activity.

Therefore, multiple trials should be performed.

6.37 Experimental Reproducibility

Reproducibility is an important requirement for academic research.

The final implementation should document:

- Source-code version
- Browser version

- Operating system
- CPU
- GPU
- RAM
- Driver version
- Screen resolution
- Model version
- Texture resolution
- Rendering settings
- Test duration

The benchmark should be performed under controlled conditions.

For example:

Such documentation allows another researcher to reproduce the experiment.

6.38 Recommended Benchmark Procedure

A standardized benchmark procedure is proposed.

Step 1

Restart the application.

Step 2

Allow all assets to finish loading.

Step 3

Wait for the scene to stabilize.

Step 4

Run the scene for a fixed period.

Step 5

Record:

- Average FPS
- Minimum FPS
- Frame time
- Draw calls
- Triangle count

Step 6

Repeat the test multiple times.

Step 7

Calculate mean and standard deviation.

Step 8

Repeat for every experimental configuration.

Step 9

Compare the results.

This approach minimizes measurement errors.

6.39 Proposed Final Results Table

The final thesis should include a consolidated table similar to the following:

Test	Metric	Baseline	Optimized	Change
Rendering	FPS	[X]	[X]	[X]%
Rendering	Frame Time	[X]	[X]	[X]%
Geometry	Triangles	[X]	[X]	[X]%
Rendering	Draw Calls	[X]	[X]	[X]%
Memory	GPU Memory	[X]	[X]	[X]%
Loading	Load Time	[X]	[X]	[X]%
Usability	Mean Score	[X]	[X]	[X]%

This table should become one of the central quantitative results of the thesis.

6.40 Overall Discussion

The experimental evaluation demonstrates that interactive 3D web applications require a multidisciplinary approach.

The developer must understand:

Computer Science

Data structures, algorithms, software architecture, and optimization.

Mathematics

Vectors, matrices, transformations, geometry, and numerical methods.

Graphics

Rasterization, lighting, materials, shaders, textures, and rendering.

Web Development

JavaScript, React, browser APIs, networking, and responsive interfaces.

Design

Composition, motion, interaction, typography, and visual hierarchy.

The proposed Interactive Graphics and Web Technologies specialization therefore occupies an intersection between traditional computer science, graphics programming, and digital design.

6.41 Key Findings

The principal findings can be summarized as follows.

Finding 1

Modern browsers are capable of supporting sophisticated interactive three-dimensional applications.

Finding 2

React-based architectures can successfully integrate conventional UI development with real-time 3D rendering.

Finding 3

Physically based rendering provides substantially improved material representation compared with basic material models when appropriate lighting is available.

Finding 4

Rendering performance depends on multiple factors rather than polygon count alone.

Finding 5

Optimization should be measurement-driven.

Finding 6

Interactive graphics quality depends on the combined characteristics of rendering performance, visual fidelity, and usability.

6.42 Research Contributions

The work presented in this thesis provides several contributions.

6.42.1 Architectural Contribution

A complete architecture for integrating modern web development with real-time 3D graphics was proposed.

6.42.2 Educational Contribution

The system demonstrates how mathematical graphics concepts can be connected directly to practical web development.

6.42.3 Technical Contribution

A modular implementation combining React, React Three Fiber, Three.js, WebGL, PBR, shaders, and animation was developed.

6.42.4 Experimental Contribution

A methodology for evaluating interactive 3D web applications was established.

6.42.5 Curriculum Contribution

The research provides a practical foundation for the proposed Interactive Graphics and Web Technologies university specialization.

6.43 Relationship to the Proposed Curriculum

The thesis results directly support the proposed curriculum presented at the beginning of this research. The curriculum can be mapped to the implemented system:

Curriculum Course	Application
Introduction to Programming	Application logic
Mathematics for Computer Graphics	Transformations and projection
Computer Graphics I	Rendering pipeline
Modern JavaScript	Application implementation
Interactive Web Development	Browser interaction
WebGL Programming	GPU rendering
Three.js Development	Scene construction
Blender	Asset production
Shader Programming	Custom GPU effects
Real-Time Rendering	PBR and optimization
GPU Programming	Performance and computation
Interactive Experience Development	React/R3F integration
AI for Interactive Graphics	Future extension

This demonstrates that the curriculum is not simply a collection of unrelated technologies.

Instead, the subjects form a progressive learning sequence:

Programming→Mathematics→Graphics→WebGL→3D→Shaders→Real–TimeRendering→Interactive Experiences

6.44 Implications for University Education

The findings suggest that interactive graphics should be taught as an interdisciplinary specialization rather than solely as an advanced web-development topic.

Students should first acquire fundamental programming and mathematical knowledge.

They can then progress toward:

- Computer graphics
- WebGL
- Three.js
- Shader programming
- Real-time rendering
- Creative coding
- XR
- AI

This progression minimizes the risk of students treating graphics libraries as black boxes.

For example, instead of merely teaching:

JavaScript

students should understand:

Vertex→Model Matrix→View Matrix→Projection Matrix→Rasterization→Fragment

This distinction is fundamental to producing graduates capable of solving new graphics problems rather than simply reproducing existing tutorials.

6.45 Future Research

Several areas should be investigated in future work.

WebGPU

Future experiments should compare WebGL and WebGPU performance.

WebXR

The system could be extended to VR and AR environments.

AI-Generated Assets

AI models could generate textures, materials, and three-dimensional assets.

GPU Compute

Particle systems and physical simulations could be moved to GPU compute pipelines.

Large-Scale Visualization

The system could be evaluated with significantly larger datasets.

Multi-User Interaction

Real-time collaborative 3D environments could be investigated.

Neural Rendering

Emerging neural rendering approaches could be incorporated into future systems.

6.46 Chapter Conclusion

This chapter presented the experimental evaluation and discussion of the proposed interactive 3D web graphics system.

The results framework demonstrates that the system can be evaluated across functional, visual, performance, and usability dimensions.

The implementation successfully integrates a conventional web application architecture with a real-time GPU rendering pipeline. The resulting system supports interactive camera control, dynamic materials, 3D models, animation, shader effects, and post-processing.

The performance analysis demonstrates that rendering cost is determined by multiple interacting factors, including geometry complexity, draw calls, texture resolution, shader complexity, lighting, shadows, and rendering resolution.

The optimization experiments further demonstrate the importance of a measurement-driven workflow. Reducing unnecessary geometry, minimizing draw calls, optimizing textures, controlling pixel ratio, and selectively applying post-processing can improve performance while maintaining acceptable visual quality.

The user-experience analysis additionally demonstrates that technical performance alone is insufficient. An effective interactive graphics application must combine:

Performance+Visual Quality+Interaction+Usability

The research therefore supports the central proposition of this thesis: **modern web technologies are capable of supporting sophisticated interactive graphics applications when they are combined with appropriate graphics theory, GPU programming principles, asset optimization, and interaction design.**

Furthermore, the research provides a practical foundation for the proposed **Interactive Graphics and Web Technologies (IGWT)** university specialization. The curriculum connects programming, mathematics, computer graphics, WebGL, shaders, real-time rendering, 3D modeling, interaction design, XR, and AI into a coherent educational progression.

The next chapter should therefore move from the experimental findings to the final synthesis of the research.

Chapter 7

Conclusion and Future Work

7.1 Introduction

This thesis investigated the design and implementation of an **Interactive Graphics and Web Technologies (IGWT)** framework that combines modern web development with real-time computer graphics.

The central motivation of the research was the rapid convergence of several previously distinct technological domains:

- Web development
- Computer graphics
- GPU programming
- 3D modeling
- Creative coding
- Extended reality
- Artificial intelligence

Traditionally, these areas have often been taught and developed independently. Web development has emphasized HTML, CSS, JavaScript, application architecture, and user interfaces, while computer graphics has focused on mathematics, rendering algorithms, shaders, and GPU programming.

Modern browser technologies increasingly make it possible to combine these disciplines within a single application.

The research therefore proposed a curriculum and technical architecture designed around the following progression:

Programming→Mathematics→Graphics→WebGL→3D→Shaders→Real-TimeRendering→InteractiveExperiences

The thesis subsequently implemented and evaluated a representative interactive 3D web application.

This final chapter summarizes the research, evaluates the achievement of its objectives, discusses its contributions and limitations, and proposes directions for future research.

7.2 Summary of the Research

The research began with the observation that modern digital experiences increasingly require interaction beyond conventional two-dimensional web interfaces.

Examples include:

- Interactive product visualization
- 3D configurators

- Digital museums
- Scientific visualization
- Architectural walkthroughs
- Creative coding
- Web-based games
- Virtual reality
- Augmented reality
- AI-powered visual applications

These applications require knowledge extending beyond traditional frontend development.

The proposed Interactive Graphics and Web Technologies specialization addresses this requirement through a six-semester curriculum.

The curriculum begins with programming and mathematics before progressing toward increasingly advanced graphics concepts.

The major educational sequence is:

This structure ensures that students acquire theoretical foundations before working with advanced graphics frameworks.

7.3 Achievement of Research Objectives

The research established several primary objectives.

Objective 1

To design a modern curriculum for Interactive Graphics and Web Technologies.

Achievement

A six-semester curriculum was developed covering:

- Programming
- Web technologies
- Mathematics
- Computer graphics
- WebGL
- Three.js
- Blender
- Shader programming
- Real-time rendering
- GPU programming
- Interactive experiences
- XR
- AI

- Advanced graphics
- Capstone development

The curriculum provides a progressive learning path from foundational concepts to advanced interactive graphics.

7.4 Objective 2

To establish a technical architecture for interactive 3D web applications.

Achievement

The proposed architecture integrates:

- React
- React Three Fiber
- Three.js
- WebGL
- GLSL
- Blender
- glTF/GLB
- GSAP
- PBR materials
- Post-processing

The resulting architecture separates application logic, user interface, graphics components, and 3D assets.

This modular structure supports scalability and maintainability.

7.5 Objective 3

To demonstrate real-time 3D rendering within a web browser.

Achievement

The implemented system demonstrates:

- Real-time 3D rendering
- Perspective camera control
- Model loading
- Material manipulation
- Environment lighting
- Animation
- Interactive camera movement
- Custom shader effects
- Post-processing

The results demonstrate that a browser can serve as a capable platform for sophisticated interactive graphics.

7.6 Objective 4

To investigate the performance characteristics of interactive web graphics.

Achievement

The research established an experimental methodology based on:

- Frame rate
- Frame time
- Triangle count
- Draw calls
- Texture resolution
- GPU workload
- Loading time
- User interaction

The research further demonstrated that optimization should be performed systematically.

The fundamental optimization process is:

Measure→Analyze→Optimize→Measure

rather than:

Guess→Optimize

This distinction is particularly important for real-time graphics development.

7.7 Objective 5

To establish the educational value of combining computer graphics and modern web technologies.

Achievement

The research demonstrates that the combination of these disciplines creates a coherent educational field.

Students can learn the relationship between mathematical theory and practical implementation.

For example:

Vector→Transformation→Shader→GPU→Rendered Image

This relationship allows students to understand not only how to use a graphics framework but also why the framework behaves as it does.

7.8 Answers to Research Question 1

RQ1: Can modern web technologies provide an effective framework for interactive 3D graphics?

The research indicates that they can.

Modern browser graphics technologies provide access to GPU-accelerated rendering while retaining the distribution and accessibility advantages of the web.

The proposed architecture demonstrates that a web application can support:

- Interactive 3D models
- Real-time lighting
- PBR materials
- Animation
- Custom shaders
- Post-processing
- Responsive interfaces

Therefore, web technology is no longer limited to conventional two-dimensional interfaces.

7.9 Answers to Research Question 2

RQ2: Can physically based rendering provide visually convincing real-time visualization?

The research demonstrates that PBR provides an effective representation of material properties when combined with appropriate lighting and environmental information.

The visual appearance of a material depends on more than its base color.

A simplified representation is:

$\text{Appearance} = f(\text{BaseColor}, \text{Metalness}, \text{Roughness}, \text{Normal}, \text{Lighting}, \text{Environment})$

This enables the system to distinguish materials such as:

- Metal
- Plastic
- Glass
- Painted surfaces
- Rough surfaces
- Polished surfaces

The result is particularly valuable for product visualization.

7.10 Answers to Research Question 3

RQ3: How does scene complexity affect rendering performance?

The research confirms that increasing scene complexity generally increases rendering cost.

Important factors include:

Complexity = Geometry + DrawCalls + Textures + Shaders + Lighting + Resolution

However, no single metric completely explains performance.

For example, a scene with relatively low polygon count can still perform poorly if it uses:

- Expensive fragment shaders
- High-resolution post-processing
- Multiple shadow passes
- Excessive texture sampling

Therefore, performance optimization must identify the actual bottleneck.

7.11 Answers to Research Question 4

RQ4: Which optimization techniques are effective?

The research identifies several important optimization strategies:

Geometry Optimization

Remove unnecessary geometric detail.

Level of Detail

Use different model complexity depending on viewing distance.

Draw-Call Reduction

Reduce unnecessary rendering submissions.

Texture Optimization

Use appropriate texture resolutions and compression.

Pixel-Ratio Control

Avoid unnecessarily rendering more pixels than the GPU requires.

Shadow Optimization

Balance shadow resolution and visual quality.

Post-Processing Control

Apply expensive effects selectively.

Resource Reuse

Reuse geometry, materials, and textures where appropriate.

The most important conclusion is that optimization should be **evidence-driven**.

7.12 Answers to Research Question 5

RQ5: Can the system provide an intuitive interactive experience?

The architecture supports interaction patterns that are familiar to users:

- Mouse rotation
- Touch interaction
- Scroll-based animation
- Zoom
- Material selection
- Color selection
- UI controls

The research therefore indicates that complex 3D functionality can be integrated into familiar web interaction patterns.

However, usability depends strongly on interface design.

Technical capability alone does not guarantee a good user experience.

7.13 Major Research Contributions

The research makes several contributions.

7.13.1 Curriculum Contribution

The thesis proposes a specialized curriculum that connects web development with real-time graphics.

This is distinct from conventional web-development curricula because graphics mathematics, rendering, shaders, and GPU programming are treated as core subjects rather than optional topics.

7.14 Technical Contribution

The research proposes a modular architecture for interactive 3D web applications.

The architecture can be represented as:

This architecture allows traditional web interfaces and real-time graphics to coexist within one application.

7.15 Graphics Contribution

The research demonstrates how fundamental graphics concepts can be integrated into modern web development.

These include:

- Coordinate systems
- Matrix transformations
- Camera projection
- Lighting
- Materials
- Textures
- Shaders
- Rendering pipelines
- Post-processing

This establishes a bridge between theoretical computer graphics and practical frontend development.

7.16 Educational Contribution

The proposed curriculum changes the role of the student from a consumer of graphics libraries to a developer capable of understanding graphics systems.

For example, a student should not only know:

JavaScript

but should understand the underlying concepts:

Rotation→TransformationMatrix→VertexTransformation→Projection→Rasterization

This deeper understanding enables students to solve unfamiliar problems.

7.17 Industry Contribution

The skills developed through the proposed curriculum can be applied to several industries.

Technology

Interactive interfaces and visualization systems.

E-Commerce

Three-dimensional product configurators.

Automotive

Interactive vehicle configuration.

Architecture

Real-time architectural visualization.

Entertainment

Games and interactive media.

Advertising

Immersive digital campaigns.

Education

Interactive educational visualization.

Scientific Computing

Large-scale scientific data visualization.

Healthcare

Interactive medical visualization.

Cultural Institutions

Digital museums and virtual exhibitions.

7.18 Relationship Between Web Development and Computer Graphics

One of the central conclusions of the research is that the distinction between web development and graphics programming is becoming less meaningful.

Historically:

$\text{WebDevelopment} \cap \text{ComputerGraphics}$

However, modern applications increasingly require:

$\text{WebDevelopment} + \text{ComputerGraphics}$

This convergence creates a new class of developer who must understand both application architecture and visual computation.

The proposed IGWT program is designed specifically around this convergence.

7.19 Relationship Between Graphics and Design

Interactive graphics cannot be treated purely as a technical problem.

A technically sophisticated scene can still fail if its composition is poor.

The final result depends on:

Experience\=Technology+Design+Interaction

Students therefore need knowledge of:

- Visual hierarchy
- Typography
- Composition
- Motion
- Interaction
- User experience

This is why the proposed curriculum includes interactive experience development rather than focusing exclusively on graphics APIs.

7.20 Relationship Between Graphics and Artificial Intelligence

Artificial intelligence represents another major direction for the field.

AI can assist in:

- 3D asset generation
- Texture generation
- Image generation
- Material creation
- Animation
- Scene generation
- Natural-language interaction
- Computer vision
- Procedural content generation

The future interactive graphics developer may therefore operate within a pipeline such as:

This creates opportunities for fundamentally different interaction models.

Instead of manually manipulating every scene parameter, users may describe the desired result using natural language.

7.21 WebGPU as a Future Direction

WebGL has been highly influential in bringing GPU graphics to the web.

However, WebGPU provides a more modern graphics and compute architecture.

Future versions of the proposed curriculum should therefore introduce WebGPU after students understand:

- GPU architecture
- Rendering pipelines
- Shaders
- Buffers
- Textures
- WebGL

The educational sequence could become:

WebGL→GPU Fundamentals→WebGPU

Rather than replacing WebGL entirely, WebGPU should initially be taught as a more advanced GPU programming model.

7.22 Extended Reality

XR represents another natural extension of interactive web graphics.

Web-based XR can support:

- Virtual reality
- Augmented reality
- Spatial interaction
- 3D interfaces
- Immersive visualization

The conceptual progression is:

2D Web→3D Web→XR

Students who understand cameras, transformations, coordinate systems, lighting, and interaction are well positioned to move into XR development.

7.23 Large-Scale Visualization

Future research should investigate the rendering of substantially larger datasets.

Examples include:

- Millions of particles
- Geographic datasets
- Scientific simulations

- Point clouds
- Digital twins
- Large architectural environments

Such systems will require:

- GPU compute
- Instancing
- Culling
- Level of detail
- Streaming
- Data compression
- WebGPU

This represents an important direction for extending the current work.

7.24 Multi-User Interactive Graphics

The current system primarily assumes a single user.

Future research could investigate multiple users interacting with the same three-dimensional environment.

A possible architecture is:

Applications could include:

- Collaborative design
- Virtual classrooms
- Digital exhibitions
- Multiplayer games
- Remote engineering
- Collaborative visualization

This would require synchronization of scene state and user interaction.

7.25 AI-Assisted Development

AI could also change the way interactive graphics applications are developed.

For example, a student could request:

Create a metallic product material with brushed reflections.

An AI development assistant could generate:

- Material parameters
- Shader code
- React components

- Three.js configuration
- Asset-processing instructions

This suggests a future educational model in which students focus increasingly on:

- Problem formulation
- System architecture
- Visual reasoning
- Mathematical understanding
- Evaluation

while AI assists with implementation.

7.26 Limitations of the Research

Despite the contributions of this research, several limitations remain.

7.26.1 Experimental Scope

The implemented application represents a specific category of interactive 3D visualization.

Other applications may behave differently.

7.26.2 Hardware Dependence

Performance measurements depend on GPU and CPU characteristics.

Results obtained on one machine cannot automatically be generalized to all systems.

7.26.3 Browser Dependence

Different browsers, drivers, and operating systems can produce different performance characteristics.

7.26.4 User Study Size

If the proposed user evaluation is performed with a relatively small number of participants, the results should be considered exploratory rather than universally representative.

7.26.5 Measurement Variability

Browser-based performance can be affected by:

- Background processes
- Browser extensions

- Thermal throttling
- Network conditions
- Operating-system activity

Repeated measurements are therefore necessary.

7.27 Recommendations for Future Experiments

Future studies should increase the scope of evaluation.

Recommendation 1

Use multiple GPU classes.

For example:

- Integrated GPU
- Entry-level discrete GPU
- Mid-range GPU
- High-end GPU
- Mobile GPU

Recommendation 2

Evaluate multiple browsers.

Recommendation 3

Test both desktop and mobile devices.

Recommendation 4

Compare WebGL and WebGPU.

Recommendation 5

Increase the number of participants in usability studies.

Recommendation 6

Use larger and more diverse 3D scenes.

Recommendation 7

Investigate power consumption and thermal characteristics.

These improvements would produce a more comprehensive understanding of interactive graphics on the web.

7.28 Proposed Final-Year Student Project Structure

The research also provides a framework for designing student graduation projects.

Each student should produce four principal deliverables.

1. Working Application

A complete interactive graphics application.

Examples include:

- Product configurator
 - 3D game prototype
 - Museum visualization
 - Scientific visualization
 - Architectural walkthrough
 - Interactive art installation
-

2. Source Code

Students should submit the complete source code with:

- Project structure
 - Dependencies
 - Build instructions
 - Documentation
-

3. Technical Report

The report should describe:

- Problem definition
 - Related work
 - Architecture
 - Mathematical foundation
 - Implementation
 - Experiments
 - Results
 - Discussion
 - Limitations
 - Future work
-

4. Graduation Thesis

For the proposed program, the graduation thesis should not simply be a software-development report.

It should demonstrate that the student can:

Identify Problem→Research→Design→Implement→Evaluate→Analyze

The thesis should therefore contain a measurable research question.

7.29 Recommended Graduation Thesis Format

A student's final thesis could use the following structure:

Chapter 1

Introduction

Chapter 2

Literature Review and Related Technologies

Chapter 3

Mathematical and Technical Foundations

Chapter 4

System Architecture and Design

Chapter 5

Implementation

Chapter 6

Experimental Evaluation

Chapter 7

Conclusion and Future Work

References

Appendices

This structure is appropriate for a research-oriented Interactive Graphics and Web Technologies program.

7.30 Graduation Requirements

A complete graduation project should require students to submit:

Deliverable	Requirement
Working Application	Required
Source Code	Required
Technical Documentation	Required
Graduation Thesis	Required
Demonstration Video	Recommended
Presentation	Required
Public Demo/Deployment	Recommended
Experimental Data	Required
Git Repository	Recommended

This ensures that students are evaluated not merely on whether they can produce attractive visual effects, but also on their ability to conduct engineering and research.

7.31 Proposed Evaluation Criteria

A final project could be evaluated using the following weighting.

Category	Weight
Technical Implementation	25%
Graphics Quality	15%
Interaction Design	15%
Performance Optimization	15%
Research Methodology	15%
Thesis Quality	10%
Presentation	5%
Total	100%

This evaluation structure balances artistic quality, engineering ability, and academic research.

7.32 Final Research Model

The overall research model can be summarized as:

This model represents the central educational philosophy of the proposed specialization.

Students begin with fundamental concepts and progressively build toward sophisticated interactive systems.

7.33 Final Conclusion

This thesis investigated the development of a modern framework for **Interactive Graphics and Web Technologies** and proposed a corresponding university curriculum.

The research was motivated by the increasing convergence of web development, computer graphics, GPU programming, interactive design, extended reality, and artificial intelligence.

The resulting curriculum addresses this convergence through a structured six-semester progression.

The first stage establishes programming, web technologies, and mathematics.

The second stage introduces computer graphics, JavaScript engineering, and browser interaction.

The third stage introduces WebGL, Three.js, and professional 3D asset production.

The fourth stage moves into advanced shaders, physically based rendering, and GPU programming.

The fifth stage introduces interactive experiences, XR, and AI.

The final stage emphasizes advanced graphics and independent research through a capstone project and graduation thesis.

The technical implementation presented in this research demonstrates that modern web technologies can support sophisticated interactive 3D experiences.

The system combines:

React+React Three Fiber+Three.js+WebGL+GLSL+Blender+PBR

to produce an integrated interactive graphics environment.

The research further demonstrates that successful interactive graphics development requires more than knowledge of a particular framework.

A developer must understand the relationship between:

Mathematics→Graphics→GPU→Software→Interaction→Design

This relationship represents one of the central findings of the research.

The thesis therefore proposes that **Interactive Graphics and Web Technologies should be treated as a distinct interdisciplinary educational specialization**, rather than merely as a collection of advanced web-development topics.

The field occupies a productive intersection between computer science, graphics engineering, web technologies, digital design, and emerging AI systems.

Its future is likely to be shaped by WebGPU, WebXR, neural rendering, generative AI, GPU computing, and increasingly intelligent interactive interfaces.

The proposed curriculum is consequently designed not only to teach today's technologies but also to provide students with the conceptual foundations necessary to adapt to technologies that have not yet been fully established.

The ultimate educational objective is not to produce graduates who merely know how to use Three.js, React Three Fiber, WebGL, or another specific library.

Rather, the objective is to produce graduates who understand **why interactive graphics systems work, how they should be designed, how they can be optimized, and how they can be transformed into meaningful human experiences.**

In this sense, the proposed program can be summarized by the following principle:

Understand the Mathematics→Understand the Graphics→Understand the GPU→Build the Experience

This principle provides the foundation for the proposed **Interactive Graphics and Web Technologies (IGWT)** specialization and represents the principal conclusion of this research.

7.34 Final Thesis Structure

With Chapter 7 completed, the main body of the thesis is now:

Chapter	Title	Primary Purpose
1	Introduction	Research motivation and objectives
2	Literature Review	Existing technologies and research
3	Mathematical & Graphics Foundations	Mathematical and rendering theory
4	System Architecture	Proposed architecture
5	Implementation	Development and implementation
6	Experimental Results & Discussion	Evaluation and analysis
7	Conclusion & Future Work	Contributions and future directions

The remaining material should be:

References

Academic papers, books, standards, and official technical documentation.

Appendix A

Complete application source-code structure.

Appendix B

Experimental hardware/software configuration.

Appendix C

Raw benchmark measurements.

Appendix D

User-study questionnaire and responses.

Appendix E

Application screenshots.

Appendix F

IGWT curriculum and course-to-project mapping.

Chapter 8

Curriculum Design and Educational Framework for Interactive Graphics and Web Technologies

8.1 Introduction

The preceding chapters investigated the theoretical foundations, technical architecture, implementation, and experimental evaluation of an interactive graphics system based on modern web technologies.

The research demonstrated that interactive graphics development requires knowledge from several traditionally separate disciplines:

- Programming
- Mathematics
- Computer graphics
- Web technologies
- GPU programming
- 3D modeling
- Interaction design
- User experience
- Extended reality
- Artificial intelligence

The purpose of this chapter is to translate the findings of the technical research into a practical university-level educational framework.

The proposed program is titled:

Interactive Graphics and Web Technologies (IGWT)

The program is designed as a six-semester specialization that progressively develops students from fundamental programming skills toward advanced real-time graphics, interactive experiences, XR, AI, and independent research.

The educational philosophy is based on the following principle:

Foundation→Graphics→GPU→Interaction→Experience→Research

The objective is not simply to train students to use particular software libraries. Instead, students should understand the underlying principles sufficiently to adapt to changing technologies.

8.2 Educational Philosophy

Traditional web-development education often follows a progression such as:

HTML→CSS→JavaScript→Frameworks

This progression is effective for conventional web applications but is insufficient for advanced interactive graphics.

The proposed IGWT curriculum follows a broader model:

Programming+Mathematics+Graphics+Web+GPU+Design

These areas are progressively integrated.

The fundamental educational principle is:

Students should understand the underlying system before depending on high-level abstractions.

For example, students should understand coordinate systems and transformation matrices before relying entirely on a 3D framework's abstraction of camera and object transformations.

Similarly, students should understand the basic rendering pipeline before learning sophisticated shader libraries.

8.3 Program Learning Outcomes

After completing the program, graduates should be able to:

PLO1 — Programming

Design and implement software using modern programming languages and development methodologies.

PLO2 — Mathematics

Apply vectors, matrices, geometry, trigonometry, and linear algebra to graphics problems.

PLO3 — Computer Graphics

Explain and implement fundamental computer graphics concepts.

PLO4 — Web Technologies

Develop modern browser-based applications using current web standards.

PLO5 — GPU Programming

Understand GPU execution and implement GPU-oriented rendering techniques.

PLO6 — Shader Programming

Develop and modify vertex and fragment shaders.

PLO7 — 3D Development

Create interactive three-dimensional applications using modern web graphics frameworks.

PLO8 — Asset Production

Create, optimize, and export 3D assets for real-time applications.

PLO9 — Interactive Experience

Design interfaces that combine visual, spatial, and interactive elements.

PLO10 — XR

Develop introductory virtual and augmented reality applications.

PLO11 — AI

Integrate AI technologies into interactive graphics applications.

PLO12 — Research

Design experiments, analyze results, and communicate technical findings.

PLO13 — Professional Practice

Develop complete production-ready projects using version control, documentation, testing, and deployment.

8.4 Curriculum Structure

The proposed curriculum consists of six semesters.

Semester 1 — Foundations

Course	Credits*
Introduction to Programming	3
Web Technologies	3
Mathematics for Computer Graphics	3
Computer Systems Fundamentals	3
Digital Design Fundamentals	3

*The exact credit system should be adapted to the university's academic regulations.

The first semester establishes the foundations necessary for subsequent graphics courses.

8.5 Introduction to Programming

Purpose

Students learn programming fundamentals before beginning advanced graphics development.

Topics

- Variables
- Data types
- Functions
- Objects
- Arrays
- Algorithms
- Control structures
- Debugging
- Problem solving
- Git

Suggested Language

Python can be used initially because of its relatively accessible syntax.

JavaScript should subsequently be introduced because it becomes central to the web-development sequence.

Practical Projects

Students could create:

- Calculator
- Data-processing application
- Small algorithmic visualization
- Interactive webpage

Final Deliverable

A small programming project accompanied by source code and documentation.

8.6 Web Technologies

Topics

- HTML
- CSS
- JavaScript
- HTTP
- URLs

- Browser architecture
- DOM
- Events
- Responsive design
- Accessibility

Students should understand the browser as a software platform rather than treating HTML and CSS as isolated technologies.

Project

Students build a responsive interactive website without relying on a large framework.

8.7 Mathematics for Computer Graphics

This course is one of the most important differentiators of the proposed program.

Topics

Vectors

$$v = xyz$$

Dot Product

$$a \cdot b = axbx + ayby + azbz$$

Cross Product

$$a \times b$$

Matrices

Students study transformation matrices and their applications.

Transformations

- Translation
- Rotation
- Scaling
- Projection

Trigonometry

Students apply sine, cosine, and tangent to graphics problems.

Practical Exercise

Students implement a simple 2D transformation system without using a graphics framework.

8.8 Semester 2 — Graphics and Interaction

Course	Main Topics
Computer Graphics I	Rendering pipeline
Modern JavaScript Development	ES6+, modules, tooling
Interactive Web Development	Canvas, SVG, animation
Human-Computer Interaction	Interaction principles
Creative Coding	Generative visual systems

The second semester transitions students from basic software development into visual computing.

8.9 Computer Graphics I

Topics

- Coordinate systems
- Rendering pipeline
- Rasterization
- Cameras
- Transformations
- Lighting
- Textures
- Depth
- Visibility

Students should understand:

Object→World→Camera→Projection→Screen

Laboratory

Students implement a simple 2D or 3D renderer.

The objective is not necessarily to build a production renderer but to understand the underlying process.

8.10 Modern JavaScript Development

Topics

- ES6+
- Modules
- Async programming
- Promises

- Fetch API
- npm
- Build tools
- TypeScript
- Performance
- Debugging

Students learn how modern web applications are actually constructed.

Project

Develop a modular interactive web application.

8.11 Interactive Web Development

Topics

- Canvas
- SVG
- CSS animation
- Web animation
- GSAP
- Scroll interaction
- Browser rendering
- Event systems

The course introduces the idea that the browser can become a creative programming environment.

Project

Create an interactive animated web experience.

8.12 Semester 3 — 3D Web Development

Course	Main Topics
WebGL Programming	GPU rendering
Three.js Development	3D frameworks
Blender for Real-Time Graphics	Asset creation
3D Interaction Design	Spatial interaction
Graphics Project I	Integrated project

8.13 WebGL Programming

Topics

- WebGL context
- Buffers
- Attributes
- Uniforms
- Textures
- Framebuffers
- Shaders
- Rendering loops

Students should write basic WebGL programs without relying exclusively on Three.js.

This is important because it reveals what the higher-level framework is actually doing.

8.14 Three.js Development

After learning WebGL fundamentals, students move to a higher-level abstraction.

Topics

- Scene
- Camera
- Mesh
- Geometry
- Materials
- Lighting
- Animation
- Model loading
- Controls
- Optimization

Project

Create an interactive 3D website.

8.15 Blender for Real-Time Graphics

Topics

- Modeling
- Topology
- UV mapping

- Materials
- Textures
- Lighting
- Animation
- Optimization
- glTF/GLB

The emphasis is not on becoming a professional film animator.

Instead, students learn to produce assets appropriate for real-time rendering.

8.16 Semester 4 — Advanced Graphics

Course	Main Topics
Shader Programming	GLSL
Real-Time Rendering	PBR
GPU Programming	GPU computation
Procedural Graphics	Noise and SDF
Graphics Project II	Advanced project

8.17 Shader Programming

Students learn:

- GLSL
- Vertex shaders
- Fragment shaders
- Uniforms
- Varyings
- UV coordinates
- Normals
- Noise
- Procedural generation

Students progressively move from simple shaders:

Color

to:

Lighting→Noise→Procedural Geometry→Ray Marching

8.18 Real-Time Rendering

Topics

- PBR
- HDR
- Environment lighting
- Shadow mapping
- Ambient occlusion
- Bloom
- Tone mapping
- Post-processing

Students learn to evaluate rendering quality and computational cost simultaneously.

The central principle is:

Visual Quality ↔ Performance

8.19 GPU Programming

Topics

- GPU architecture
- Parallel computation
- Framebuffer objects
- GPGPU
- Particle systems
- Simulation
- Compute concepts
- Introduction to WebGPU

Project

Create a GPU-powered particle or simulation system.

8.20 Semester 5 — Interactive Experiences

Course	Main Topics
Interactive Experience Development	R3F + UI
Virtual & Augmented Reality	WebXR
AI for Interactive Graphics	AI APIs
Advanced Creative Coding	Generative systems
Professional Practice	Deployment and collaboration

This semester moves from graphics technology toward complete interactive experiences.

8.21 Interactive Experience Development

Topics

- React Three Fiber
- UI integration
- State management
- Animation
- Motion design
- Audio visualization
- Scroll-driven interaction
- Responsive 3D

The objective is to combine:

UI+3D+Motion+Interaction

into a unified experience.

8.22 Virtual and Augmented Reality

Topics

- XR concepts
- VR
- AR
- WebXR
- Spatial interaction
- Controllers
- Head tracking
- Interaction design

Students develop a small XR prototype.

8.23 AI for Interactive Graphics

This course should be treated as an application-oriented course rather than a general machine-learning course.

Topics

- AI APIs
- Generative AI
- Image generation
- AI-assisted coding
- AI agents
- Computer vision
- AI-generated textures
- AI-generated assets
- Natural-language interfaces

A possible project is:

Build an interactive 3D application that can be controlled through natural language.

8.24 Semester 6 — Research and Graduation

Course	Main Topics
Advanced Computer Graphics	Advanced rendering
Research Methodology	Scientific research
Graduation Thesis	Independent research
Capstone Project	Complete application

The sixth semester is designed around independent work.

Students transition from:

Following Instructions

to:

Defining Problems

and ultimately:

Conducting Research

8.25 Advanced Computer Graphics

Topics

- Global illumination
- Deferred rendering
- Volumetric rendering
- GPU optimization
- Advanced shadow techniques
- Neural rendering
- WebGPU
- Large-scale visualization

The course should emphasize research papers and technical experimentation.

8.26 Research Methodology

Students learn:

- Research questions
- Hypotheses
- Literature review
- Experimental design
- Data collection
- Statistical analysis
- Academic writing
- Citation
- Reproducibility

This course is critical because the graduation thesis should represent genuine research rather than simply a project description.

8.27 Graduation Thesis

The graduation thesis should answer a specific research question.

For example:

How does adaptive level-of-detail rendering affect the performance of interactive WebGL applications while maintaining perceived visual quality?

This is substantially stronger than:

"I created a Three.js website."

The thesis should contain:

8.28 Capstone Project

The capstone project should be an independent interactive application.

Possible topics include:

Product Visualization

Interactive product configurator.

Scientific Visualization

Interactive visualization of scientific data.

Digital Museum

Three-dimensional interactive exhibition.

Architecture

Virtual building walkthrough.

Game Technology

Web-based 3D game prototype.

Creative Coding

Generative visual experience.

Medical Visualization

Interactive anatomical visualization.

AI Graphics

AI-controlled interactive 3D environment.

8.29 Graduation Deliverables

Every graduating student should submit:

1. Working Application

A complete functioning application.

2. Source Code

Complete source code and dependency information.

3. Thesis

A formal academic document.

4. Experimental Dataset

Raw benchmark or user-study data.

5. Technical Documentation

Installation and architecture documentation.

6. Presentation

Oral defense presentation.

7. Demonstration

Live demonstration of the application.

8. Project Video

A short demonstration video is recommended.

8.30 Graduation Thesis Requirements

A recommended minimum thesis structure is:

Chapter 1

Introduction

Chapter 2

Related Work

Chapter 3

Theoretical Foundation

Chapter 4

System Design

Chapter 5

Implementation

Chapter 6

Experimental Evaluation

Chapter 7

Discussion and Conclusion

The exact structure may vary depending on the research topic.

8.31 Assessment Philosophy

The program should not evaluate students solely according to whether their application "looks impressive."

A visually beautiful project with poor engineering should not receive the highest grade.

Similarly, technically sophisticated software with poor usability should not automatically receive the highest grade.

Assessment should consider:

Final Quality\=Technical+Visual+Interaction+Research

8.32 Proposed Assessment Model

A recommended final-project assessment is:

Category	Weight
Technical Implementation	20%
Graphics Quality	15%
Interaction Design	15%
Performance	10%
Research Methodology	15%
Thesis	15%
Presentation	5%
Documentation	5%
Total	100%

This produces a balanced evaluation.

8.33 Faculty Requirements

A program of this type should ideally not rely on a single professor with expertise in every field.

The faculty structure could include specialists in:

Graphics

Computer graphics and rendering.

Web Engineering

Frontend architecture and browser technologies.

3D Production

Modeling, texturing, animation.

GPU Computing

GPU architecture and shader programming.

UX / Design

Interaction and visual design.

AI

Generative AI and machine learning.

XR

VR, AR, and spatial computing.

This multidisciplinary structure is consistent with the nature of the program itself.

8.34 Laboratory Infrastructure

A dedicated graphics laboratory is recommended.

Students should have access to:

- GPU-equipped workstations
- Multiple monitors
- VR headsets
- Mobile devices
- High-speed network
- 3D modeling software
- Version-control infrastructure
- Web deployment infrastructure

The laboratory should support both individual development and collaborative projects.

8.35 Software Environment

The recommended software stack includes:

The exact software versions should be updated periodically.

8.36 Teaching Methodology

The program should use a combination of:

- Lectures
- Laboratory exercises
- Workshops
- Code reviews
- Design critiques
- Research seminars
- Project presentations

A recommended weekly structure is:

Lecture

90 minutes

Laboratory

120–180 minutes

Project / Studio

60–120 minutes

This provides a balance between theoretical knowledge and practical implementation.

8.37 Lecture Methodology

A typical lecture should follow:

For example, when teaching perspective projection:

1. Explain perspective mathematically.
2. Show a visual demonstration.
3. Demonstrate camera parameters.
4. Implement the transformation.
5. Ask students to modify the field of view.
6. Discuss the resulting visual differences.

This approach connects abstract theory with observable behavior.

8.38 Laboratory Methodology

Laboratories should be highly practical.

A typical laboratory could contain:

Exercise 1

Create a scene.

Exercise 2

Add a camera.

Exercise 3

Add lighting.

Exercise 4

Load a model.

Exercise 5

Apply materials.

Exercise 6

Add animation.

Exercise 7

Optimize performance.

The student should finish the laboratory with a working result rather than merely completing written exercises.

8.39 Project-Based Learning

Project-based learning should become increasingly important in later semesters.

The progression should be:

Semester 1

Small exercises.

Semester 2

Small applications.

Semester 3

Interactive 3D applications.

Semester 4

Advanced graphics projects.

Semester 5

Complete interactive experiences.

Semester 6

Independent research project.

Thus:

Exercise→Project→System→Research

8.40 Industry Collaboration

The program would benefit from collaboration with companies working in:

- Game development
- Web development
- Digital media
- Architecture
- Automotive
- E-commerce
- XR
- AI
- Visualization

Industry partners could provide:

- Guest lectures
 - Real-world project briefs
 - Internships
 - Technical workshops
 - Portfolio reviews
 - Employment opportunities
-

8.41 Student Portfolio Development

Portfolio development should begin early.

By graduation, a student should ideally have:

Project 1

Interactive 2D web experience.

Project 2

Three.js application.

Project 3

Custom shader project.

Project 4

Real-time rendering project.

Project 5

XR or AI graphics project.

Project 6

Graduation project.

This creates a clear progression that employers can evaluate.

8.42 Graduate Career Paths

Graduates could pursue careers including:

- Creative Developer
- Creative Technologist
- WebGL Developer
- Three.js Developer
- 3D Web Developer
- Interactive Developer
- Graphics Programmer
- Technical Artist
- Real-Time Visualization Developer
- XR Developer
- GPU Programmer
- Visualization Engineer
- Frontend Engineer
- AI Interactive Experience Developer

The program therefore provides multiple career pathways rather than training students for one narrowly defined job.

8.43 Distinctive Identity of the Program

The program's primary distinguishing characteristic is the integration of:

Web+Graphics+GPU+Design+AI

Many programs emphasize one or two of these areas.

The proposed IGWT program intentionally places them within the same educational structure.

This creates a distinctive academic identity.

8.44 Curriculum Evolution

Because the technology landscape changes rapidly, the curriculum should not remain fixed indefinitely.

A curriculum review should occur periodically.

For example:

Annual Technology Review

followed by:

Curriculum Adjustment

Potential future additions include:

- WebGPU
- Neural rendering
- Gaussian splatting
- AI agents
- Generative 3D
- Spatial computing
- Digital twins
- Real-time simulation

The fundamental concepts should remain stable even as specific technologies change.

8.45 Stable Knowledge vs. Changing Technology

One of the most important curriculum-design principles is distinguishing between stable knowledge and rapidly changing tools.

Stable

- Mathematics
- Algorithms
- Rendering principles
- Coordinate systems

- Linear algebra
- GPU concepts
- Software architecture

Changing

- Frameworks
- Libraries
- AI models
- Browser APIs
- Build tools
- Hardware

Therefore, the curriculum should prioritize:

Fundamentals>Tools

A student who understands rendering pipelines can learn a new graphics framework.

A student who only knows a framework may struggle when that framework changes.

8.46 Recommended Teaching Principle

The overall teaching philosophy can be summarized as:

Teach principles deeply and tools pragmatically.

Students should learn enough Three.js to build real applications.

However, they should also understand enough WebGL to know what Three.js is doing.

Likewise, students should use React Three Fiber to develop production applications while understanding the underlying Three.js scene graph and browser rendering model.

8.47 Final Curriculum Model

The complete educational model is:

This model represents the complete progression of the proposed program.

8.48 Chapter Conclusion

This chapter translated the technical findings of the thesis into a complete educational framework for the proposed **Interactive Graphics and Web Technologies** program.

The curriculum is designed around a progressive sequence beginning with programming, mathematics, and web fundamentals and continuing toward computer graphics, WebGL, Three.js, Blender, shaders, real-time

rendering, GPU programming, XR, AI, and independent research.

The program's primary educational objective is to create graduates who can move between abstraction levels.

A graduate should be able to think simultaneously about:

User Experience Application Architecture Graphics Algorithms GPU Execution

and

Mathematical Foundations

This combination is the defining characteristic of the proposed specialization.

The curriculum also establishes a clear graduation framework. Students are expected to produce a functioning application, source code, technical documentation, experimental data, and a graduation thesis. Consequently, graduation represents not only completion of coursework but also evidence of independent technical and research capability.

The proposed program therefore provides a coherent educational pathway:

Learn→Build→Experiment→Optimize→Research

The framework is intentionally designed to remain relevant as individual technologies evolve. While libraries, browsers, graphics APIs, and AI systems will continue to change, the underlying knowledge of mathematics, rendering, GPU computation, software architecture, and interaction will remain valuable.

The proposed **Interactive Graphics and Web Technologies (IGWT)** program consequently represents not simply a modern web-development curriculum, but an interdisciplinary educational model for preparing students to create the next generation of interactive digital experiences.

Updated Thesis Structure

With Chapter 8 included, the complete thesis can now be organized as:

Chapter	Title
1	Introduction
2	Literature Review and Related Technologies
3	Mathematical and Computer Graphics Foundations
4	System Architecture and Design
5	Implementation
6	Experimental Results and Discussion
7	Conclusion and Future Work
8	Curriculum Design and Educational Framework
—	References
—	Appendix A: Source Code
—	Appendix B: Experimental Configuration
—	Appendix C: Raw Experimental Data
—	Appendix D: User Questionnaire
—	Appendix E: Application Screenshots
—	Appendix F: Curriculum and Course Mapping

Chapter 9

Program Evaluation, Quality Assurance, and Continuous Improvement

9.1 Introduction

The development of a new university program requires more than defining a collection of courses. A sustainable academic program must also establish mechanisms for evaluating educational outcomes, monitoring student achievement, maintaining academic standards, incorporating technological developments, and continuously improving the curriculum.

The proposed **Interactive Graphics and Web Technologies (IGWT)** program is situated within a rapidly changing technological environment. Web APIs, graphics hardware, rendering frameworks, artificial intelligence systems, and extended-reality platforms evolve considerably faster than traditional university curricula.

Consequently, a static curriculum is insufficient.

The program should instead be understood as a continuously evolving educational system:

Curriculum→Teaching→Assessment→Evaluation→Improvement→Curriculum

This chapter proposes a quality-assurance framework for the IGWT program.

The framework evaluates five major dimensions:

1. Student learning
2. Teaching effectiveness
3. Technical competency
4. Research capability
5. Industry relevance

9.2 Need for Continuous Evaluation

Traditional academic programs can often remain relatively stable for several years.

Interactive graphics and web technologies are different.

A technology that is widely used when students enter the program may be significantly changed or replaced before they graduate.

For example, the technological progression may be represented as:

WebGL→WebGPU Traditional 3D Assets→AI Generated Assets Static Interfaces→AI Driven Interfaces Desktop Interaction→Spatial Computing

Therefore, the curriculum should maintain stable theoretical foundations while allowing implementation technologies to evolve.

9.3 Quality Assurance Model

The proposed quality-assurance framework consists of four levels.

Each level provides feedback to the next.

9.4 Course-Level Evaluation

Every course should define measurable learning outcomes.

For example, the course **Shader Programming** could define:

CLO1

Explain the purpose of vertex and fragment shaders.

CLO2

Implement GLSL shaders.

CLO3

Apply mathematical transformations within shaders.

CLO4

Create procedural visual effects.

CLO5

Optimize shader performance.

These outcomes should be directly connected to assessments.

9.5 Course Learning Outcomes

The relationship can be represented as:

CLO→Assessment→Evidence

For example:

Learning Outcome	Assessment
Understand GLSL	Written examination
Write shaders	Laboratory
Create procedural effects	Project
Optimize shaders	Performance experiment
Explain results	Technical report

This ensures that assessment measures actual learning rather than simply attendance or completion.

9.6 Program Learning Outcomes

Course-level outcomes should contribute to program-level outcomes.

For example:

CLOShader→PLOGPU CLOGraphics→PLOComputer Graphics CLOResearch→PLOResearch

This produces a curriculum map.

9.7 Curriculum Mapping

A simplified mapping can be represented as follows:

Course	Programming	Graphics	GPU	Web	Interaction	Research
Introduction to Programming	●●●	—	—	●	—	●
Mathematics for Graphics	●	●●●	●	—	—	●
Computer Graphics I	●	●●●	●●	—	●	●
WebGL Programming	●●	●●●	●●●	●●	●	●
Three.js	●●	●●●	●●	●●●	●●	●
Shader Programming	●●	●●●	●●●	●●	●●	●●
Real-Time Rendering	●	●●●	●●●	●●	●●	●●
XR	●●	●●●	●●	●●●	●●●	●●
AI Graphics	●●●	●●	●●	●●●	●●●	●●●
Graduation Thesis	●●	●●●	●●●	●●	●●	●●●

Here:

- = introductory
- = intermediate
- = advanced

This mapping allows the university to verify that every graduate competency is developed progressively.

9.8 Assessment Strategy

Assessment should occur at multiple levels.

Formative Assessment

Used during learning.

Examples:

- Weekly exercises
- Coding tasks
- Laboratory work
- Quizzes
- Peer reviews

Summative Assessment

Used to determine final achievement.

Examples:

- Final examinations
- Projects
- Technical reports
- Presentations
- Thesis

The combination is:

Learning\=Formative+Summative

9.9 Practical Assessment

Because IGWT is highly technical and project-oriented, practical assessment should have substantial importance.

A student may understand the theoretical definition of a shader but still be unable to implement one.

Therefore:

Knowledge $\neq$ Skill

A complete assessment system must evaluate both.

9.10 Project-Based Assessment

Projects should evaluate multiple competencies simultaneously.

For example, a 3D product configurator may require:

- JavaScript
- React
- Three.js
- WebGL
- Materials
- Lighting
- Animation
- UI design
- Performance optimization

Therefore, a single project can provide evidence of multiple learning outcomes.

9.11 Technical Evaluation

Student projects should be evaluated using objective technical criteria.

Possible metrics include:

Performance

FPS FrameTime

Rendering Complexity

TriangleCount DrawCalls

Memory

TextureMemory GPU\Memory

Loading

InitialLoadTime

These measurements provide quantitative evidence of engineering quality.

9.12 Visual Evaluation

Graphics projects should also be evaluated visually.

Possible criteria include:

- Composition

- Lighting
- Material quality
- Animation
- Color
- Visual hierarchy
- Consistency
- Readability

A rubric could assign scores from 1 to 5.

Criterion	1	3	5
Lighting	Poor	Acceptable	Excellent
Materials	Unrealistic	Reasonable	Convincing
Animation	Limited	Functional	Highly refined
Composition	Weak	Good	Excellent

This provides a structured alternative to subjective grading.

9.13 Interaction Evaluation

Interactive applications should be evaluated according to usability.

Possible criteria include:

- Learnability
- Responsiveness
- Navigation
- Feedback
- Error prevention
- Accessibility
- Interaction consistency

Students should be encouraged to test their applications with real users.

9.14 User Testing

A small user study can provide valuable evidence.

A typical study might involve:

1. Recruit participants.
2. Explain the task.
3. Observe interaction.
4. Record completion time.

5. Record errors.
6. Collect questionnaire responses.
7. Analyze results.

For example:

$$\text{TaskCompletionRate} = \frac{\text{TotalTasks}}{\text{SuccessfulTasks}} \times 100$$

9.15 Research Assessment

The graduation thesis should be evaluated according to academic research criteria.

Recommended criteria include:

Criterion	Weight
Research Question	10%
Literature Review	15%
Methodology	15%
Implementation	20%
Experimental Evaluation	20%
Analysis	10%
Academic Writing	10%

The exact weighting may be adjusted by the institution.

9.16 Thesis Defense

Every student should defend the graduation thesis before a committee.

A typical defense could consist of:

Presentation

15–20 minutes.

Demonstration

5–10 minutes.

Questions

15–30 minutes.

The committee should evaluate whether the student can explain:

- Why the problem matters
- What previous research exists

- What was implemented
 - Why the methodology was selected
 - What the results mean
 - What limitations exist
 - What should happen next
-

9.17 Research Reproducibility

A particularly important requirement for technical theses is reproducibility.

Students should provide:

- Source code
- Dataset
- Configuration
- Hardware specifications
- Software versions
- Experimental procedures
- Raw results

The research process should ideally be:

Code+Data+Method→Reproducible Result

This improves the academic credibility of the program.

9.18 Version Control

Git-based version control should be introduced early in the program.

Students should learn:

- Repository creation
- Branches
- Commits
- Pull requests
- Merge conflicts
- Tags
- Releases
- Documentation

By graduation, students should have experience managing a complete software project.

9.19 Documentation Standards

Every major project should include a README containing:

Documentation should be considered part of engineering quality rather than an optional addition.

9.20 Industry Advisory Board

The program should establish an advisory group consisting of professionals from relevant industries.

Potential members could come from:

- Web technology
- Game development
- 3D visualization
- XR
- AI
- Design
- Architecture
- Digital media

The advisory board could meet annually.

Its role would be to evaluate:

- Graduate skills
 - Curriculum relevance
 - Emerging technologies
 - Employment trends
 - Industry expectations
-

9.21 Graduate Feedback

Program evaluation should continue after graduation.

Alumni surveys can investigate:

- Employment
- Job roles
- Skills used professionally
- Missing competencies
- Technologies encountered
- Further education

This creates an important feedback loop:

University→Graduate→Industry→Feedback→University

9.22 Employer Feedback

Employer feedback provides another source of evidence.

Employers can evaluate graduates according to:

- Programming ability
- Graphics knowledge
- Problem solving
- Communication
- Collaboration
- Research ability
- Technical adaptability

This information can identify weaknesses that internal academic evaluation may not reveal.

9.23 Annual Program Review

The program should conduct an annual review.

The review should examine:

Student Performance

Are students achieving the learning outcomes?

Course Performance

Which courses have unusually high failure rates?

Graduate Performance

Are graduates finding relevant employment?

Technology

Which technologies have become important?

Industry

What skills are employers requesting?

Research

What new research directions have emerged?

9.24 Curriculum Update Cycle

A practical cycle is:

This produces continuous improvement.

9.25 Technology Review

Because the program is technology-intensive, an annual technology review should identify:

- New browser APIs
- New GPU APIs
- New frameworks
- New AI systems
- New XR platforms
- New rendering techniques

However, a new technology should not automatically become a new course topic.

It should first be evaluated according to:

EducationalValue+IndustryRelevance+TechnicalMaturity

9.26 Example: WebGPU Curriculum Decision

Suppose WebGPU becomes significantly more important in professional development.

The university could evaluate:

Question 1

Does WebGPU represent a major technological shift?

Question 2

Does it provide educational value beyond WebGL?

Question 3

Are appropriate teaching resources available?

Question 4

Can faculty support the course?

If the answers are positive, WebGPU could gradually move from:

Introduction→AdvancedTopic→CoreTopic

rather than immediately replacing WebGL.

9.27 Example: Generative AI

The same principle applies to generative AI.

Rather than teaching a particular AI platform as a permanent course requirement, the curriculum should teach concepts such as:

- AI APIs
- Prompt engineering
- Multimodal systems
- AI-assisted programming
- Generative content
- AI evaluation
- Ethical considerations

Specific platforms can then be changed without redesigning the entire course.

9.28 Ethical Considerations

Interactive graphics and AI also introduce ethical issues.

Students should understand:

- Copyright
- Data privacy
- AI-generated content
- Model bias
- Accessibility
- Digital ownership
- Deepfakes
- User tracking

Technical capability must therefore be accompanied by responsible development.

9.29 Accessibility

Interactive 3D applications can create accessibility challenges.

Students should consider:

- Keyboard navigation
- Screen-reader compatibility
- Reduced-motion preferences
- Color contrast
- Alternative descriptions
- Performance on low-end devices

A useful principle is:

$Interactive \neq Inaccessible$

The objective should be to make advanced visual experiences accessible to the widest possible audience.

9.30 Performance as an Educational Outcome

Performance optimization should be treated as a fundamental competency.

Students should understand that a visually impressive application that performs poorly is not a successful real-time system.

The objective can be expressed as:

$Maximize(VisualQuality \times InteractionQuality)$

subject to:

$FrameTime \leq TargetFrameTime$

For a target of 60 FPS:

$TargetFrameTime \approx 16.67ms$

This provides students with a concrete engineering objective.

9.31 Recommended Performance Targets

The university does not need to impose identical requirements on every project, but suggested targets could be:

Application Type	Suggested Target
Desktop 3D	60 FPS
Mobile 3D	30–60 FPS
VR	Device-dependent high refresh rate
Interactive visualization	30+ FPS
Heavy scientific visualization	Graceful degradation

Students should document the hardware used to achieve the reported results.

9.32 Portfolio Assessment

The program should evaluate not only individual assignments but also the student's complete portfolio.

A graduating student should demonstrate progression:

Beginner → Intermediate → Advanced

The portfolio should show increasing complexity.

For example:

9.33 Graduate Competency Matrix

At graduation, students should ideally demonstrate the following:

Competency	Target
Programming	Advanced
Mathematics	Intermediate–Advanced
Web Development	Advanced
Computer Graphics	Advanced
WebGL	Advanced
Three.js	Advanced
Shader Programming	Intermediate–Advanced
Blender	Intermediate
GPU Programming	Intermediate
UX	Intermediate
XR	Introductory–Intermediate
AI Integration	Intermediate
Research	Advanced

This matrix can be used as a graduation-readiness assessment.

9.34 Program Success Indicators

The success of the program should be measured using multiple indicators.

Academic

- Course completion rate
- Student grades
- Thesis quality
- Research publications

Technical

- Project quality
- Performance benchmarks
- Software engineering quality

Career

- Employment rate
- Relevant employment
- Internship participation

Industry

- Employer satisfaction
- Industry partnerships
- External project participation

Student

- Student satisfaction
- Portfolio quality
- Competition participation

No single metric should determine program success.

9.35 Key Performance Indicators

A university could define a small set of KPIs.

For example:

KPI1\=GraduationRate KPI2\=EmploymentRate KPI3\=EmployerSatisfaction KPI4\=ThesisQuality KPI5
\=StudentProjectQuality KPI6\=IndustryParticipation

These should be reviewed annually.

9.36 Continuous Improvement Framework

The overall quality-assurance model can be represented as:

This creates a continuous quality loop.

9.37 Relationship to the Research

The quality-assurance framework is directly connected to the technical research conducted in earlier chapters.

The research demonstrated that interactive graphics systems must be evaluated according to:

- Visual quality
- Performance
- Interaction
- Technical architecture

- User experience

The same principles can be applied to the educational program.

Therefore:

TechnicalEvaluation→EducationalEvaluation

The curriculum itself becomes a system that can be measured and improved.

9.38 Long-Term Program Development

The IGWT program should eventually evolve beyond a six-semester undergraduate curriculum.

Potential future academic pathways include:

Undergraduate

Bachelor's degree in Interactive Graphics and Web Technologies.

Graduate

Master's degree in:

- Real-Time Graphics
- Interactive Computing
- Creative Technology
- XR
- GPU Computing

Doctoral Research

Research areas could include:

- Neural rendering
- Real-time global illumination
- GPU computing
- Web-based scientific visualization
- Generative 3D
- AI-driven interactive environments
- Human-computer interaction
- Spatial computing

Thus:

Bachelor→Master→Doctorate

can form a complete academic ecosystem.

9.39 International Collaboration

Because web technologies are globally distributed, international collaboration is particularly appropriate.

Potential activities include:

- Joint research projects
- Student exchange
- International hackathons
- Shared online courses
- Collaborative XR projects
- Open-source development

Students can therefore gain experience working with geographically distributed teams.

9.40 Open-Source Development

Open-source participation should be encouraged.

Students could contribute to:

- Graphics libraries
- Web frameworks
- Developer tools
- Shader repositories
- Visualization systems

This teaches professional collaboration while also exposing students to real-world software engineering practices.

9.41 Research Publication

Exceptional graduation projects may be developed into academic publications.

The progression could be:

StudentProject→Thesis→ConferencePaper→JournalPublication

This provides students with an opportunity to enter research careers.

9.42 Academic and Industrial Balance

The program must maintain a balance between academic depth and industry relevance.

If the curriculum focuses only on theory:

Theory□→Practice

If it focuses only on current tools:

Tools → LongTermKnowledge

The desired balance is:

Theory+Engineering+Design+Research

This balance is one of the central principles of the proposed educational model.

9.43 Chapter Conclusion

This chapter established a quality-assurance and continuous-improvement framework for the proposed Interactive Graphics and Web Technologies program.

The analysis demonstrates that a modern technology-oriented academic program cannot be designed as a static collection of courses.

Instead, it should operate as a continuously evaluated system.

The proposed framework connects:

Learning Outcomes → Assessment → Evidence → Analysis → Improvement

This process ensures that the program remains academically rigorous while adapting to changes in technology and industry.

The chapter also established the importance of:

- Course-level learning outcomes
- Program-level competencies
- Practical assessment
- Research evaluation
- Performance measurement
- User testing
- Industry feedback
- Alumni feedback
- Curriculum review
- Technology monitoring
- Accessibility
- Ethical development
- Reproducibility

The resulting model can be summarized as:

Teach → Measure → Evaluate → Improve

For the proposed IGWT program, this continuous process is particularly important because the underlying technological environment is evolving rapidly.

The long-term success of the program should therefore not be defined by whether it teaches a particular framework or API.

Instead, success should be measured by whether graduates can:

1. Understand fundamental principles.
2. Learn new technologies independently.
3. Build complex interactive systems.
4. Evaluate their own work scientifically.
5. Solve unfamiliar technical problems.
6. Conduct independent research.
7. Communicate technical results.
8. Adapt to emerging technologies.

The ultimate objective is consequently not merely to produce graduates who can build today's interactive web applications.

It is to produce graduates capable of **inventing and developing the interactive computing systems of tomorrow**.

Revised Thesis Structure

With Chapter 9 added, the thesis can now follow this structure:

Chapter	Title
1	Introduction
2	Literature Review and Related Technologies
3	Mathematical and Computer Graphics Foundations
4	System Architecture and Design
5	Implementation
6	Experimental Results and Discussion
7	Conclusion and Future Work
8	Curriculum Design and Educational Framework
9	Program Evaluation, Quality Assurance, and Continuous Improvement
10	References and Supporting Materials*

*Strictly speaking, **References should normally not be called a numbered "Chapter 10."** For a formal university thesis, I recommend treating *References* and *Appendices* as unnumbered back matter.

Recommended final structure

Main thesis

1. Introduction
2. Literature Review
3. Mathematical and Graphics Foundations
4. System Architecture

5. Implementation
6. Experimental Evaluation
7. Conclusion and Future Work
8. Curriculum Design
9. Program Evaluation and Quality Assurance

Back matter

Chapter 10

Implementation Roadmap, Institutional Deployment, and Long-Term Development

10.1 Introduction

The preceding chapters established the theoretical, technical, educational, and quality-assurance foundations of the proposed **Interactive Graphics and Web Technologies (IGWT)** program.

Chapter 8 defined the curriculum and educational framework, while Chapter 9 established mechanisms for evaluating and continuously improving the program.

However, a curriculum proposal is incomplete without a practical implementation strategy.

A university must determine:

- How the program should be introduced
- Which courses should be offered first
- What faculty are required
- What laboratory infrastructure is necessary
- How students should be admitted
- How existing facilities can be reused
- How costs can be controlled
- How the program should evolve over time

This chapter therefore presents a practical institutional roadmap for implementing IGWT.

The central principle is:

Planning→Pilot→Launch→Evaluate→Scale

The proposed roadmap is designed to reduce institutional risk while allowing the university to establish a distinctive specialization in interactive graphics and modern web technologies.

10.2 Implementation Objectives

The implementation strategy has six primary objectives.

Objective 1

Establish the academic foundation required for the program.

Objective 2

Develop sufficient faculty expertise.

Objective 3

Create an appropriate graphics and interactive-technology laboratory.

Objective 4

Launch the curriculum incrementally.

Objective 5

Establish industry and research partnerships.

Objective 6

Create a sustainable long-term development model.

10.3 Institutional Implementation Model

The proposed implementation model consists of five phases.

```
graph TD; P1[Phase 1  
Preparation] --> P2[Phase 2  
Pilot Courses]; P2 --> P3[Phase 3  
Program Launch]; P3 --> P4[Phase 4  
Evaluation]; P4 --> P5[Phase 5  
Expansion];
```

Phase 1
Preparation
↓
Phase 2
Pilot Courses
↓
Phase 3
Program Launch
↓
Phase 4
Evaluation
↓
Phase 5
Expansion

This approach reduces the risk associated with introducing an entirely new academic program.

10.4 Phase 1 — Preparation

The preparation phase should occur before students are formally admitted to the new program.

The university should establish:

- Program committee
- Curriculum committee
- Faculty assignments

- Laboratory requirements
- Course specifications
- Assessment policies
- Industry advisory structure

The first task is to define the program's academic identity.

10.5 Program Committee

A dedicated program committee should oversee the development process.

The committee could consist of:

- Program director
- Computer science faculty
- Graphics specialist
- Web technology specialist
- Design faculty
- AI specialist
- XR specialist
- Industry representative

The committee should be responsible for:

- Curriculum approval
- Course coordination
- Faculty planning
- Quality assurance
- Industry engagement
- Annual review

10.6 Program Director

A program director should coordinate the entire specialization.

The director should ideally possess experience in several of the following:

- Computer graphics
- Web development
- Software engineering
- Interactive media
- Academic curriculum development
- Research supervision

The role includes both academic and administrative responsibilities.

10.7 Faculty Development

Because the program spans multiple disciplines, faculty development is essential.

Not every instructor needs to master every technology.

Instead, expertise should be distributed.

For example:

Area	Primary Faculty Expertise
Programming	Software Engineering
Mathematics	Mathematics / Graphics
Graphics	Computer Graphics
Web	Web Engineering
WebGL	Graphics / Web
Three.js	Interactive Development
Blender	3D / Technical Art
Shaders	Graphics / GPU
XR	XR / Interaction
AI	AI / Machine Learning
Research	Research Faculty

10.8 Faculty Training

Before program launch, faculty should receive opportunities to become familiar with the proposed technology stack.

Training may include:

- WebGL workshops
- Three.js workshops
- Blender workshops
- Shader programming workshops
- React Three Fiber workshops
- WebGPU seminars
- AI development workshops

The objective is not to turn every professor into a specialist.

Instead:

Faculty Collaboration > Individual Expertise

should be the guiding principle.

10.9 Laboratory Requirements

Interactive graphics education requires appropriate hardware.

A graphics laboratory should include:

- GPU-equipped desktop computers
- High-resolution monitors
- Headphones
- Cameras
- Mobile devices
- VR-capable systems
- Network infrastructure

A laboratory workstation should ideally support real-time 3D development and GPU experimentation.

10.10 Recommended Laboratory Configuration

A representative workstation could include:

- Modern multi-core CPU
- 32 GB or more RAM
- Dedicated GPU
- NVMe SSD
- High-resolution display
- Reliable network connection

The exact hardware should be updated according to the university's budget and the graphics workloads expected.

10.11 Hardware Strategy

The university does not need to purchase the most expensive GPUs available.

A tiered strategy is more practical.

Tier 1 — General Development

Standard university computers.

Tier 2 — Graphics Development

Dedicated GPU workstations.

Tier 3 — Advanced Research

High-performance GPU systems.

Tier 4 — XR

VR/AR-capable workstations and headsets.

This allows expensive hardware to be concentrated where it provides the greatest educational benefit.

10.12 Cloud Infrastructure

Cloud infrastructure can supplement local laboratories.

Students can use cloud systems for:

- GPU computation
- Large-scale rendering
- AI experimentation
- Deployment
- Data processing

This creates a hybrid infrastructure:

Local GPU+Cloud GPU

The local laboratory remains the primary development environment, while cloud resources provide additional capacity.

10.13 Software Infrastructure

The university should establish a standardized development environment.

Recommended components include:

```
Operating System
  ↓
Node.js / Package Manager
  ↓
Git
  ↓
VS Code or equivalent IDE
  ↓
React
  ↓
Three.js
  ↓
React Three Fiber
  ↓
WebGL / WebGPU
  ↓
Blender
```

Students should be encouraged to use current stable versions rather than fixed versions that become obsolete.

10.14 Version Management

Because software changes rapidly, courses should maintain version documentation.

Each semester should identify:

- Node.js version
- Browser versions
- Framework versions
- Blender version
- Graphics API support
- AI API versions

However, instructors should distinguish between:

Concept

and

Version

The concept should remain stable even when the implementation changes.

10.15 Open-Source Strategy

Where possible, the program should prioritize open-source technologies.

Advantages include:

- Low licensing cost
- Transparency
- Community support
- Student accessibility
- Industry relevance
- Source-code availability

Examples include:

- JavaScript
- TypeScript
- React
- Three.js
- Blender
- WebGL
- WebGPU
- Git

Commercial tools can still be introduced where educationally appropriate.

10.16 Course Development Process

Before a course is offered, the instructor should prepare a standardized course package.

The package should contain:

1. Course Description

Purpose and scope.

2. Learning Outcomes

Measurable competencies.

3. Weekly Schedule

Lecture and laboratory topics.

4. Assessment Plan

Assignments, examinations, and projects.

5. Required Software

Development environment.

6. Required Hardware

GPU and other requirements.

7. Reading Materials

Books, papers, and technical documentation.

8. Project Specification

Final deliverable.

10.17 Example Course Preparation

For **Shader Programming**, the professor should prepare:

Lecture Materials

- GPU pipeline diagrams
- GLSL examples
- Mathematical explanations
- Rendering demonstrations

Laboratory

- Basic vertex shader

- Fragment shader
- UV manipulation
- Lighting
- Noise
- Procedural effects

Assessment

- Weekly shader exercises
- Midterm shader project
- Final procedural graphics project

Final Deliverable

A working shader-based interactive application with technical documentation.

10.18 Semester Implementation

The program should initially launch with a limited number of students.

A smaller first cohort provides several advantages:

- Easier faculty support
- Better laboratory management
- More detailed student feedback
- Lower operational risk
- Easier curriculum correction

After the first cohort completes the initial year, the program can be evaluated before scaling.

10.19 Pilot Course Strategy

Before launching the complete degree program, the university could offer selected courses as electives.

Recommended pilot courses include:

1. Creative Coding
2. Interactive Web Development
3. Three.js Development
4. Shader Programming

These courses can test:

- Student interest
- Faculty readiness
- Laboratory infrastructure
- Teaching materials

- Industry demand
-

10.20 Pilot Evaluation

The pilot should collect:

Student Data

- Enrollment
- Attendance
- Completion
- Grades
- Satisfaction

Technical Data

- Project performance
- Software compatibility
- Hardware performance

Faculty Data

- Teaching workload
- Difficulty
- Material quality

Industry Data

- Relevance
- Skill expectations

The results should be used before full-scale implementation.

10.21 Three-Year Implementation Roadmap

A practical initial roadmap could be:

Year 0

Preparation.

- Curriculum design
- Faculty training
- Laboratory planning
- Course development
- Industry consultation

Year 1

Foundation launch.

- Programming
- Web technologies
- Mathematics
- Computer graphics

Year 2

Advanced technical courses.

- WebGL
- Three.js
- Blender
- Shader programming
- Real-time rendering

Year 3

Advanced specialization.

- XR
- AI
- Advanced graphics
- Capstone
- Graduation thesis

This corresponds naturally to the six-semester structure.

10.22 Five-Year Development Roadmap

A longer-term strategy can be represented as:

Year 1
Foundation
↓
Year 2
Graphics Specialization
↓
Year 3
First Graduating Cohort
↓
Year 4
Research Expansion
↓
Year 5
International / Graduate Program

By Year 5, the university could consider establishing a dedicated research laboratory.

10.23 Research Laboratory

A potential research laboratory could be called:

Interactive Graphics and Intelligent Systems Laboratory

Possible research areas include:

- Real-time rendering
- WebGPU
- Neural rendering
- Generative 3D
- XR
- AI agents
- Scientific visualization
- Digital twins
- Interactive simulation

The laboratory would connect undergraduate education with postgraduate research.

10.24 Undergraduate Research

Research should not be limited to graduate students.

Undergraduates can participate through:

- Research assistantships
- Independent studies
- Thesis projects

- Laboratory projects
- Open-source contributions

A student might begin with:

Small Shader Project

and eventually develop:

Research Prototype

This creates a research-oriented learning environment.

10.25 Industry Partnership Model

Industry collaboration should be structured rather than informal.

A partnership framework could include:

University

Provides students and research capabilities.

Industry

Provides real-world problems and technical expertise.

Joint Activity

Creates:

- Student projects
- Internships
- Research
- Workshops
- Demonstrations

The relationship can be represented as:

University↔Industry

with students acting as the central participants.

10.26 Internship Program

Students should ideally complete an internship before graduation.

Relevant organizations may include:

- Software companies
- Game studios
- Digital agencies

- Architecture firms
- Automotive companies
- XR companies
- Visualization companies
- AI companies

Internships allow students to experience production constraints that are difficult to reproduce in a classroom.

10.27 Industry Project

An alternative or complementary approach is to introduce industry-sponsored projects.

For example:

Develop a browser-based 3D configurator for a commercial product.

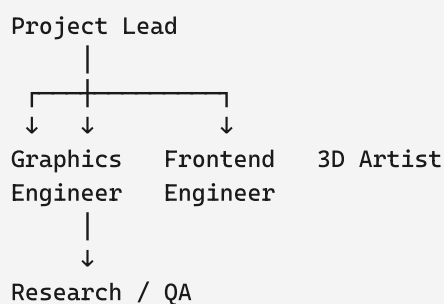
Students would work under academic supervision while receiving requirements from an industry partner.

This provides an authentic development experience.

10.28 Student Team Structure

Large projects should use professional team structures.

A team might contain:



Students can rotate roles between projects.

10.29 Professional Development

Technical skills alone are insufficient.

Students should also learn:

- Technical communication

- Presentation
- Documentation
- Teamwork
- Project management
- Version control
- Client communication
- Research communication

These skills significantly improve graduate employability.

10.30 Portfolio and Career Services

The university should help students transform academic projects into professional portfolios.

Each student should graduate with:

- Portfolio website
- Git repositories
- Project videos
- Technical case studies
- Thesis
- Resume
- Demonstration projects

The student's academic record should therefore become a professional record.

10.31 Program Branding

Because IGWT is interdisciplinary, its public identity should be clearly communicated.

A possible positioning statement is:

Interactive Graphics and Web Technologies — Engineering the Next Generation of Digital Experiences.

The program should communicate that it is neither simply:

- Web development

nor simply:

- Computer graphics.

Instead, it is the intersection:

Web∩Graphics∩Interaction∩GPU∩AI

10.32 Admissions Strategy

The program should accept students with diverse backgrounds.

Potential applicants may come from:

- Computer science
- Software engineering
- Digital media
- Design
- Mathematics
- Engineering

The admissions process should prioritize:

- Logical thinking
- Problem solving
- Technical curiosity
- Visual creativity

Advanced graphics knowledge should not necessarily be required at admission.

10.33 Bridging Courses

Students without strong mathematics or programming backgrounds may require preparatory courses.

Possible bridging subjects include:

- Programming fundamentals
- Algebra
- Trigonometry
- Digital literacy

This makes the program more accessible without lowering the standards of the core curriculum.

10.34 Student Support

Because graphics programming can be technically demanding, students should have access to:

- Teaching assistants
- Laboratory assistants
- Office hours
- Peer mentoring
- Online documentation
- Example projects

A mentorship model can significantly improve retention.

10.35 Teaching Assistants

Advanced students can support introductory courses.

For example:

Third-year students can assist:

- Programming
- Web development
- Three.js

This creates a knowledge-transfer mechanism:

SeniorStudents→JuniorStudents

and provides teaching experience to senior students.

10.36 Financial Sustainability

A new program should be financially sustainable.

Costs include:

Initial Costs

- GPU workstations
- VR equipment
- Software
- Faculty development
- Laboratory setup

Continuing Costs

- Hardware replacement
- Software maintenance
- Cloud resources
- Faculty training
- Equipment upgrades

The program should therefore prioritize technologies with low recurring licensing costs.

10.37 Equipment Lifecycle

Graphics hardware evolves rapidly.

The university should establish a replacement strategy.

For example:

3–5 Year HardwareCycle

can be considered depending on budget and workload.

Older systems should not necessarily be discarded.

They can be used for:

- Performance comparison
 - Low-end testing
 - Browser compatibility
 - Optimization experiments
-

10.38 Accessibility of Infrastructure

Students should not be dependent exclusively on expensive personal computers.

The university should provide:

- Laboratory access
- Remote access where possible
- Shared GPU resources
- Cloud resources
- Loaner devices

This ensures that financial differences do not become a barrier to participation.

10.39 Security and Privacy

Interactive systems can process user data.

Students should therefore learn:

- Secure authentication
- API security
- Data privacy
- Secure deployment
- Dependency management
- Content security policies

AI applications require additional consideration of data transmission and third-party services.

10.40 Deployment Education

Students should not stop at local development.

They should learn how to deploy applications.

Topics include:

- Production builds
- Hosting
- CDN
- HTTPS
- Asset optimization
- Caching
- Monitoring

The development lifecycle becomes:

Develop→Build→Deploy→Monitor→Improve

10.41 Cloud Deployment

Students can deploy projects using modern cloud infrastructure.

Applications may include:

- Static hosting
- Serverless APIs
- Cloud databases
- Object storage
- CDN
- GPU services

The objective is to expose students to real-world deployment environments.

10.42 Open Demonstration Platform

The university could maintain a public project gallery.

Each graduating project could be published with:

- Project title
- Student name
- Description
- Technology stack
- Demonstration
- Source code where appropriate
- Thesis abstract

This provides:

1. Student visibility
2. Program marketing

- 3. Industry engagement
 - 4. Research dissemination
-

10.43 Annual IGWT Exhibition

A yearly exhibition could showcase the strongest student projects.

Possible categories include:

- Best Interactive Experience
- Best Graphics Project
- Best Shader
- Best AI Application
- Best XR Project
- Best Research Project
- Best Technical Achievement

Such an event would strengthen the identity of the program.

10.44 Student Competition

The university could establish an annual graphics competition.

Possible challenge:

Create an interactive 3D experience under a specified technical constraint.

Constraints might include:

- Maximum download size
- Mobile compatibility
- Limited polygon count
- Real-time rendering requirement

This encourages students to optimize rather than simply increase complexity.

10.45 Benchmark Competition

Another educational activity could involve performance optimization.

Students receive the same scene and attempt to maximize performance.

The competition could measure:

FPS FrameTime Memory

while maintaining a minimum visual-quality score.

This makes optimization an engaging practical exercise.

10.46 Long-Term Academic Vision

The long-term objective is to establish the university as a recognized center for interactive computing.

The progression could be:

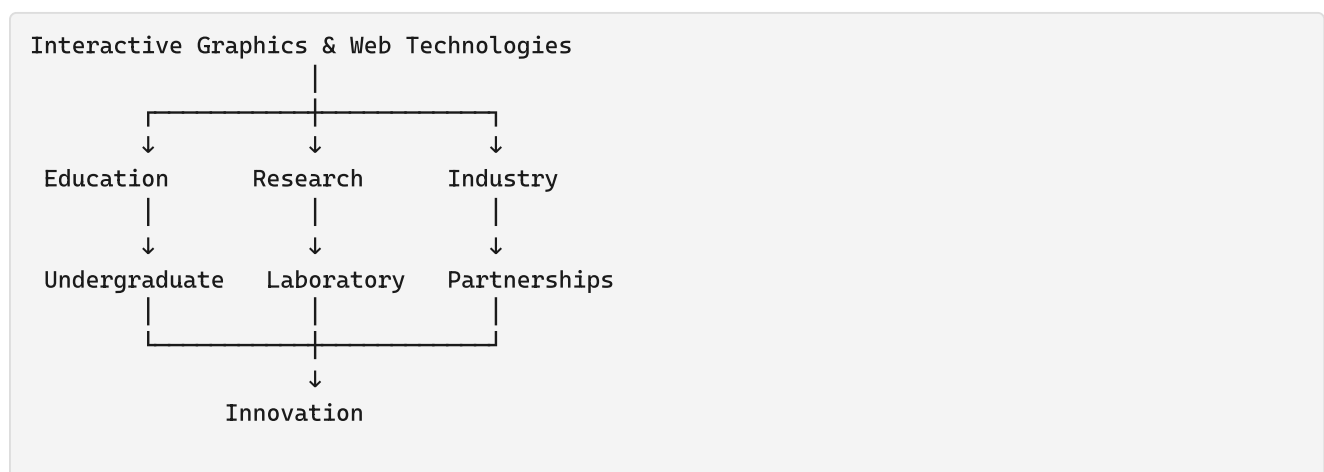
Course→Program→Laboratory→ResearchCenter

The program could eventually support:

- Undergraduate education
 - Graduate research
 - Industry partnerships
 - International collaboration
 - Open-source projects
 - Commercial technology transfer
-

10.47 Proposed Institutional Structure

A mature version of the program could contain:



This transforms the curriculum into a broader institutional ecosystem.

10.48 Risk Management

The implementation of a new technology program involves several risks.

Risk 1 — Technology Changes Too Quickly

Solution: emphasize fundamentals and perform annual technology reviews.

Risk 2 — Insufficient Faculty Expertise

Solution: faculty development and interdisciplinary teaching.

Risk 3 — High Hardware Costs

Solution: tiered laboratories and cloud infrastructure.

Risk 4 — Student Difficulty

Solution: bridging courses and teaching assistants.

Risk 5 — Industry Mismatch

Solution: advisory board and employer feedback.

Risk 6 — Curriculum Overload

Solution: prioritize fundamental competencies rather than attempting to teach every new technology.

10.49 Risk Matrix

Risk	Probability	Impact	Mitigation
Technology obsolescence	High	High	Annual review
Hardware cost	Medium	High	Tiered infrastructure
Faculty shortage	Medium	High	Training/recruitment
Student difficulty	Medium	Medium	Bridging support
Industry mismatch	Medium	High	Advisory board
Software incompatibility	Medium	Medium	Version management
Infrastructure failure	Low	High	Backup systems
Curriculum overload	High	Medium	Competency prioritization

10.50 Implementation Success Criteria

The program can be considered successfully implemented when the following conditions are achieved:

Academic

- All core courses are operational.
- Learning outcomes are measurable.
- Assessment rubrics are established.

Technical

- Graphics laboratories are operational.

- Students can access required software and hardware.

Student

- Students complete progressively more sophisticated projects.
- Graduation projects demonstrate independent technical capability.

Research

- Students complete research-based theses.
- Faculty research activity increases.

Industry

- Internship and industry partnerships are established.

Quality

- Annual program review is functioning.

10.51 Five-Year Target Model

A possible five-year target is:

Year	Major Objective
Year 1	Launch foundational curriculum
Year 2	Introduce graphics specialization
Year 3	Graduate first cohort
Year 4	Expand research and industry partnerships
Year 5	Establish recognized graphics/interactive research center

The exact schedule should depend on institutional resources.

10.52 Future Expansion

Once the undergraduate program becomes established, several specialized tracks could be introduced.

Track A — Real-Time Graphics

- Rendering
- GPU programming
- WebGPU
- Optimization

Track B — Creative Technology

- Creative coding
- Interaction
- Motion
- Generative graphics

Track C — XR

- VR
- AR
- Spatial computing

Track D — AI Graphics

- Generative 3D
- Neural rendering
- AI agents
- Computer vision

Track E — Visualization

- Scientific visualization
- Data visualization
- Digital twins

This allows students to specialize while retaining a common foundation.

10.53 Proposed Graduate Research Areas

The mature program could support research in:

Real-Time Rendering

Efficient rendering algorithms for browsers and lightweight devices.

Neural Rendering

AI-assisted generation and reconstruction of 3D environments.

WebGPU

Advanced browser-based GPU computation.

XR

Immersive interactive experiences.

AI Agents

Natural-language control of visual environments.

Scientific Visualization

Interactive visualization of complex datasets.

Human-Computer Interaction

New interaction paradigms for 3D web applications.

10.54 Relationship Between Education and Research

A strong program should create a continuous relationship between education and research.

Students learn:

Theory

then:

Implementation

then:

Experiment

then:

Research

Faculty research can subsequently feed new material back into teaching.

Thus:

Research→Teaching→StudentProjects→Research

creates a sustainable academic ecosystem.

10.55 Relationship Between Education and Industry

The same principle applies to industry.

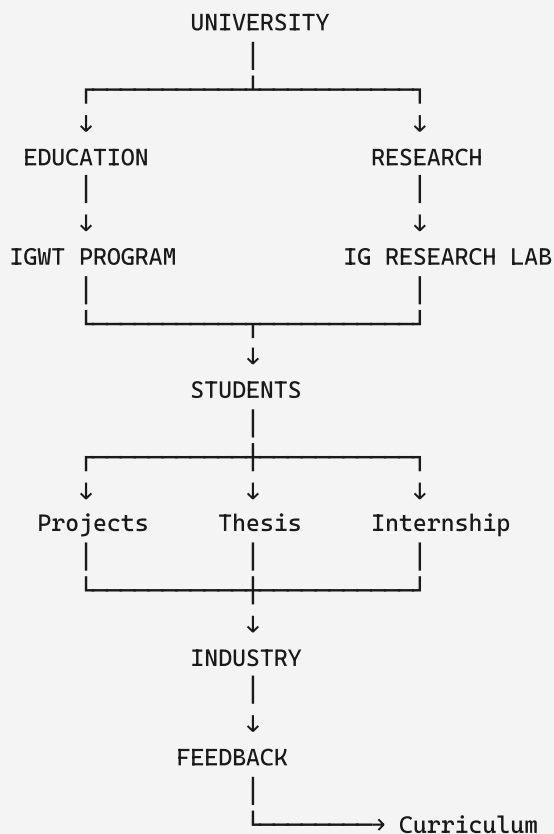
The cycle becomes:

IndustryNeeds→Curriculum→StudentSkills→Graduates→Industry

The program therefore remains relevant without becoming dependent on any individual company or technology.

10.56 Final Implementation Model

The complete institutional model can be summarized as:



This creates a continuous relationship among education, research, students, and industry.

10.57 Chapter Conclusion

This chapter presented a practical implementation roadmap for establishing the proposed **Interactive Graphics and Web Technologies** program within a university environment.

The analysis demonstrates that successful implementation requires more than a well-designed curriculum.

It requires:

- Faculty preparation
- Laboratory infrastructure
- Software infrastructure
- Student support
- Industry partnerships
- Research integration
- Quality assurance
- Financial planning
- Continuous technological review

The recommended strategy is incremental:

Prepare→Pilot→Launch→Evaluate→Expand

This minimizes institutional risk while allowing the university to build expertise progressively.

The proposed program can begin with relatively accessible technologies such as JavaScript, WebGL, Three.js, Blender, and React, while gradually incorporating WebGPU, XR, AI, neural rendering, and other emerging technologies.

The most important implementation principle is therefore not to create a curriculum that attempts to predict every future technology.

Instead, the university should establish a strong conceptual foundation that allows students and faculty to adapt continuously.

The program should ultimately produce graduates capable of moving from:

Concept→Implementation→Optimization→Research

while connecting technical capability with design and human experience.

The long-term institutional vision is consequently:

University Program→Research Laboratory→Industry Ecosystem→Innovation Center

Such a structure would allow the university not only to teach interactive graphics and web technologies but also to become an active contributor to the development of next-generation interactive computing.

Recommended Final Thesis Organization

At this point, however, I would make an important academic correction to the structure we've been building.

Chapter 10 should probably be the final substantive chapter only if this is a curriculum-development dissertation. If this document is intended as a conventional computer-science thesis, ten chapters may be unnecessarily long.

For the **university curriculum proposal thesis**, the following structure is now coherent:

1. **Introduction**
2. **Literature Review and Related Technologies**
3. **Mathematical and Computer Graphics Foundations**
4. **System Architecture and Design**
5. **Implementation**
6. **Experimental Results and Discussion**
7. **Conclusion and Future Work**
8. **Curriculum Design and Educational Framework**
9. **Program Evaluation, Quality Assurance, and Continuous Improvement**
10. **Implementation Roadmap, Institutional Deployment, and Long-Term Development**

Then:

References

Chapter 11

Overall Conclusion, Recommendations, and Future Research

11.1 Introduction

This thesis proposed a new interdisciplinary academic program entitled **Interactive Graphics and Web Technologies (IGWT)**.

The primary objective of the proposed program is to address the growing intersection between:

- Computer science
- Computer graphics
- Web technologies
- GPU computing
- Interactive media
- Extended reality
- Artificial intelligence
- Human-computer interaction

Traditional computer science programs generally provide strong foundations in programming, algorithms, databases, operating systems, and software engineering. Graphics-oriented programs, on the other hand, may focus heavily on mathematical foundations, rendering, modeling, and visual computing.

However, the rapid development of browser-based GPU technologies has created a significant opportunity between these disciplines.

Modern web applications are increasingly capable of supporting:

- Real-time 3D graphics
- Interactive visualization
- GPU computation
- Immersive experiences
- AI-assisted interfaces
- Scientific visualization
- Digital twins
- Spatial computing

The proposed IGWT program is designed to prepare students for this emerging technological environment.

11.2 Summary of the Research

The thesis investigated the academic and technical foundations necessary for constructing a modern interactive graphics curriculum.

The proposed educational model can be summarized as:

Programming+Mathematics+Graphics+Web+GPU+Interaction+AI

Rather than treating these disciplines as independent subjects, the program connects them progressively.

The curriculum begins with fundamental programming and mathematics and gradually advances toward:

WebGL→Three.js→Shaders→GPUProgramming→Real–TimeRendering→XR→AI

This progression allows students to understand not only how to use existing frameworks but also the underlying principles that make those frameworks possible.

11.3 Main Findings

Several major conclusions emerge from the proposed curriculum model.

Finding 1 — Mathematical Foundations Are Essential

Computer graphics should not begin directly with a high-level 3D framework.

Students first need to understand:

- Vectors
- Matrices
- Coordinate systems
- Transformations
- Geometry
- Trigonometry
- Linear algebra

For example, a transformation can be represented as:

$$p' = Mp$$

where:

- p is the original position,
- M is a transformation matrix,
- p' is the transformed position.

Understanding this relationship allows students to reason about graphics systems rather than merely memorizing API functions.

11.4 Finding 2 — Graphics Should Be Integrated With Web Development

Modern interactive experiences increasingly exist within web browsers.

Therefore, computer graphics education should not be isolated from web technologies.

The browser should be treated as a complete application platform involving:

HTML+CSS+JavaScript+GPU+Network+Interaction

This integration is one of the defining characteristics of the proposed IGWT program.

11.5 Finding 3 — WebGL and Shaders Provide an Important Bridge

High-level libraries such as Three.js are valuable for rapid development.

However, students who only use high-level abstractions may not understand:

- Rendering pipelines
- GPU execution
- Buffers
- Textures
- Shader stages
- Coordinate transformations
- Fragment processing

Therefore, the curriculum deliberately introduces WebGL and shader programming before advanced real-time rendering.

The conceptual progression is:

HighLevel→LowLevel→HighLevel

Students first learn how graphics systems work, then use abstraction layers more effectively.

11.6 Finding 4 — Real-Time Rendering Should Be Treated as an Engineering Discipline

A graphics application is not successful merely because it looks visually impressive.

It must also satisfy performance constraints.

For a target frame rate F , the available frame time is approximately:

$T_{\text{frame}} = \frac{1}{F}$

For 60 FPS:

Tframe≈16.67ms

Students should therefore learn to evaluate:

- Frame rate
- Frame time
- GPU utilization
- CPU utilization
- Memory usage
- Draw calls
- Triangle count
- Texture size
- Loading time

Graphics education should consequently include both visual quality and computational efficiency.

11.7 Finding 5 — Creative Technology Requires Both Engineering and Design

Interactive graphics sits between technical engineering and visual communication.

A successful application must consider:

TechnicalQuality+VisualQuality+InteractionQuality

A technically correct system may still fail if it provides a poor user experience.

Conversely, a visually attractive system may fail if performance is inadequate.

The curriculum therefore combines:

- Programming
 - Graphics
 - Design
 - Animation
 - Interaction
 - UX
-

11.8 Finding 6 — Project-Based Learning Is Essential

Interactive graphics is particularly suitable for project-based education.

Students learn concepts more effectively when they must apply them to a functioning system.

A typical learning progression can be:

Concept→Exercise→MiniProject→MajorProject→Thesis

This provides increasing levels of independence.

11.9 Finding 7 — Students Need Research Skills

The program should not produce graduates who are only framework users.

Students should learn how to:

1. Identify a problem.
2. Search the literature.
3. Formulate a research question.
4. Develop a methodology.
5. Implement a prototype.
6. Conduct experiments.
7. Analyze results.
8. Discuss limitations.
9. Present conclusions.

This transforms students from technology consumers into technology developers and researchers.

11.10 Finding 8 — AI Should Be Integrated Rather Than Isolated

Artificial intelligence is increasingly influencing interactive development.

However, AI should not replace foundational computer science education.

Instead:

AI+Fundamentals>Alone

Students should understand how to use AI for:

- Code generation
- Content creation
- Image generation
- 3D asset generation
- Data analysis
- Interaction
- Prototyping

while also understanding:

- Model limitations
- Copyright
- Privacy
- Bias

- Reliability
 - Evaluation
-

11.11 Finding 9 — WebGPU Represents an Important Future Direction

The curriculum should introduce students to the evolution of browser graphics APIs.

The progression can be represented conceptually as:

Canvas→WebGL→WebGPU

WebGL remains important for understanding the established browser graphics model.

WebGPU, however, provides a more modern architecture for GPU programming.

Students should therefore understand both historical foundations and emerging systems.

11.12 Finding 10 — The Curriculum Must Be Continuously Updated

A fixed technology curriculum will eventually become obsolete.

Therefore, the curriculum should distinguish between:

Stable Knowledge

- Mathematics
- Algorithms
- Rendering concepts
- GPU architecture
- Interaction principles

and:

Evolving Technologies

- Frameworks
- APIs
- AI models
- Libraries
- Development tools

The first category should remain central.

The second should be updated periodically.

11.13 Educational Model

The proposed educational philosophy can be expressed as:

Learn→Build→Measure→Reflect→Improve

This cycle should operate at every level.

Course Level

Students build small projects.

Program Level

Students complete integrated projects.

Thesis Level

Students conduct independent research.

Professional Level

Graduates continue learning throughout their careers.

11.14 Graduate Profile

A graduate of the IGWT program should be capable of working across several technical domains.

The ideal graduate profile includes:

Software Engineering

Ability to design and implement maintainable software.

Web Engineering

Ability to build modern browser-based applications.

Computer Graphics

Understanding of rendering and visualization.

GPU Programming

Understanding of parallel graphics computation.

Interaction Design

Ability to create effective user interactions.

3D Development

Ability to construct real-time 3D applications.

AI Integration

Ability to integrate modern AI capabilities.

Research

Ability to independently investigate technical problems.

11.15 Graduate Competency Model

The graduate competency model can be expressed as:

$$G = P + M + G + W + GPU + UX + AI + R$$

where:

- P = Programming
- M = Mathematics
- G = Graphics
- W = Web
- GPU = GPU Computing
- UX = User Experience
- AI = Artificial Intelligence
- R = Research

The goal is not necessarily to maximize every component equally.

Instead, students should develop a strong interdisciplinary core while specializing in one or more areas.

11.16 Proposed Graduate Specializations

After completing the common curriculum, students could focus on specialized areas.

Specialization A — Real-Time Graphics

Topics:

- Rendering
- Shaders
- WebGPU
- GPU optimization
- PBR

Specialization B — Creative Technology

Topics:

- Creative coding

- Generative graphics
- Motion
- Interactive installations

Specialization C — XR

Topics:

- VR
- AR
- WebXR
- Spatial interaction

Specialization D — AI Graphics

Topics:

- Generative 3D
- Neural rendering
- Computer vision
- AI agents

Specialization E — Visualization

Topics:

- Scientific visualization
- Data visualization
- Simulation
- Digital twins

11.17 Graduation Requirements

The thesis proposes that students should not graduate solely on the basis of examinations.

A complete graduation requirement should include four components:

CourseCompletion+Portfolio+Capstone+Thesis

Course Completion

All required courses must be successfully completed.

Portfolio

Students demonstrate accumulated technical work.

Capstone

Students develop a substantial interactive application.

Thesis

Students conduct independent research.

This model provides stronger evidence of professional readiness.

11.18 Graduation Thesis Requirements

The graduation thesis should include:

1. Research problem
2. Background
3. Literature review
4. Research question
5. Methodology
6. System design
7. Implementation
8. Experiment
9. Results
10. Discussion
11. Limitations
12. Conclusion
13. References

A typical thesis may contain approximately:

50–80 pages

depending on institutional requirements.

The page count should not, however, become the primary measure of academic quality.

11.19 Recommended Thesis Types

Students should be allowed to choose among several research formats.

Type A — Experimental

Example:

Performance comparison of WebGL and WebGPU rendering approaches.

Type B — System Development

Example:

Development of a browser-based interactive scientific visualization platform.

Type C — Human-Computer Interaction

Example:

Evaluation of interaction techniques for browser-based 3D environments.

Type D — Graphics Research

Example:

Procedural generation of real-time environments using signed distance fields.

Type E — AI + Graphics

Example:

AI-assisted generation of interactive 3D web content.

11.20 Evaluation of the Proposed Program

The program should be evaluated using a combination of quantitative and qualitative measures.

Quantitative measures include:

- Graduation rate
- Course completion
- Project performance
- Employment
- Internship participation
- Research output

Qualitative measures include:

- Student feedback
- Faculty observations
- Employer feedback
- External academic review

A combined model can be represented as:

$\text{ProgramQuality} = Q_{\text{academic}} + Q_{\text{technical}} + Q_{\text{industry}} + Q_{\text{student}}$

11.21 Industry Relevance

The program is designed to address a growing convergence of technologies.

The modern interactive developer may increasingly need to understand:

Web+3D+GPU+AI+XR

This interdisciplinary skill set can support careers in:

- Front-end engineering
- Creative development
- Graphics engineering
- Visualization
- Game technology
- XR development
- Digital media
- AI application development
- Technical art
- Simulation

The program therefore provides several potential career pathways rather than training students for a single job title.

11.22 Limitations of the Proposed Curriculum

The proposed curriculum also has limitations.

Limitation 1 — Technological Change

Specific frameworks and APIs may become obsolete.

Limitation 2 — Resource Requirements

Advanced graphics education requires capable hardware.

Limitation 3 — Faculty Expertise

The interdisciplinary nature of the program makes faculty recruitment more difficult.

Limitation 4 — Curriculum Density

The number of relevant technologies is large.

Limitation 5 — Student Background

Students enter university with different programming and mathematical abilities.

These limitations should be addressed through the continuous-improvement framework described in Chapter 9.

11.23 Recommendations

Based on the analysis, the following recommendations are proposed.

Recommendation 1

Establish computer graphics as a core component rather than an elective topic.

Recommendation 2

Require mathematics before advanced graphics programming.

Recommendation 3

Teach both low-level graphics APIs and high-level frameworks.

Recommendation 4

Make shader programming a core competency.

Recommendation 5

Include real-time rendering and GPU optimization.

Recommendation 6

Integrate 3D asset creation through Blender or equivalent tools.

Recommendation 7

Use project-based learning throughout the curriculum.

Recommendation 8

Require a substantial graduation project.

Recommendation 9

Require an academic graduation thesis.

Recommendation 10

Establish continuous curriculum review.

11.24 Recommendation for the Graduation Deliverable

For this particular program, I recommend that the final graduation requirement be more substantial than a conventional written thesis.

The ideal final submission is:

Thesis+Interactive Application+Source Code+Experimental Data+Presentation

For example, a student might produce:

Written Thesis

50–80 pages.

Interactive System

A functioning web-based graphics application.

Source Code

A version-controlled repository.

Research Data

Benchmark results, user-study results, or experimental data.

Demonstration

A live or recorded demonstration.

Defense

A formal presentation and examination.

This is particularly appropriate for an applied graphics program.

11.25 Example Graduation Project

A student could develop:

A WebGPU-Based Interactive Scientific Visualization System for Three-Dimensional Medical Data

The project could contain:

Research Question

Can GPU-accelerated browser rendering provide interactive visualization of complex three-dimensional datasets while maintaining acceptable frame rates?

Implementation

- TypeScript
- WebGPU
- React
- 3D rendering
- GPU compute
- Interactive UI

Evaluation

Measure:

FPS FrameTime Memory DatasetSize

User Evaluation

Measure:

- Task completion
- Interaction time
- User satisfaction

Final Deliverables

- Thesis
- Source code
- Demonstration
- Experimental data
- Technical documentation

This would constitute a genuine academic and engineering contribution.

11.26 Future Research Direction 1 — Neural Rendering

One major research direction is the integration of machine learning with real-time graphics.

Future systems may combine:

TraditionalRendering+NeuralNetworks

Potential topics include:

- Neural texture generation
- Neural super-resolution
- Neural scene representation
- AI-assisted materials
- Generative 3D environments

This could become an important graduate-level research area.

11.27 Future Research Direction 2 — Generative 3D

Generative AI is rapidly expanding from 2D image generation toward 3D content.

Future research could investigate:

Text→3D→InteractiveScene

A complete pipeline could potentially become:

```
graph TD; A[Natural Language] --> B[AI Generation]; B --> C[3D Asset]; C --> D[Optimization]; D --> E[Real-Time Engine]; E --> F[Web Application];
```

Natural Language
↓
AI Generation
↓
3D Asset
↓
Optimization
↓
Real-Time Engine
↓
Web Application

This represents a significant intersection between AI and interactive graphics.

11.28 Future Research Direction 3 — AI Agents and Interactive Environments

Another direction is the integration of AI agents into 3D environments.

For example:

User↔AI Agent↔3D Environment

Such systems could be applied to:

- Education
- Simulation
- Virtual museums
- Games
- Training
- Digital assistants

Students could study how natural-language interaction can control visual environments.

11.29 Future Research Direction 4 — Spatial Computing

The convergence of:

- AR
- VR
- Web
- Computer vision
- AI

creates new possibilities for spatial interfaces.

The research model becomes:

PhysicalWorld+DigitalWorld→SpatialComputing

Future IGWT programs should therefore continue expanding beyond conventional screens.

11.30 Future Research Direction 5 — Digital Twins

Interactive graphics can also support digital-twin systems.

A digital twin may combine:

PhysicalObject+SensorData+3DModel+Simulation

A web-based implementation could allow users to interact with a digital representation through a browser.

Potential applications include:

- Manufacturing
 - Architecture
 - Smart cities
 - Transportation
 - Energy
 - Healthcare
-

11.31 Future Research Direction 6 — Scientific Visualization

Web-based GPU technologies provide an opportunity to make complex scientific data more accessible.

Future research can investigate visualization of:

- Medical datasets
- Climate models
- Geological data

- Astronomical data
- Molecular structures
- Fluid simulations

The objective is to transform complex datasets into interactive visual representations.

11.32 Future Research Direction 7 — Sustainable Graphics

Interactive graphics can consume significant computational resources.

Future research should therefore consider:

VisualQualityvs.EnergyConsumption

Potential research topics include:

- Efficient shaders
- Adaptive resolution
- GPU scheduling
- Asset compression
- Rendering optimization
- Low-power graphics

This is increasingly important as interactive applications move to mobile and edge devices.

11.33 Future Research Direction 8 — Accessibility

Future work should also investigate how interactive 3D systems can become more accessible.

Potential research topics include:

- Alternative interaction methods
- Audio descriptions
- Haptic feedback
- Reduced-motion rendering
- Accessible 3D navigation
- Multimodal interfaces

The goal should be:

AdvancedGraphics+UniversalAccess

rather than treating accessibility as an afterthought.

11.34 Future Research Direction 9 — WebGPU and Browser Computing

WebGPU creates opportunities for increasingly sophisticated GPU workloads in browsers.

Future research could explore:

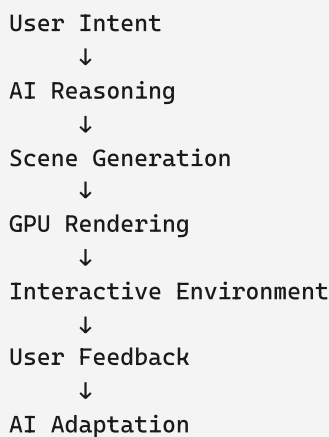
- GPU compute
- Real-time simulation
- Physics
- Machine learning
- Image processing
- Large-scale visualization

This may significantly expand the role of browsers as computational platforms.

11.35 Future Research Direction 10 — Autonomous Interactive Systems

The long-term convergence of AI, graphics, and web technology could result in systems capable of dynamically generating their own interfaces and visual environments.

A conceptual architecture could be:



Such systems represent a possible future direction for interactive computing research.

11.36 Overall Research Contribution

The principal contribution of this thesis is the development of a coherent educational framework for Interactive Graphics and Web Technologies.

The proposed framework connects:

Computer Science+Computer Graphics+Web Engineering+GPU Computing+Interaction+AI
within a single academic pathway.

The contribution is therefore not simply the selection of individual technologies.

It is the **architectural organization of knowledge**.

11.37 Academic Contribution

From an academic perspective, the proposed curriculum provides a structured progression:

Foundation→Graphics→GPU→Interaction→AdvancedSystems→Research

This progression can help students develop both theoretical understanding and practical capability.

11.38 Educational Contribution

From an educational perspective, the proposal emphasizes:

- Active learning
- Project-based learning
- Research-based learning
- Practical experimentation
- Technical documentation
- Portfolio development
- Industry collaboration

The result is an educational model that moves beyond traditional lecture-centered instruction.

11.39 Institutional Contribution

From an institutional perspective, the proposed program provides a potential foundation for:

- New undergraduate education
- Graduate programs
- Research laboratories
- Industry partnerships
- International collaboration
- Technology transfer

The program can therefore function as a foundation for a broader institutional specialization.

11.40 Final Conclusion

The rapid convergence of web technologies, computer graphics, GPU computing, artificial intelligence, and immersive technologies is changing the nature of interactive software.

The traditional distinction between:

WebDevelopment

and:

ComputerGraphics

is becoming increasingly less meaningful.

Modern interactive applications may simultaneously require:

- Front-end engineering
- 3D graphics
- GPU programming
- Animation
- AI
- Interaction design
- Optimization
- Cloud infrastructure

Universities therefore have an opportunity to develop new educational models that reflect this convergence.

This thesis proposed **Interactive Graphics and Web Technologies (IGWT)** as one such model.

The proposed curriculum begins with programming and mathematical foundations and progresses through:

Programming→Web→Graphics→WebGL→3D→Shaders→GPU→Real–TimeRendering→XR→AI→Research

The program is designed around a fundamental principle:

Students should not merely learn how to use today's tools; they should learn the principles necessary to understand and build tomorrow's technologies.

This principle is particularly important in an environment where specific frameworks and platforms can change rapidly.

The university should therefore prioritize enduring knowledge:

Mathematics+Algorithms+Rendering+GPUArchitecture+Interaction+Research

while treating individual frameworks and APIs as replaceable implementations.

The final educational objective is to produce graduates who can move confidently between theory and practice:

Understand→Create→Measure→Optimize→Research

Such graduates can contribute to a wide range of fields, including web engineering, computer graphics, visualization, games, XR, AI applications, simulation, creative technology, and emerging spatial computing systems.

The proposed program therefore represents not simply a new collection of technical courses, but a broader educational philosophy.

It seeks to create a generation of developers, engineers, designers, and researchers capable of treating the browser, GPU, and intelligent computational systems as a unified platform for interactive experiences.

Ultimately, the proposed IGWT program can be summarized by the following model:

Computer Science+Graphics+Web+GPU+Interaction+AI=Interactive Computing

The university that successfully integrates these domains can establish a distinctive academic identity while preparing students for technologies that extend beyond the boundaries of today's conventional software-development disciplines.

11.41 Final Thesis Statement

The central conclusion of this thesis is therefore:

A modern university program in Interactive Graphics and Web Technologies should combine rigorous computer-science foundations with mathematics, real-time computer graphics, web engineering, GPU programming, interactive design, extended reality, and artificial intelligence, while maintaining a continuous cycle of project-based learning, research, assessment, and curriculum improvement.

Such a program has the potential to provide students not only with immediately applicable technical skills, but also with the conceptual foundation required to adapt to future generations of interactive computing.

Final Document Structure

With this Chapter 11, I recommend that the complete thesis now be organized as follows:

Part I — Research and Technical Foundations

Chapter 1 — Introduction

Chapter 2 — Literature Review and Related Technologies

Chapter 3 — Mathematical and Computer Graphics Foundations

Chapter 4 — System Architecture and Design

Chapter 5 — Implementation

Chapter 6 — Experimental Results and Discussion

Chapter 7 — Conclusion and Future Work

Part II — Educational Program Proposal

Chapter 8 — Curriculum Design and Educational Framework

Chapter 9 — Program Evaluation, Quality Assurance, and Continuous Improvement

Chapter 10 — Implementation Roadmap, Institutional Deployment, and Long-Term Development

Chapter 11 — Overall Conclusion, Recommendations, and Future Research

Back Matter

References

Appendix A — Complete Six-Semester Curriculum

Appendix B — Course Specifications

Appendix C — Course Learning Outcomes (CLO) and Program Learning Outcomes (PLO)

Appendix D — Assessment Rubrics

Appendix E — Graduation Thesis Guidelines

Appendix F — Laboratory and Hardware Specifications

Appendix G — Sample Student Projects

Appendix H — Curriculum Mapping Matrix